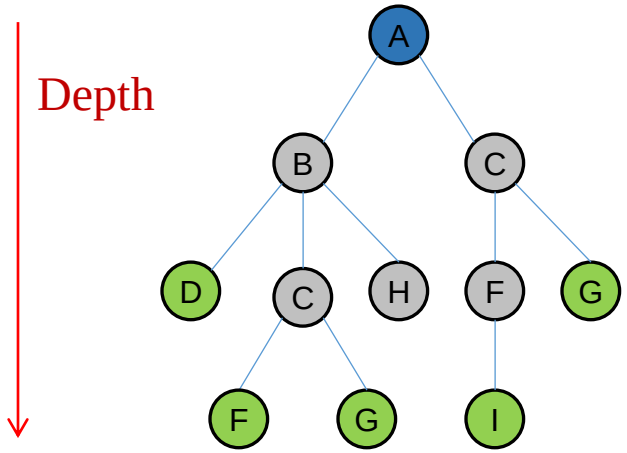
CHAPTER 3 – PART 2
SOLVING PROBLEMS BY SEARCHING

INTRODUCTION TO ARTIFICIAL INTELLIGENT (AI)

REV: 12.03.2024

Infrastructure for Search Algorithms



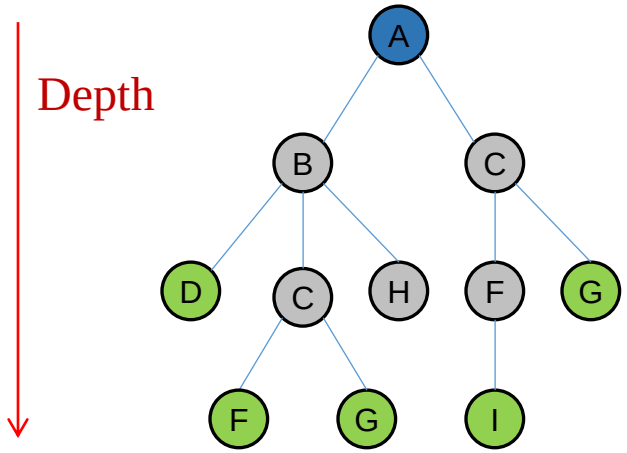
Search algorithms require a data structure to keep track of the search tree. For each node n of the tree, we use a structure that contains four components:

- **n.STATE:** the state in the state space to which the node n corresponds.
- **n.PARENT:** the node in the search tree that generated the node n.
- **n.ACTION:** the action that was applied to the parent to generate the node n.
- **n.PATH-COST:** the path cost from the initial state to the node n. We will use $g(n)$ to denote the path cost.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Infrastructure for Search Algorithms



Given a parent node and an action, a child node can be generated as follows:

```
function CHILD-NODE(problem, parent , action) returns a node
   return a node with
            STATE     = problem.RESULT(parent.STATE, action),
            PARENT = parent,
            ACTION = action,
            PATH-COST = parent.PATH-COST + problem.STEP-COST(parent.STATE, action)
```

Infrastructure for Search Algorithms

The frontier needs to be stored in such a way that the search algorithm can easily choose the next node to expand according to its preferred strategy. The appropriate data structure for this is a queue. The operations on a queue are as follows:

- **EMPTY(queue):** returns true only if there are no elements in the queue.
- **POP(queue):** removes the first element of the queue and returns it.
- **INSERT(element, queue):** inserts an element and returns the resulting queue.

There are three types of queues that we will use:

- **FIFO (first-in, first-out) queue:** pops the oldest element of the queue.
- **LIFO (the last-in, first-out) queue or stack:** pops the newest element of the queue.
- **Priority queue:** pops the element of the queue with the highest priority according to some ordering function.

Infrastructure for Search Algorithms

The explored set can be implemented with a hash table to allow efficient checking for repeated states. By using a hash table, insertion and lookup can be done in roughly constant time, regardless of how many states are stored.

Measuring Problem-Solving Performance

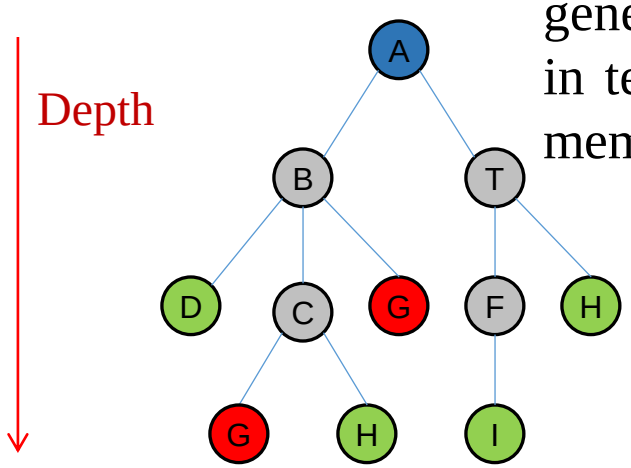
We can evaluate an algorithm's performance in four ways:

- **Completeness:** Is the algorithm guaranteed to find a solution when there is one?
- **Optimality:** Does the strategy find the optimal solution?
- **Time Complexity:** How long does it take to find a solution?
- **Space Complexity:** How much memory is needed to perform the search?

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Measuring Problem-Solving Performance



b = 3
d = 2
m = 3

G The goal state

Time is measured in terms of the number of nodes generated during the search, while space is measured in terms of the maximum number of nodes stored in memory.

In AI, the complexity is expressed in terms of three quantities:

- The branching factor (**b**): maximum number of branches rooted in a node.
- The depth (**d**): the number of steps along the path from the root to the shallowest goal.
- The maximum length of any path (**m**): The maximum length of any path in the state space.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

SUPPLEMENTARY INFORMATION:

ASYMPTOTIC ANALYSIS AND O() (BIG-OH) NOTATION

To compare algorithms with respect to time and memory, benchmark tests can be used, and we can measure the speeds in seconds and memories in bytes. However, the results are dependent on:

- a particular program written in a particular language
- a particular computer
- a particular compiler
- a particular input data

Instead of benchmark tests, asymptotic analysis can be used.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

SUPPLEMENTARY INFORMATION:

ASYMPTOTIC ANALYSIS AND O() (BIG-O) NOTATION

```
function SUMMATION(sequence) returns a number
- sum ← 0
- for i = 1 to LENGTH(sequence) do
      - sum ← sum + sequence[ i ]
- return sum
```

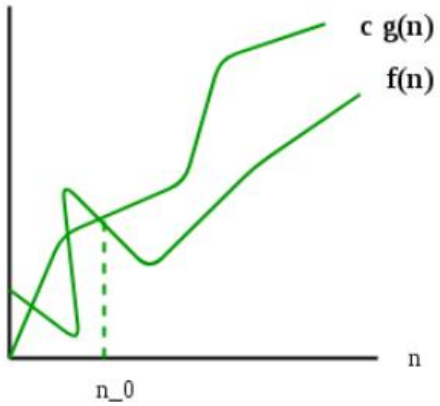
total number of steps performed by the algorithm

$f(n) = 2n + 2$

Definition: **Asymptotic Analysis**

$f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq cg(n)$ for all $n > n_0$.

$O()$ notation represents the upper bound of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm.



f(n) = O(g(n))

$0 \leq f(n) \leq cg(n)$ for all $n > n_0$ [1]

[1] https://www.geeksforgeeks.org/types-of-asymptotic-notations-in-complexity-analysis-of-algorithms/

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

SUPPLEMENTARY INFORMATION:

n and m are inputs, k is a constant.

better (faster)

$f_1(n) = 10, f_2(n) = \log(100), f_3(n) = 10^3$	$O(1)$
$f_1(n) = 3\log(n), f_2(n) = \log(n) + 10$	$O(\log(n))$
$f_1(n) = 4n + 1, f_2(n) = \frac{n}{2} + \log(n)$	$O(n)$
$f_1(n) = 2n\log(n) + 1, f_2(n) = n\log(n) + 7n$	$O(n\log(n))$
$f_1(n) = 3mn + 3n + 2$	$O(nm)$
$f_1(n) = n^2 + 3n, f_2(n) = 3n^2 + n\log(n) + 1$	$O(n^2)$
$f_1(n) = n^3 + 3n, f_2(n) = 3n^3 + \log(n) + 5n^2$	$O(n^3)$
$f_1(n) = n^k + 3n^{k-1} + 3n^3 + \log(n) + 5n^2$	$O(n^k)$
$f_1(n) = 2^n + 3n^5 + \log(n) + 1$	$O(2^n)$
$f_1(n) = k^n + 30n^7 + \log(n) + 1$	$O(3^n)$
$f_1(n) = 5n! + n^2 + 1$	$O(n!)$

polynomial time

same!?

exponential time

worse (slower)

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Total Cost

To assess the effectiveness of a search algorithm, we can consider the search cost or the total cost.

Search Cost ⇒ Usually depends on the time complexity, but also space complexity can be considered.

Path Cost ⇒ The path cost from the initial state to the goal state

$$\boldsymbol{Total\ Cost = Search\ Cost + Path\ Cost}$$

Consider the Romanian map example: Search cost is computed in milliseconds, whereas path cost is computed in terms of km.

$$\boldsymbol{milliseconds + kilometers} = ?$$

It can be converted into millisecond by using the average of the agent's speed.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Search Strategies (Methods)

Uninformed Search (Blind Search)

No additional information about states except as given in the problem definition.

Informed Search (Heuristic Search)

If a “more promising” state exists in the frontier, we can first choose this state (node). In this case, the agent employs a more rational strategy.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Uninformed (Blind) Search Algorithms

- Breadth-First Search Algorithm
- Uniform-Cost Search Algorithm
- Depth-First Search Algorithm
- Depth-limited Search Algorithm
- Iterative Deepening Depth-first Search Algorithm
- Bidirectional Search Algorithm

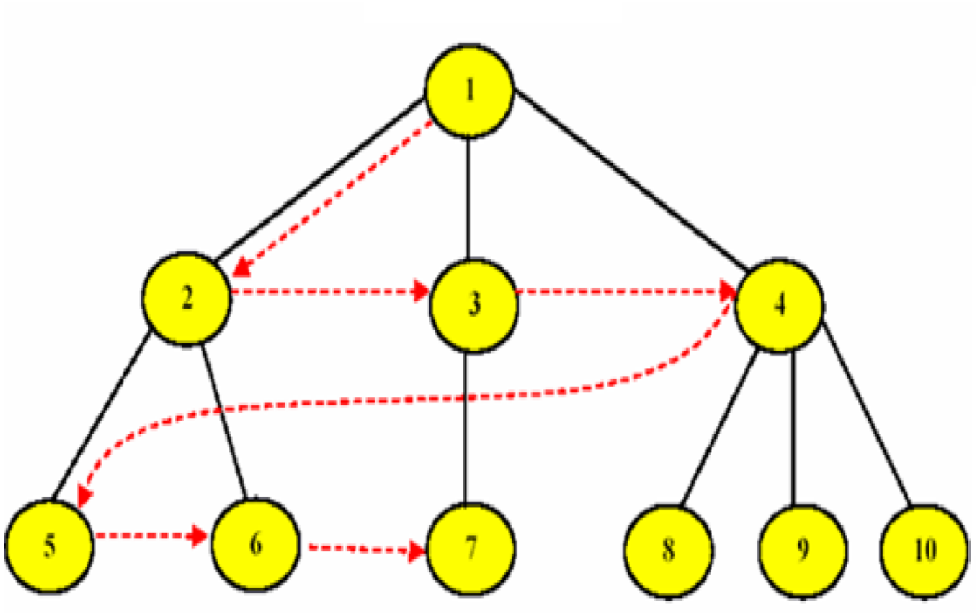
Informed (Heuristic) Search Algorithms

- Greedy Best-First Search Algorithm
- A* Search Algorithm

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

1. Breadth-First Search



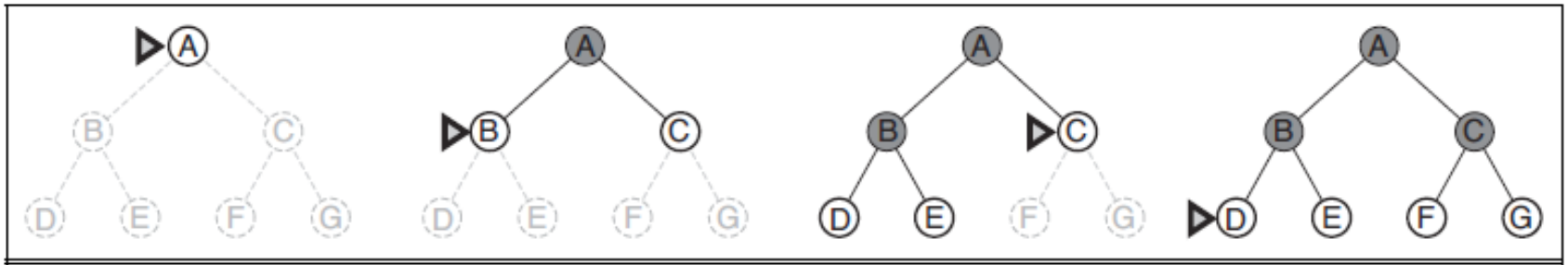
Breadth-first search is a simple strategy in which the root node is expanded first, then all the successors (children) of the root node are expanded next, then their successors, and so on.

We use a FIFO queue when choosing a node in the queue.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

1. Breadth-First Search



Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by a marker.

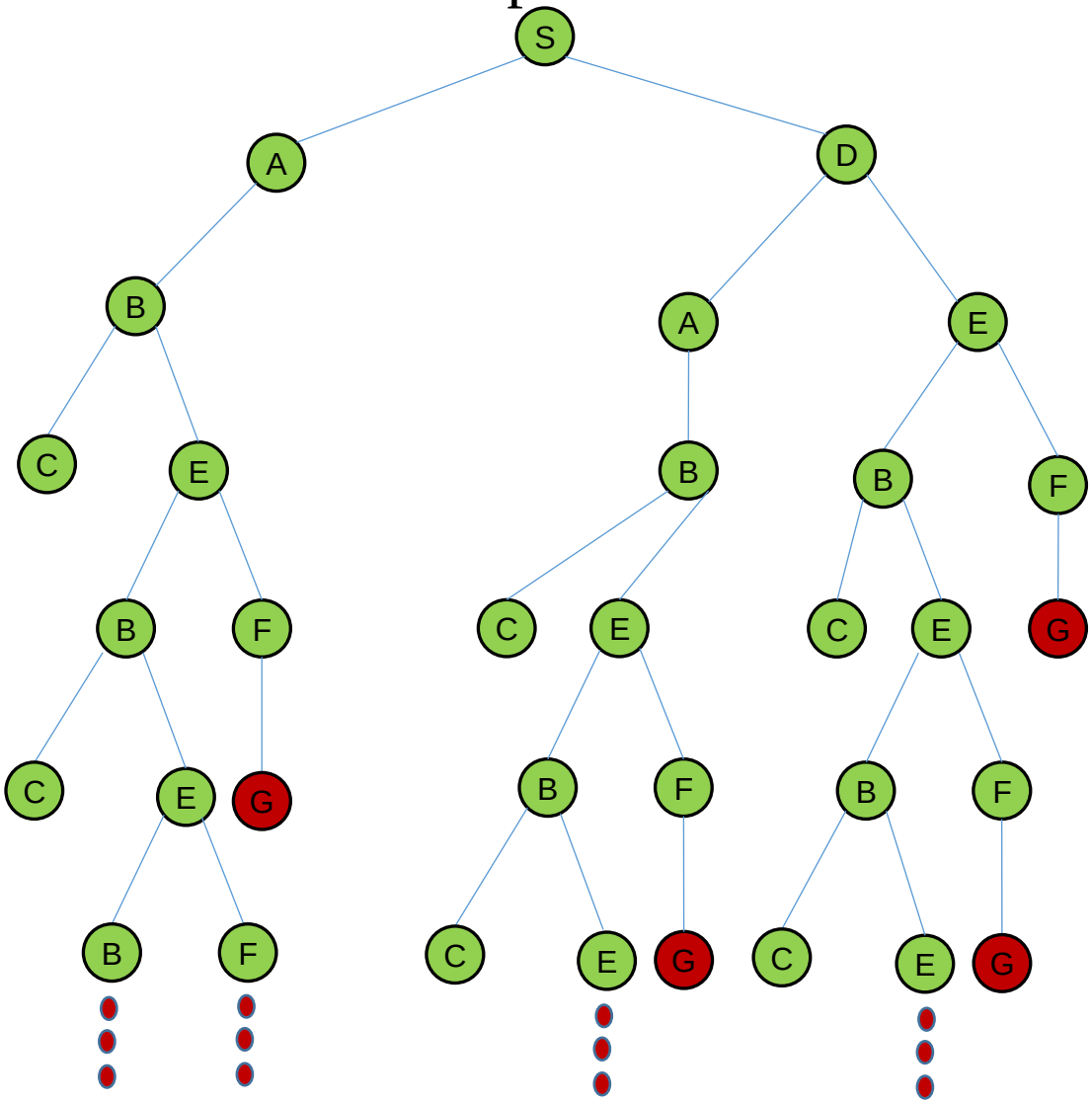
Solving Problems by Searching - 2

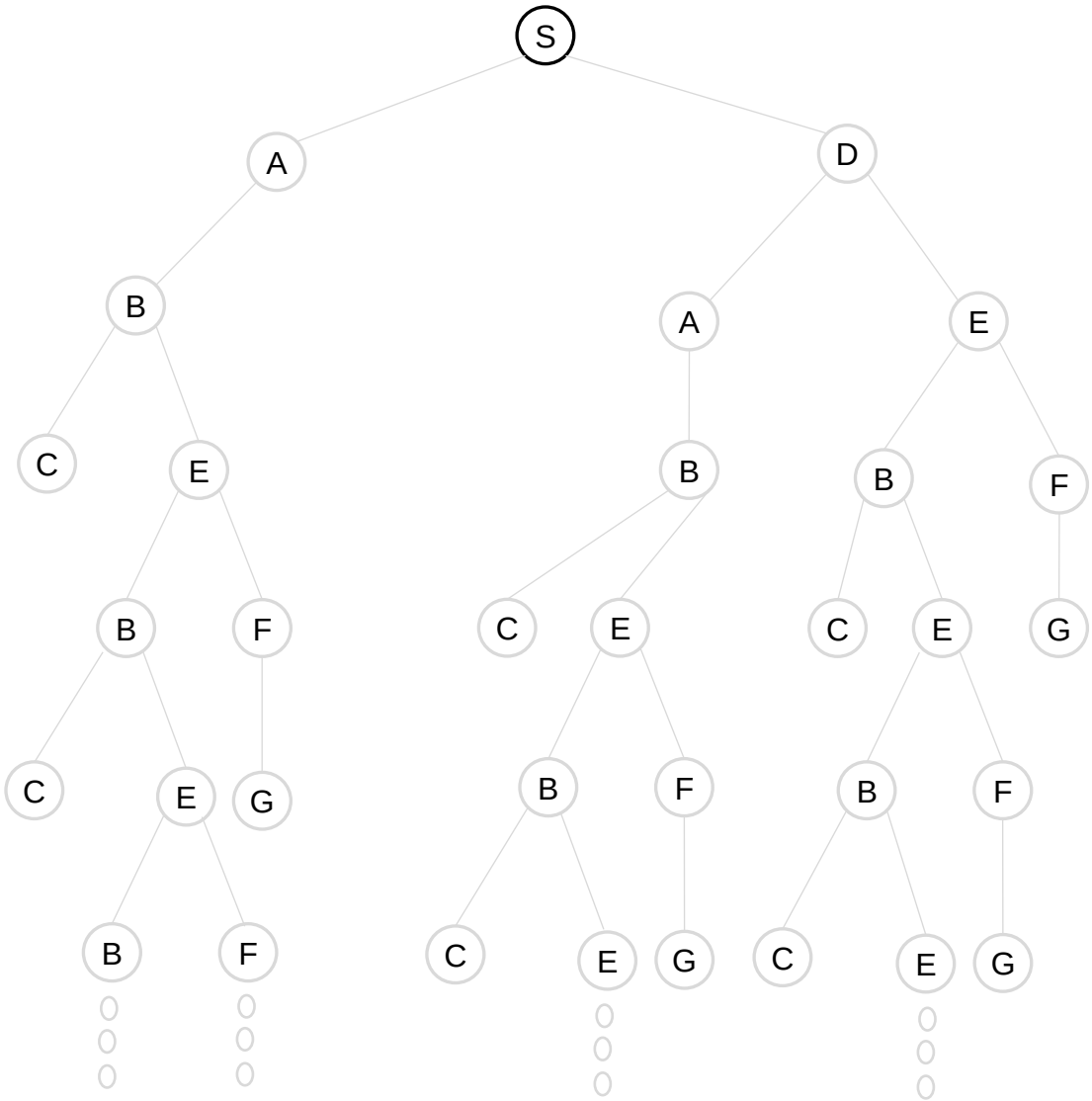
INTRODUCTION TO ARTIFICIAL INTELLIGENT

The tree-search algorithm for breadth-first search is given below.

```
function BREADTH-FIRST-SEARCH-TREE  (problem) returns a solution, or failure
   - node ← a node with STATE = problem.INITIAL-STATE(),
                                PATH-COST = 0
   - if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
   - frontier   ← a FIFO queue with the root node
   - loop do
      - if EMPTY( frontier) then return failure
      - node←POP( frontier )  /* chooses the shallowest node in frontier */
      - for each action in problem.ACTIONS(node.STATE) do
            - child ←CHILD-NODE(problem, node, action)
            - if problem.GOAL-TEST(child .STATE) then return SOLUTION(child)
            - frontier ←INSERT(child , frontier )
```

Example: Determine a path from the initial state to the goal state by using the tree search of the breadth-first algorithm. Show the frontier for each step.

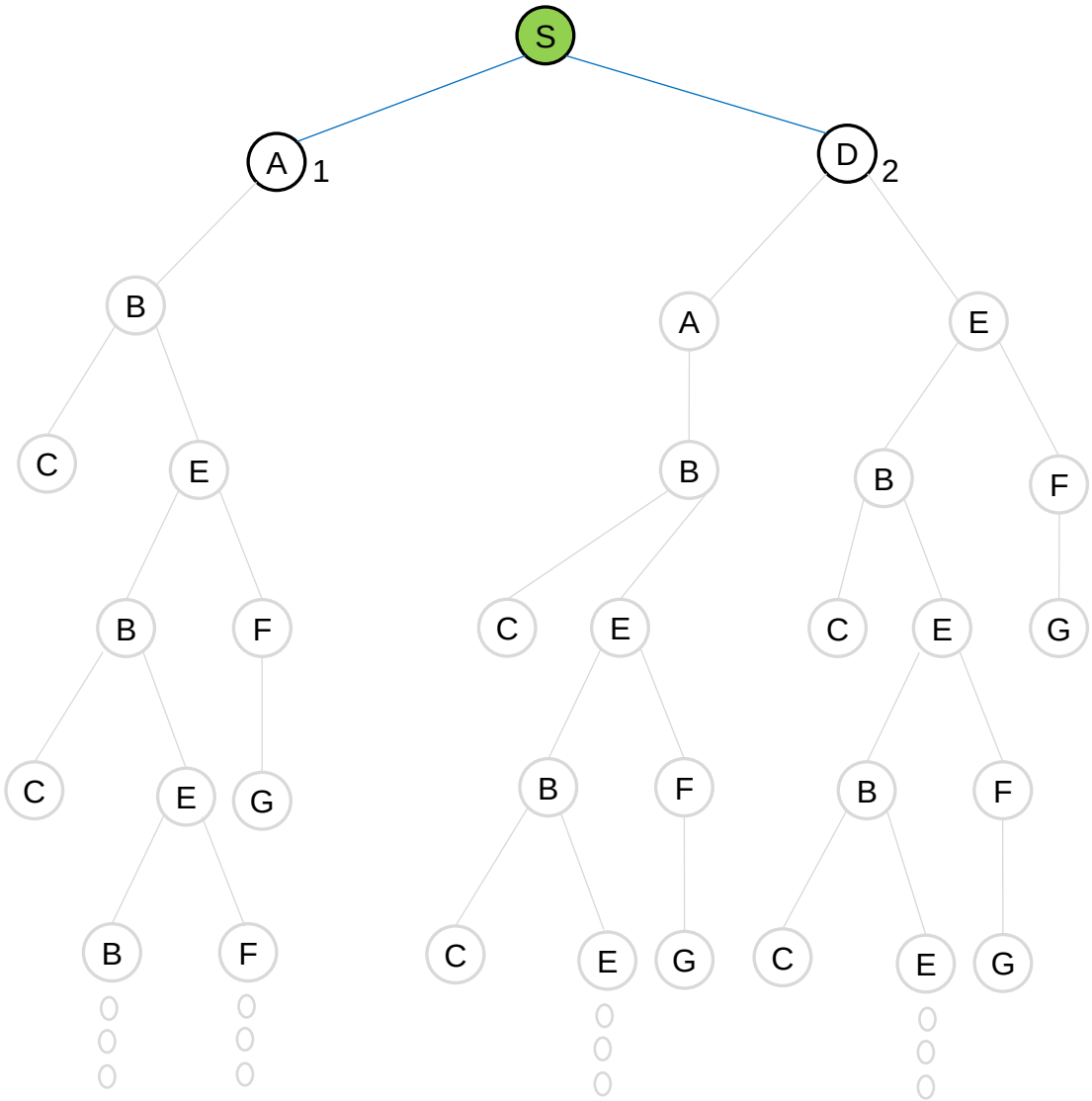




[**S**]

FRONTIER (FIFO)

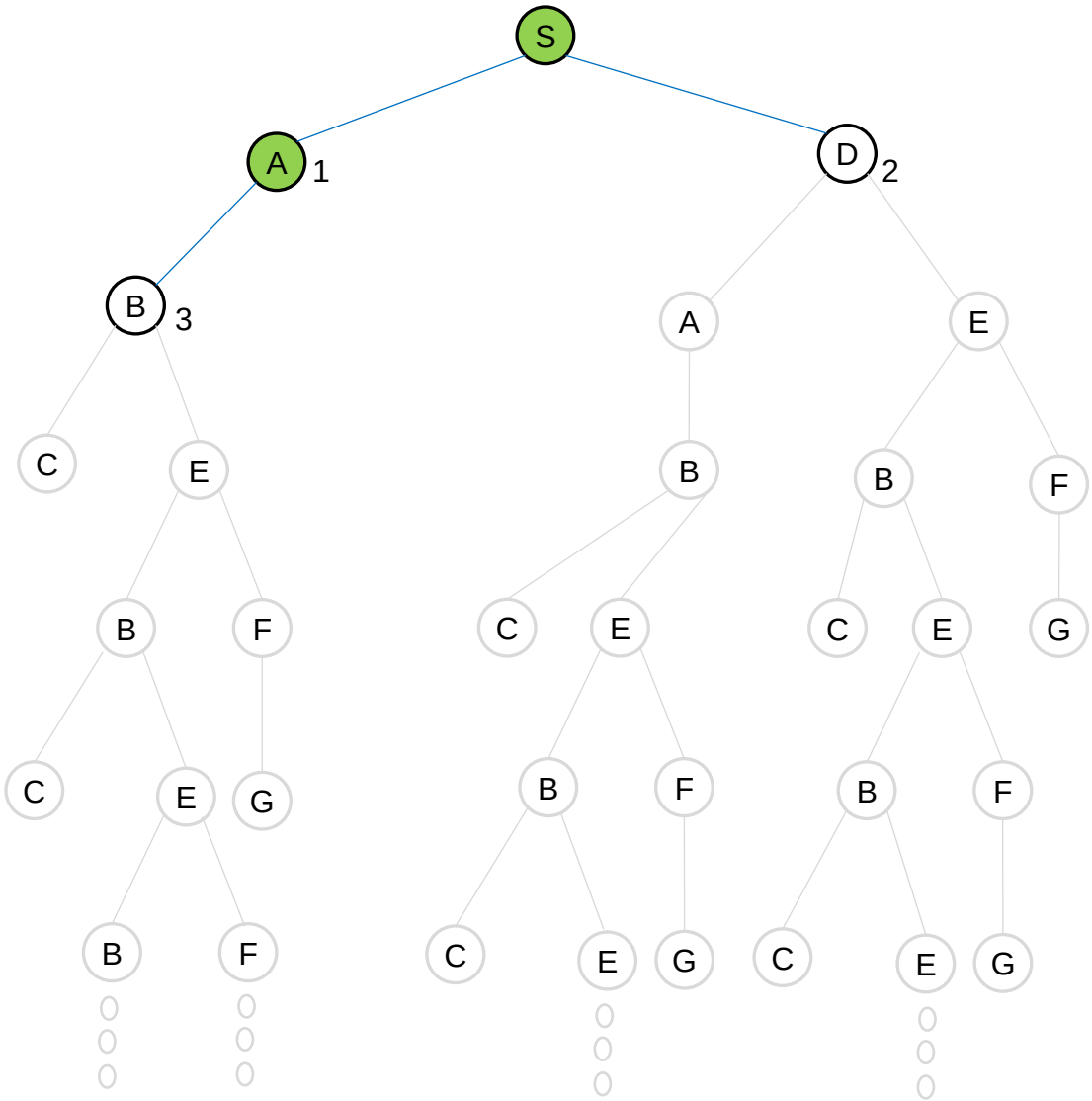
inserts(**S**)



FRONTIER (FIFO)

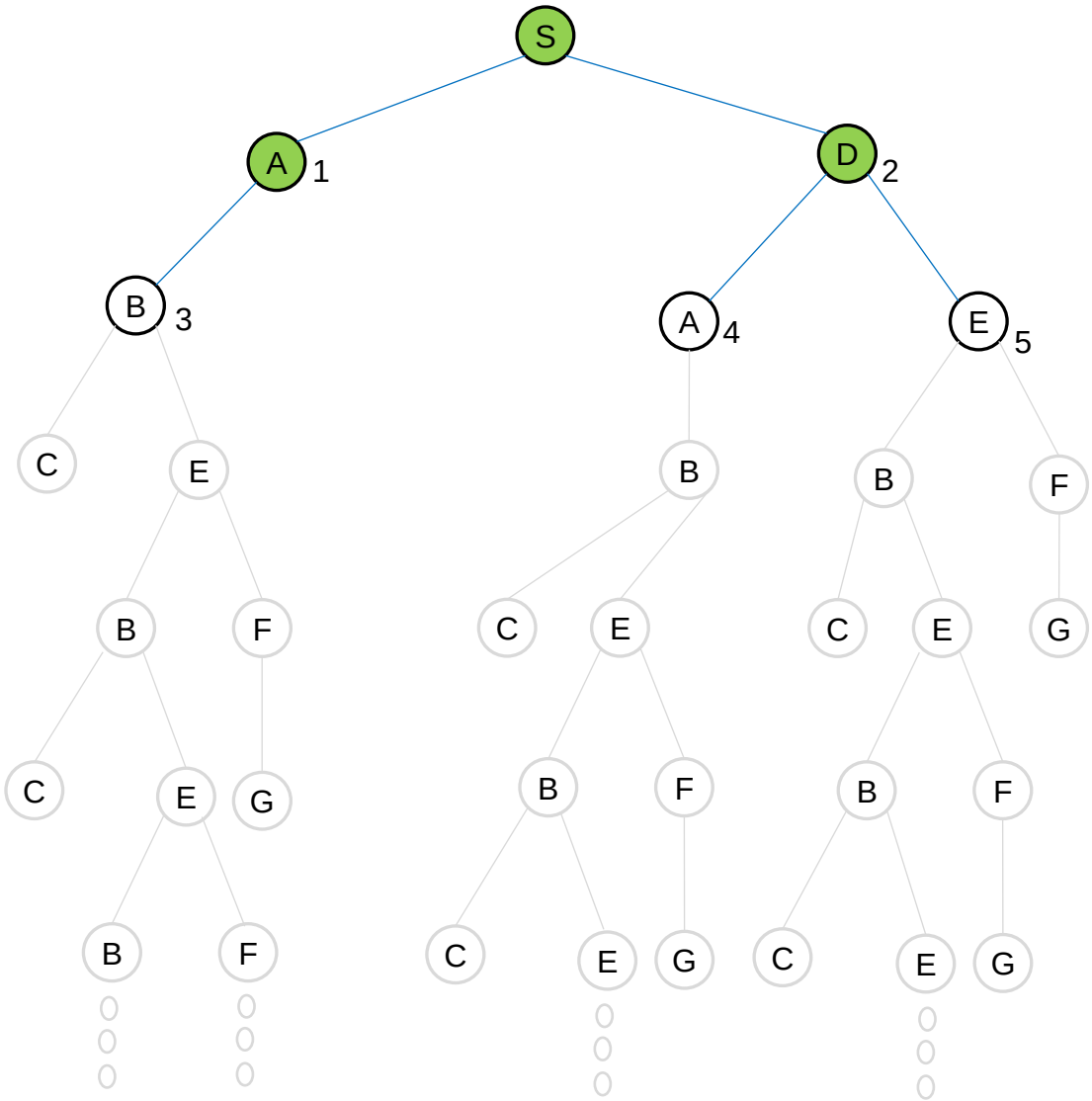
[**S**] inserts(**S**)

[**A_1**,D_2] pop(S), inserts(A_1,D_2)



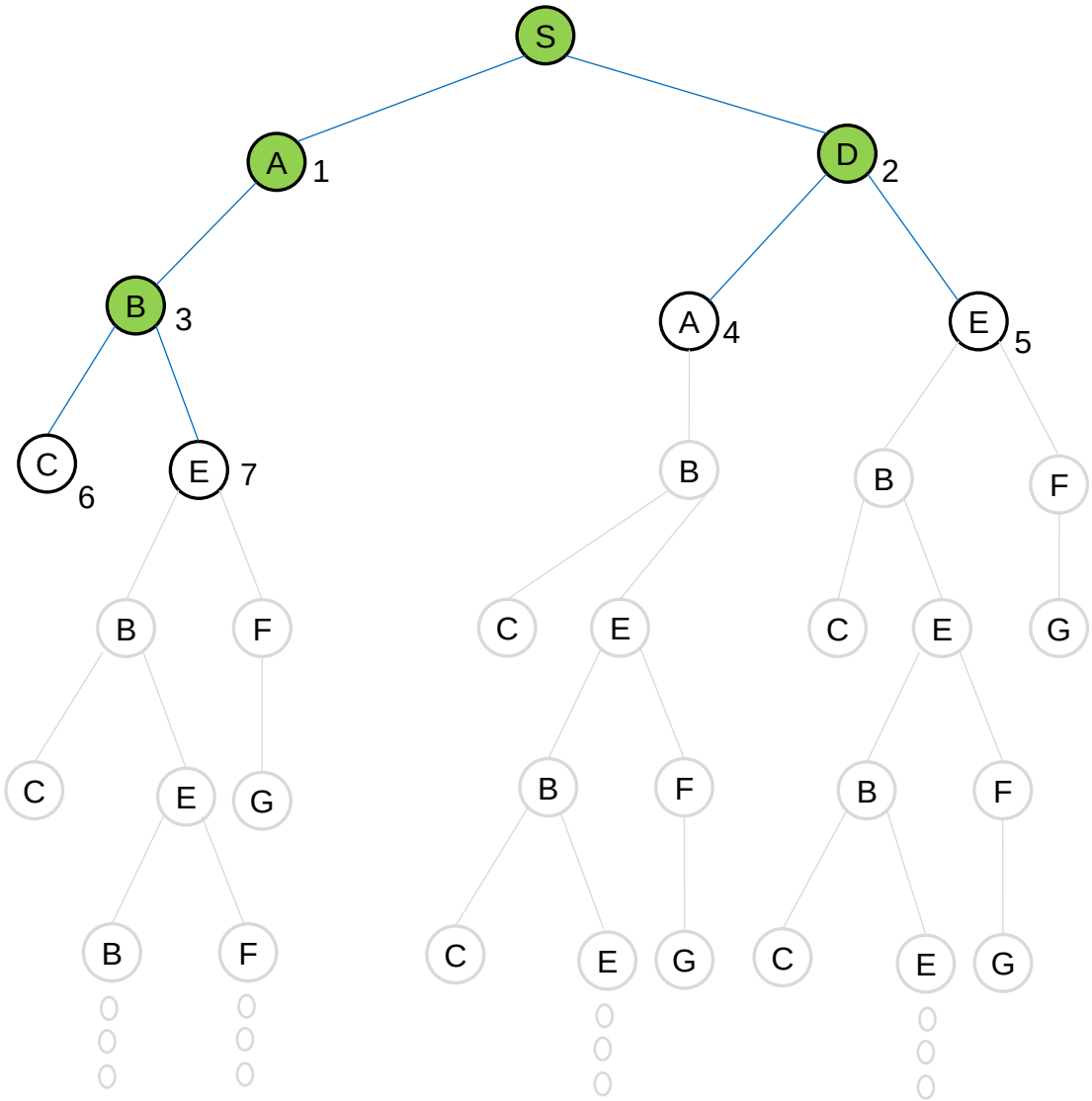
FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A_1**,D_2]	pop(S), inserts(A_1,D_2)
[**D_2**,B_3]	pop(A_1), inserts(B_3)



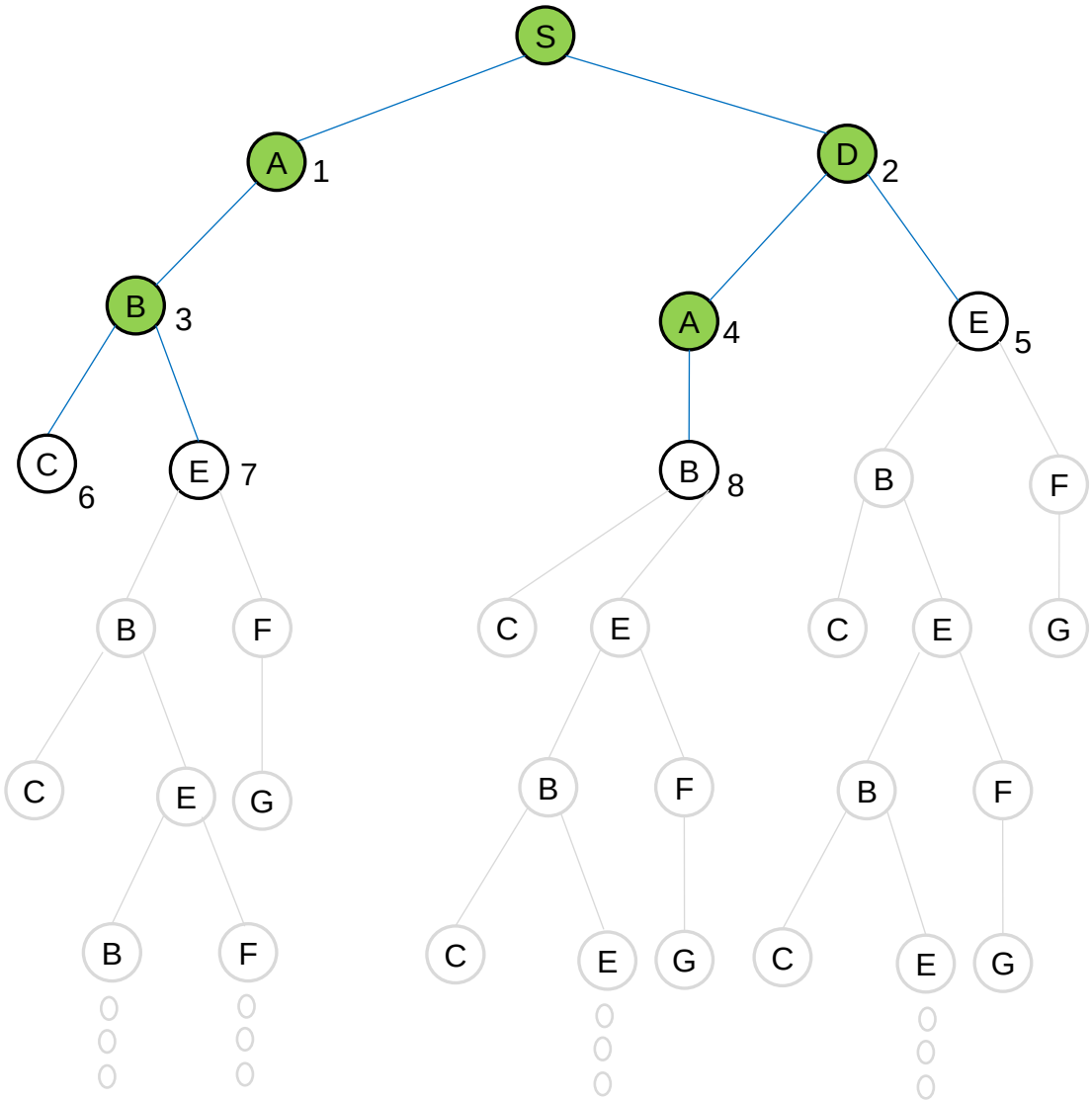
FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A_1**,D_2]	pop(S), inserts(A_1,D_2)
[**D_2**,B_3]	pop(A_1), inserts(B_3)
[**B_3**, A_4,E_5]	pop(D_2), inserts(A_4,E_5)



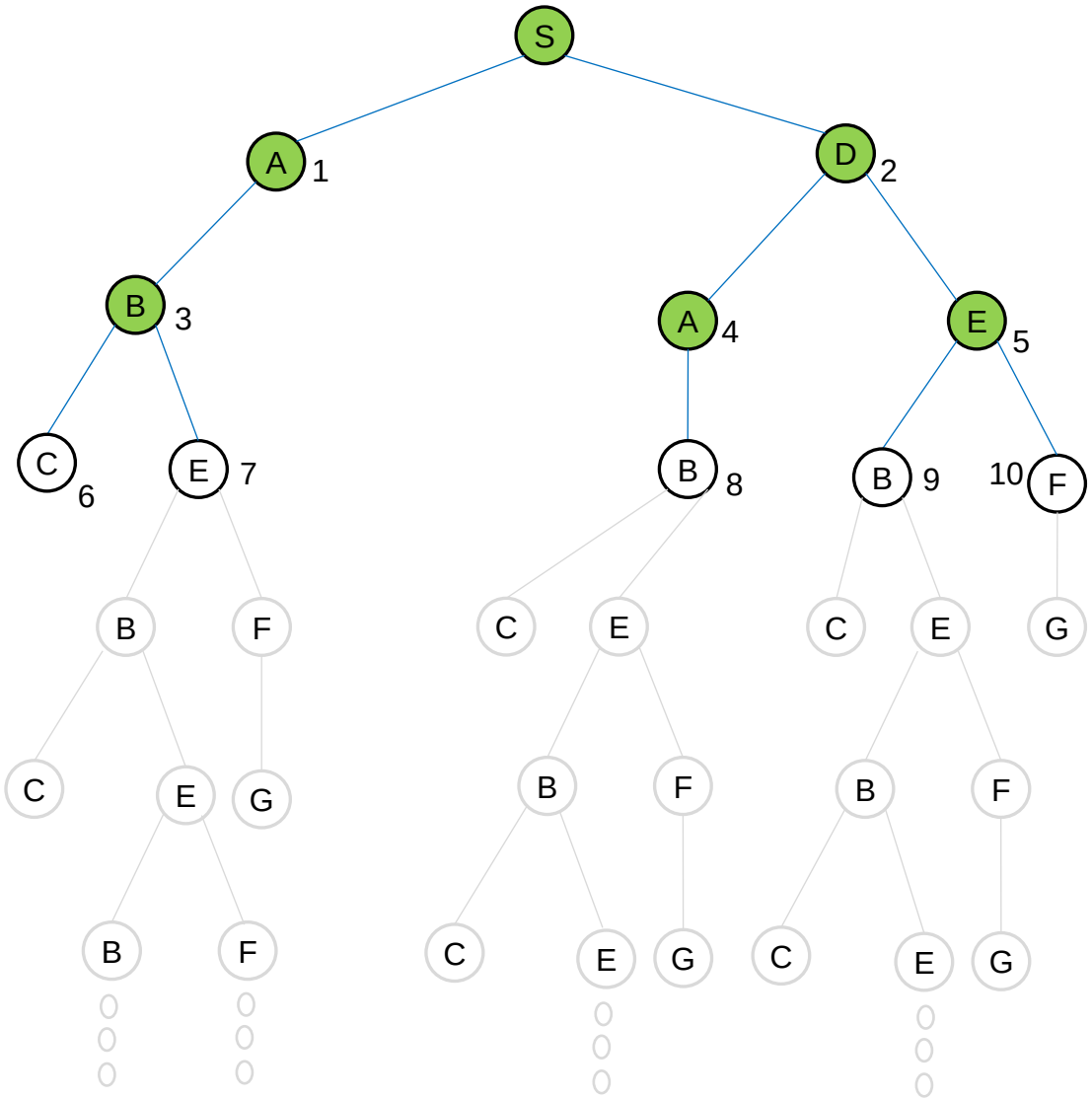
FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A_1**,D_2]	pop(S), inserts(A_1,D_2)
[**D_2**,B_3]	pop(A_1), inserts(B_3)
[**B_3**, A_4,E_5]	pop(D_2), inserts(A_4,E_5)
[**A_4**, E_5, C_6, E_7]	pop(B_3), inserts(C_6,E_7)



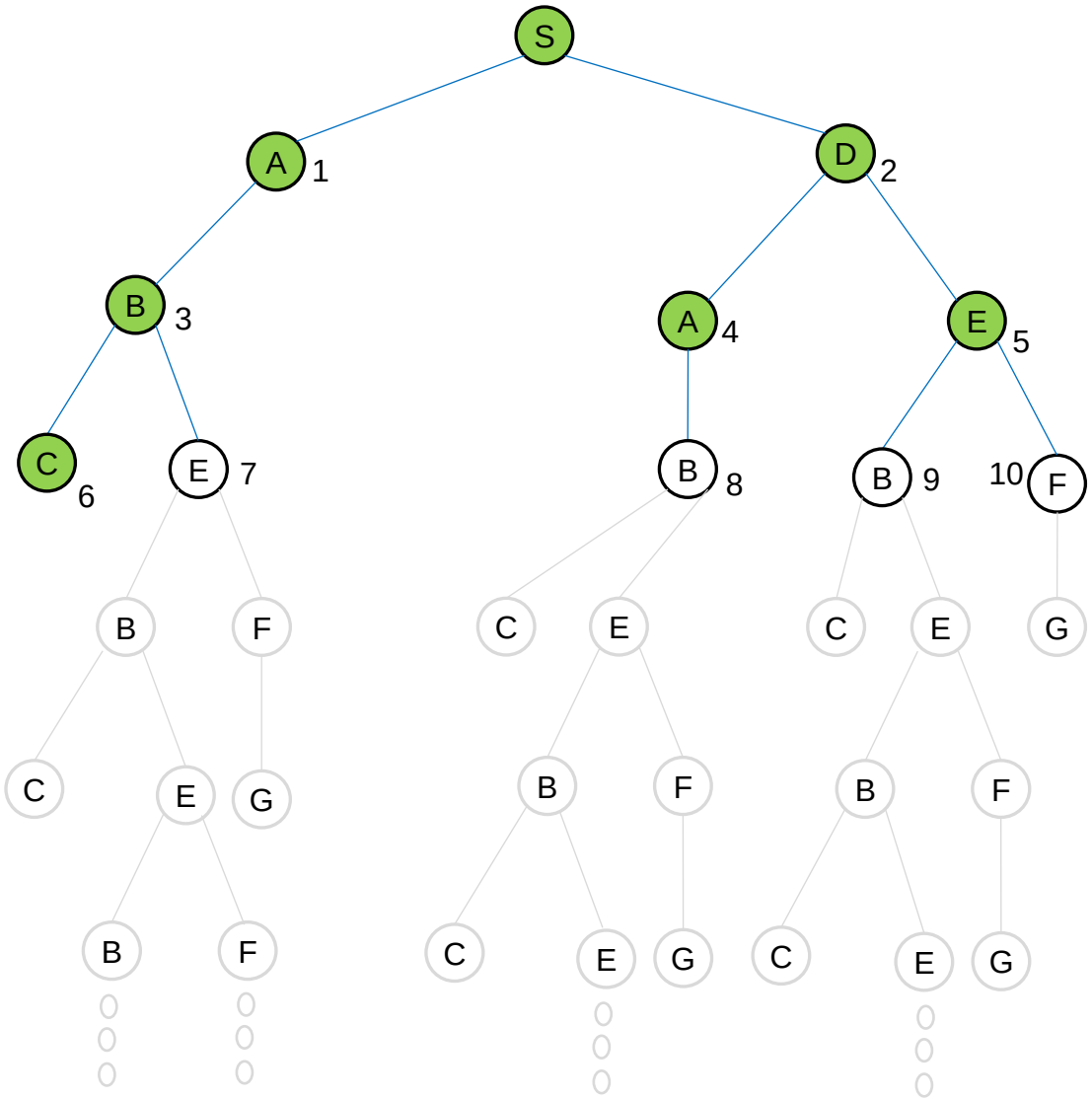
FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A_1**,D_2]	pop(S), inserts(A_1,D_2)
[**D_2**,B_3]	pop(A_1), inserts(B_3)
[**B_3**, A_4,E_5]	pop(D_2), inserts(A_4,E_5)
[**A_4**, E_5, C_6, E_7]	pop(B_3), inserts(C_6,E_7)
[**E_5**, C_6, E_7, B_8]	.
	.
	.



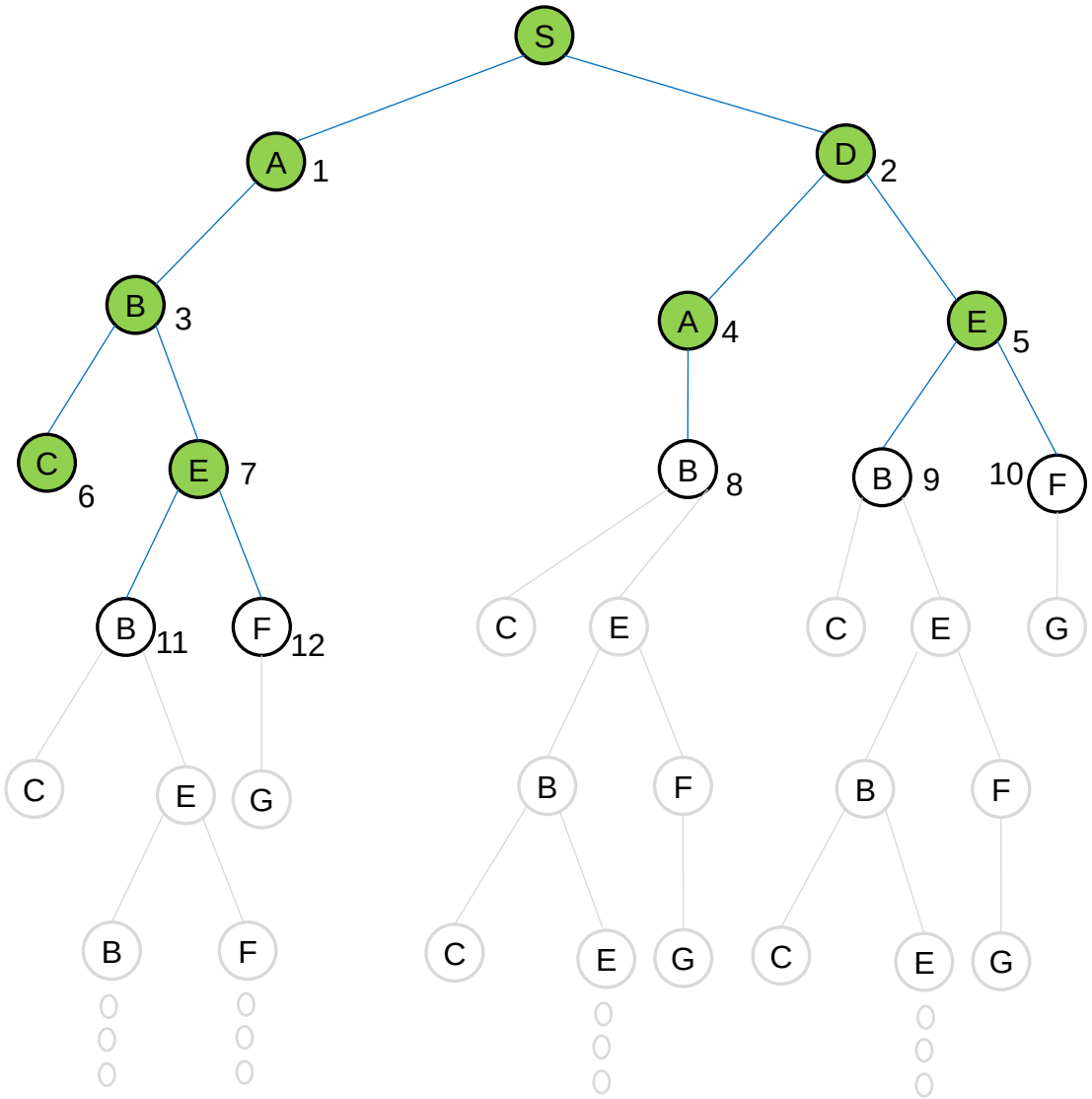
FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A_1**,D_2]	pop(S), inserts(A_1,D_2)
[**D_2**,B_3]	pop(A_1), inserts(B_3)
[**B_3**, A_4,E_5]	pop(D_2), inserts(A_4,E_5)
[**A_4**, E_5, C_6, E_7]	pop(B_3), inserts(C_6,E_7)
[**E_5**, C_6, E_7, B_8]	.
[**C_6**, E_7, B_8 , B_9, F_{10}]	.
	.



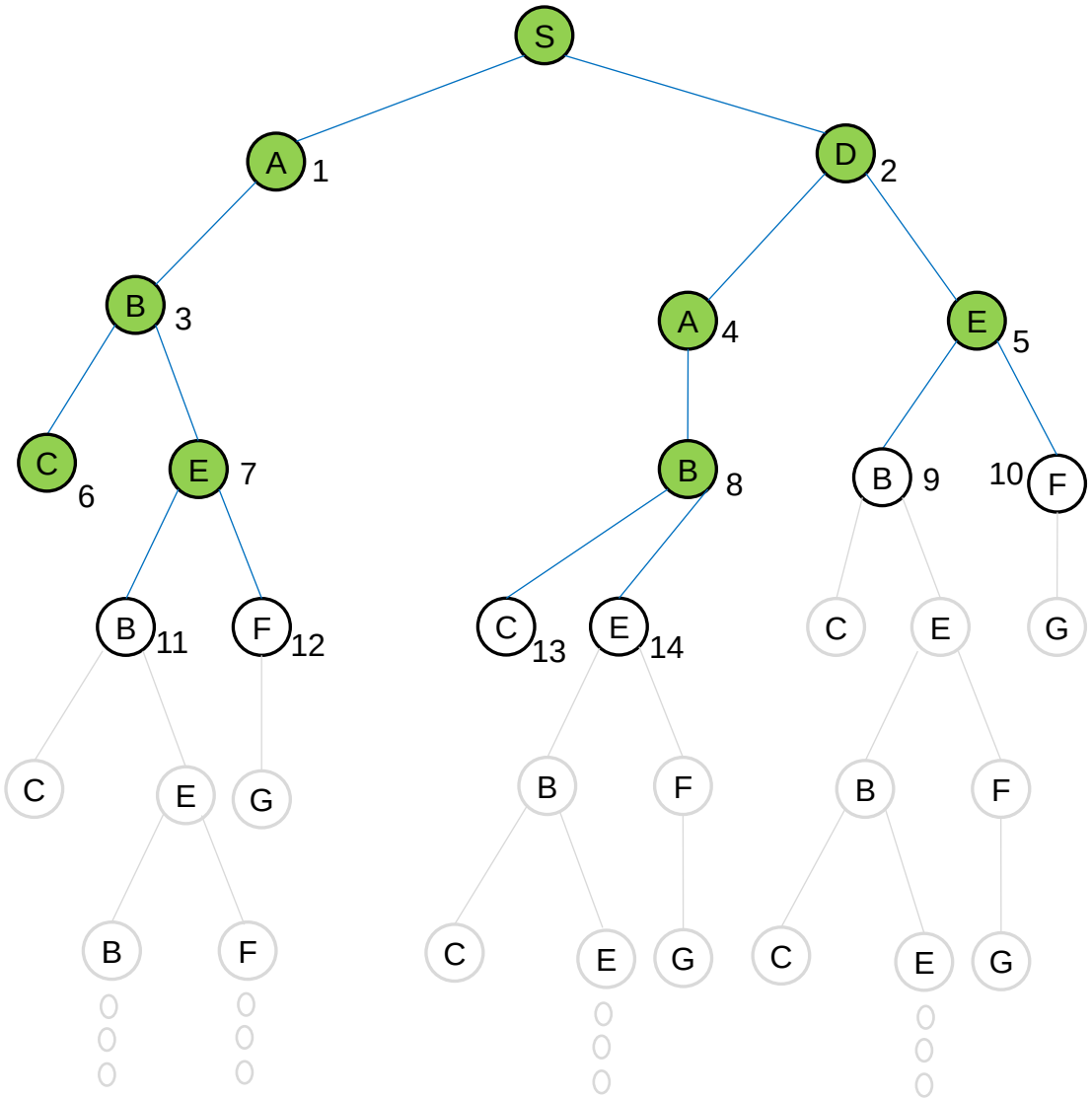
FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A_1**,D_2]	pop(S), inserts(A_1,D_2)
[**D_2**,B_3]	pop(A_1), inserts(B_3)
[**B_3**, A_4,E_5]	pop(D_2), inserts(A_4,E_5)
[**A_4**, E_5, C_6, E_7]	pop(B_3), inserts(C_6,E_7)
[**E_5**, C_6, E_7, B_8]	.
[**C_6**, E_7, B_8 , B_9, F_{10}]	.
[**E_7**, B_8, B_9, F_{10}]	.



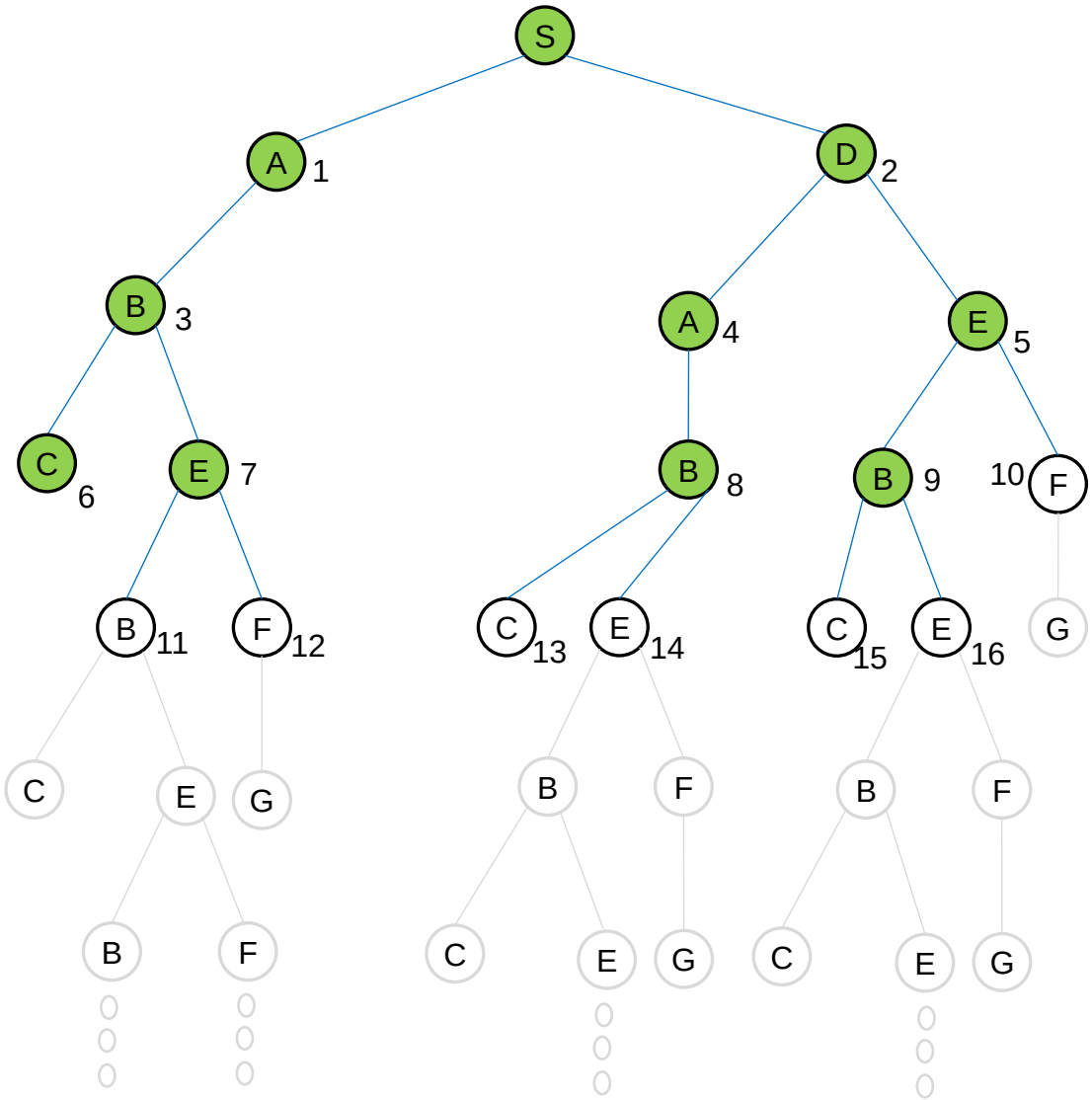
FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A_1**,D_2]	pop(S), inserts(A_1,D_2)
[**D_2**,B_3]	pop(A_1), inserts(B_3)
[**B_3**, A_4,E_5]	pop(D_2), inserts(A_4,E_5)
[**A_4**, E_5, C_6, E_7]	pop(B_3), inserts(C_6,E_7)
[**E_5**, C_6, E_7, B_8]	.
[**C_6**, E_7, B_8 , B_9, F_{10}]	.
[**E_7**, B_8, B_9, F_{10}]	.
[**B_8**, B_9, F_{10}, B_{11}, F_{12}]	



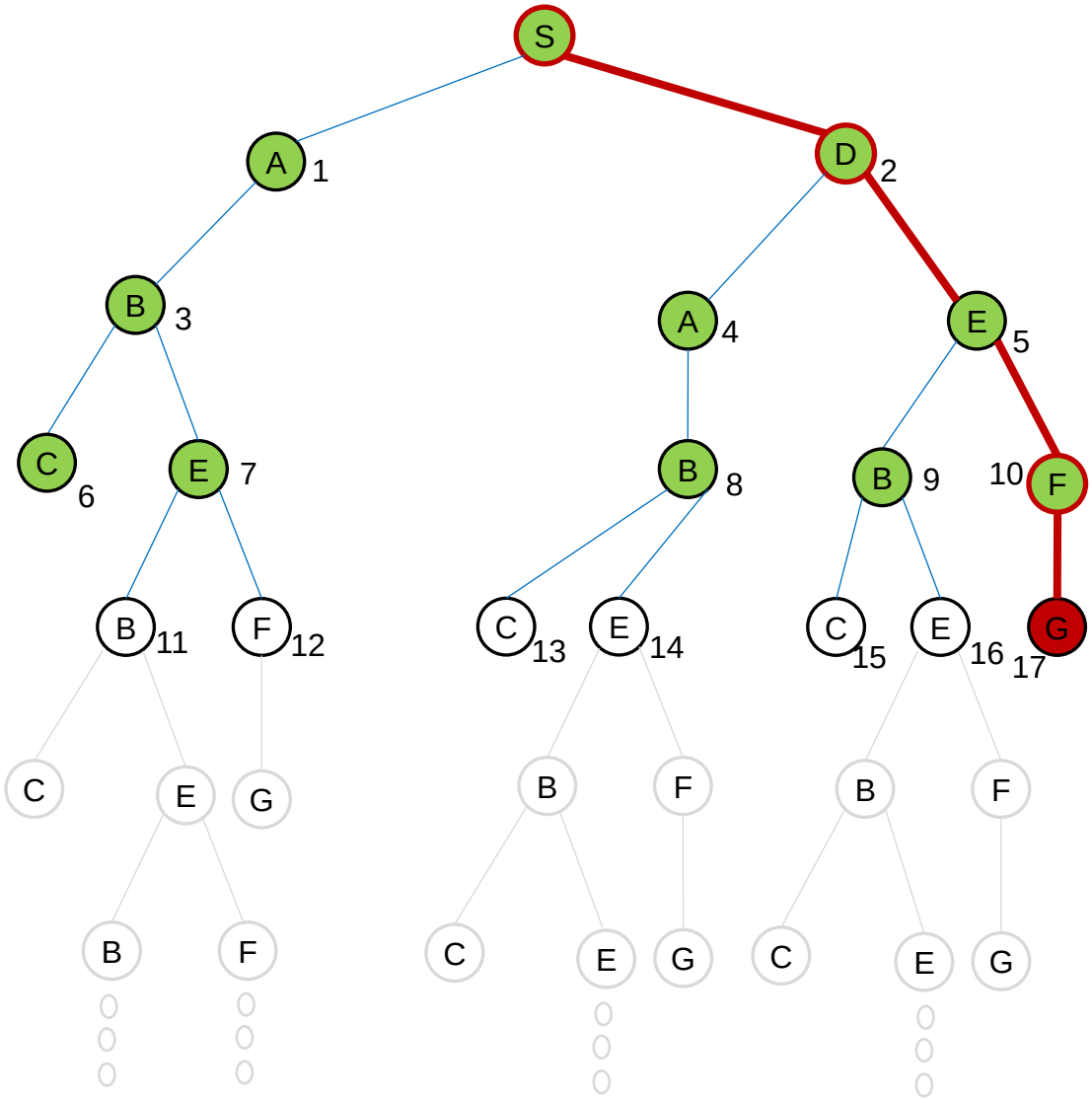
FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A_1**,D_2]	pop(S), inserts(A_1,D_2)
[**D_2**,B_3]	pop(A_1), inserts(B_3)
[**B_3**, A_4,E_5]	pop(D_2), inserts(A_4,E_5)
[**A_4**, E_5, C_6, E_7]	pop(B_3), inserts(C_6,E_7)
[**E_5**, C_6, E_7, B_8]	.
[**C_6**, E_7, B_8 , B_9, F_{10}]	.
[**E_7**, B_8, B_9, F_{10}]	.
[**B_8**, B_9, F_{10}, B_{11}, F_{12}]	
[**B_9**, F_{10}, B_{11}, F_{12}, C_{13}, E_{14}]	



FRONTIER (FIFO)

[**S**] inserts(**S**)
[$\mathbf{A_1}$,D_2] pop(S), inserts(A_1,D_2)
[$\mathbf{D_2}$,B_3] pop(A_1), inserts(B_3)
[$\mathbf{B_3}$, A_4,E_5] pop(D_2), inserts(A_4,E_5)
[$\mathbf{A_4}$, E_5, C_6, E_7] pop(B_3), inserts(C_6,E_7)
[$\mathbf{E_5}$, C_6, E_7, B_8] .
[$\mathbf{C_6}$, E_7, B_8 , B_9, F_{10}] .
[$\mathbf{E_7}$, B_8, B_9, F_{10}] .
[$\mathbf{B_8}$, B_9, F_{10}, B_{11}, F_{12}]
[$\mathbf{B_9}$, F_{10}, B_{11}, F_{12}, C_{13}, E_{14}]
[$\mathbf{F_{10}}$, B_{11}, F_{12}, C_{13}, E_{14}, C_{15}, E_{16}]



FRONTIER (FIFO)

[**S**]	inserts(**S**)
[A_1,D_2]	pop(S), inserts(A_1,D_2)
[D_2,B_3]	pop(A_1), inserts(B_3)
[B_3, A_4,E_5]	pop(D_2), inserts(A_4,E_5)
[A_4, E_5, C_6, E_7]	pop(B_3), inserts(C_6,E_7)
[E_5, C_6, E_7, B_8]	.
[C_6, E_7, B_8 , B_9, F_{10}]	.
[E_7, B_8, B_9, F_{10}]	.
[B_8, B_9, F_{10}, B_{11}, F_{12}]	
[B_9, F_{10}, B_{11}, F_{12}, C_{13}, E_{14}]	
[F_{10}, B_{11}, F_{12}, C_{13}, E_{14}, C_{15}, E_{16}]	
[B_{11}, F_{12}, C_{13}, E_{14}, C_{15}, E_{16}]	

BFS = {S,D,E,F,G}

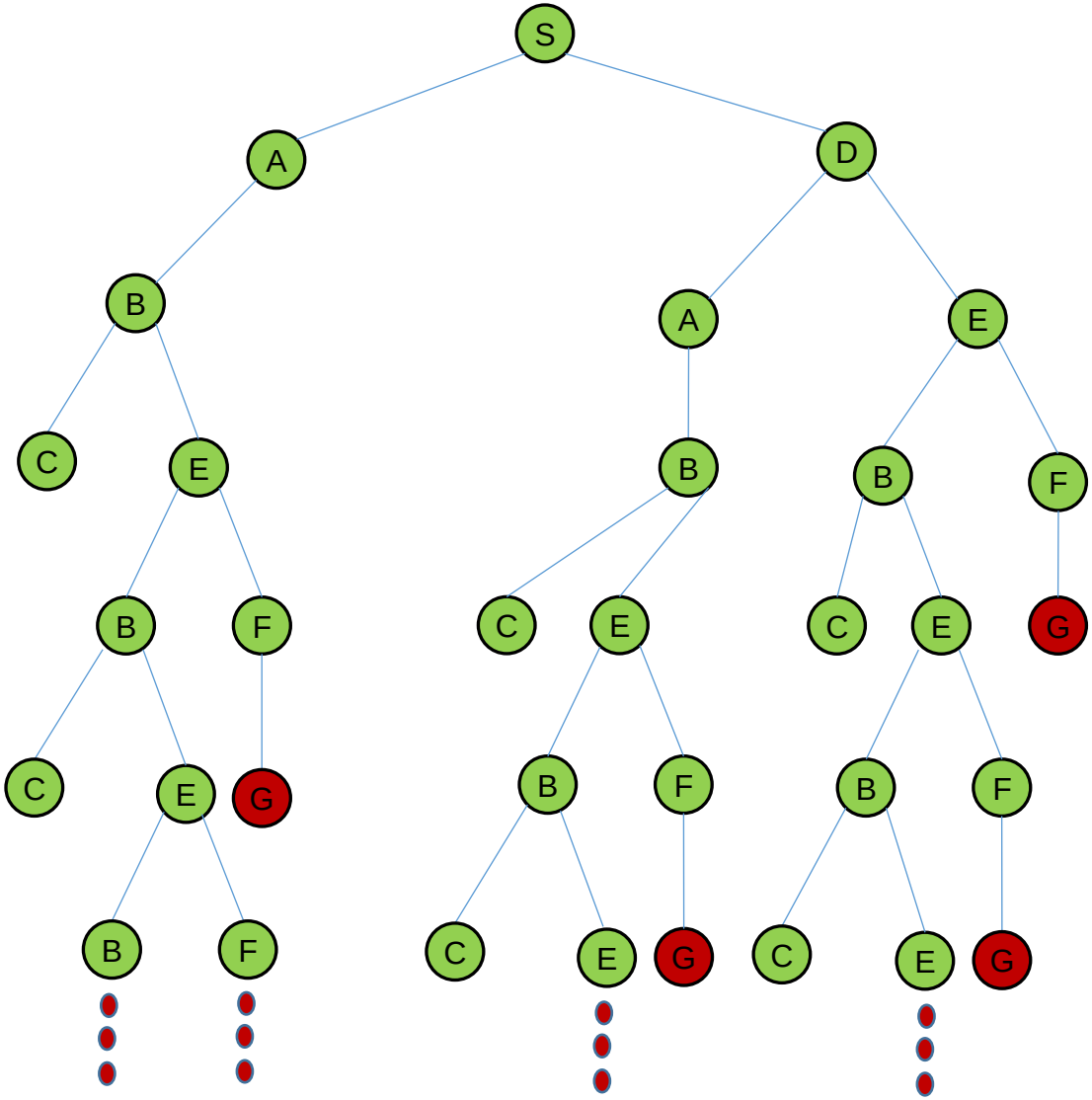
Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The graph-search algorithm for breadth-first search is given below.

```
function BREADTH-FIRST-SEARCH-GRAPH  (problem) returns a solution, or failure
   - node ← a node with STATE = problem.INITIAL-STATE(),
                                PATH-COST = 0
   - if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
   - frontier   ← a FIFO queue with the root node
   - explored ← an empty set
   - loop do
      - if EMPTY( frontier) then return failure
      - node←POP( frontier )  /* chooses the shallowest node in frontier */
      - add node.STATE to explored
      - for each action in problem.ACTIONS(node.STATE) do
            - child ←CHILD-NODE(problem, node, action)
            - if child .STATE is not in explored set or frontier then
                  - if problem.GOAL-TEST(child .STATE) then return SOLUTION(child )
                  - frontier ←INSERT(child , frontier )
```

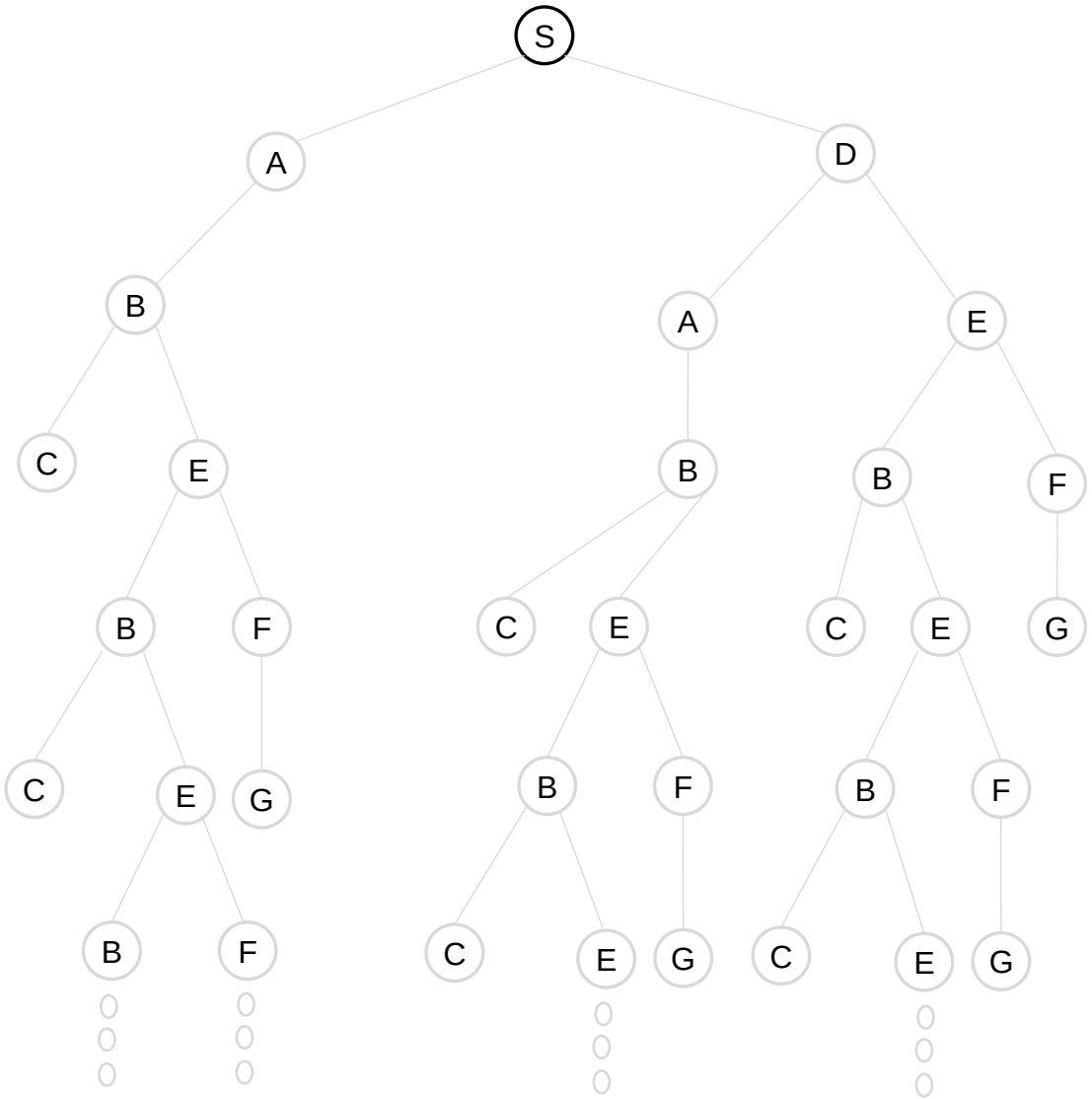
Example: Determine a path from the initial state to the goal state by using the graph search of the breadth-first algorithm. Show the frontier and the explored set for each step.



EXPLORED SET:

[**S**]

FRONTIER (FIFO)
inserts(**S**)

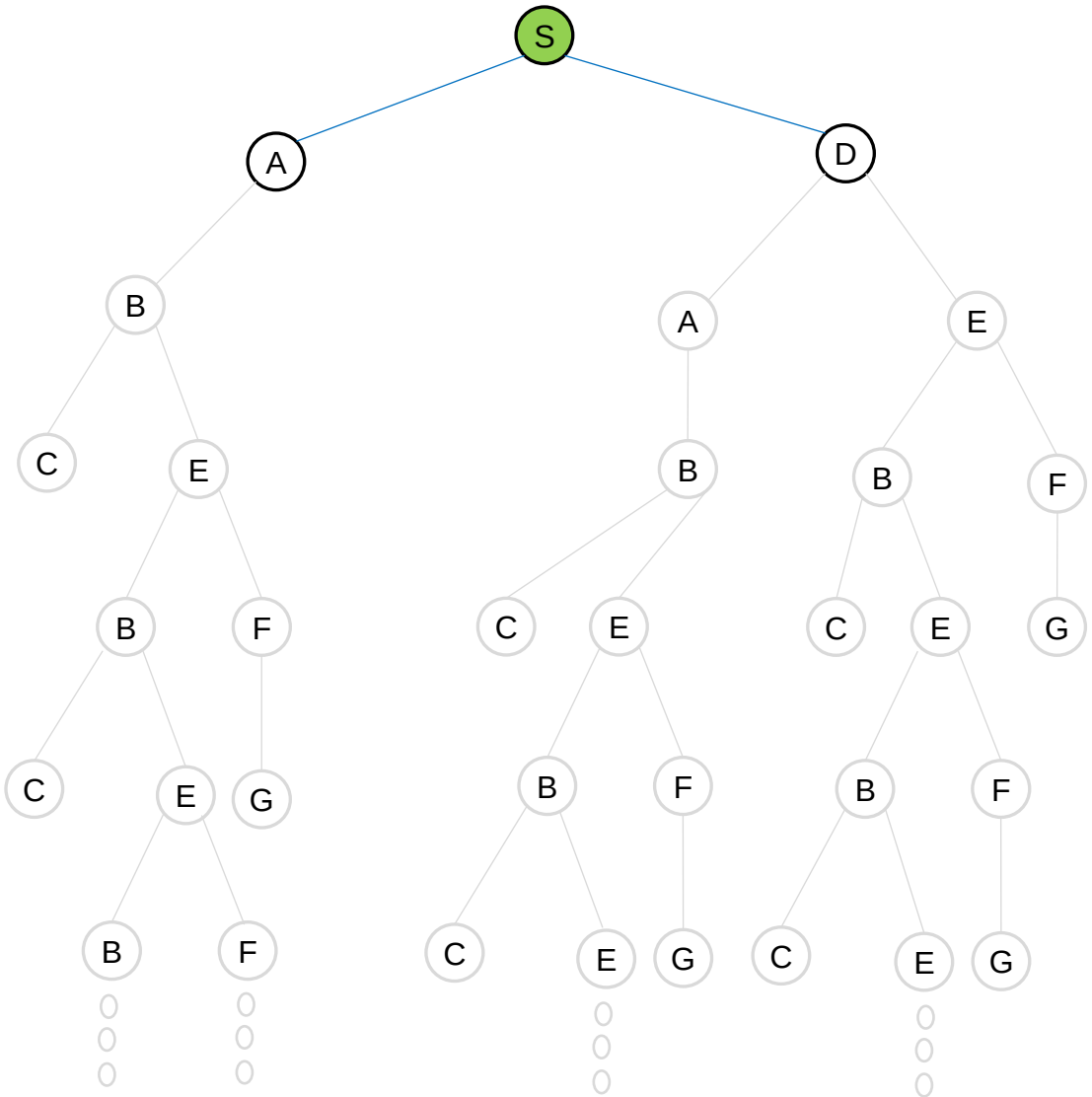


EXPLORED SET: S,

FRONTIER (FIFO)

[S] inserts(**S**)

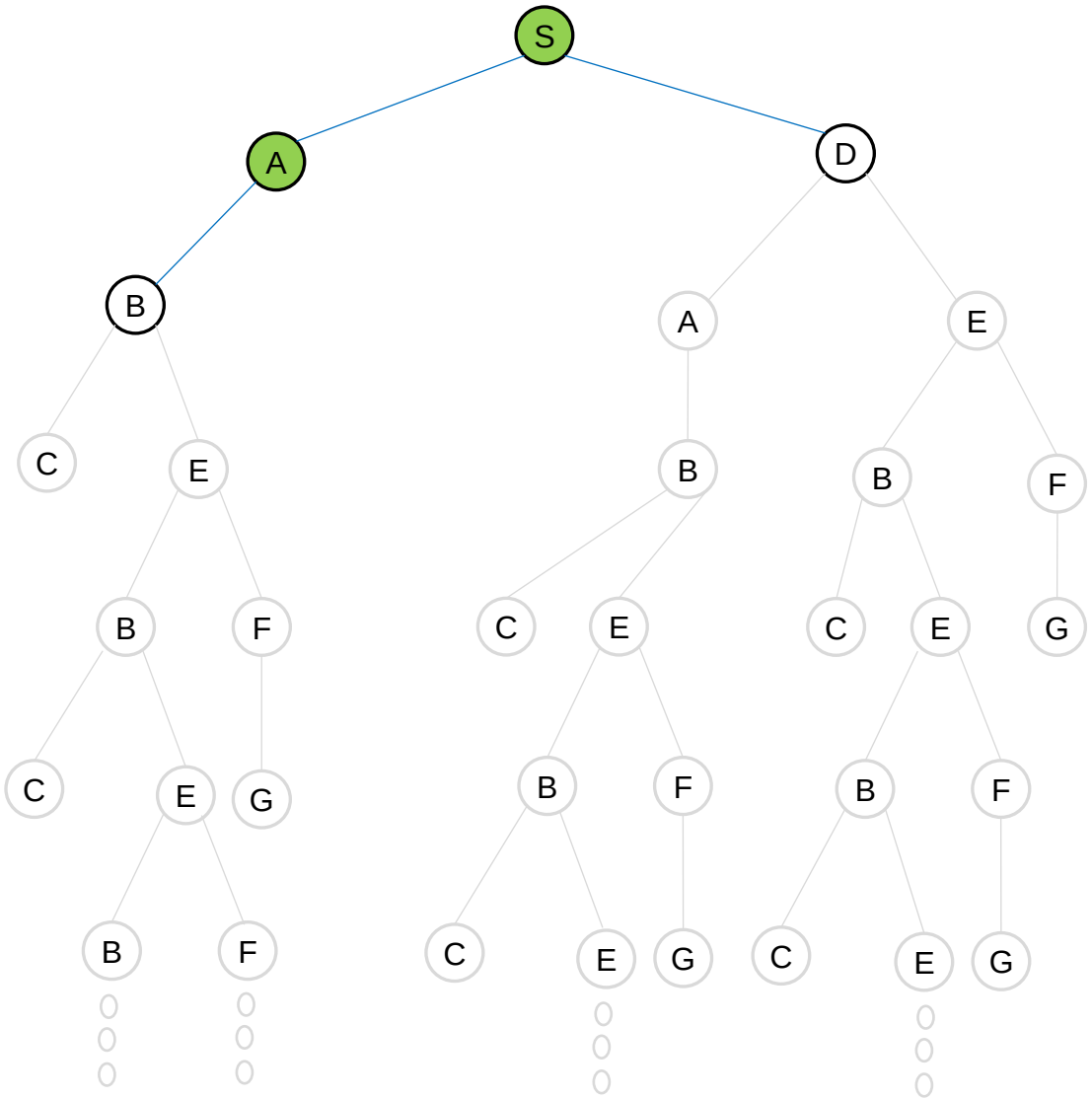
[A,D] pop(S), inserts(A,D)



EXPLORED SET: S, A,

FRONTIER (FIFO)

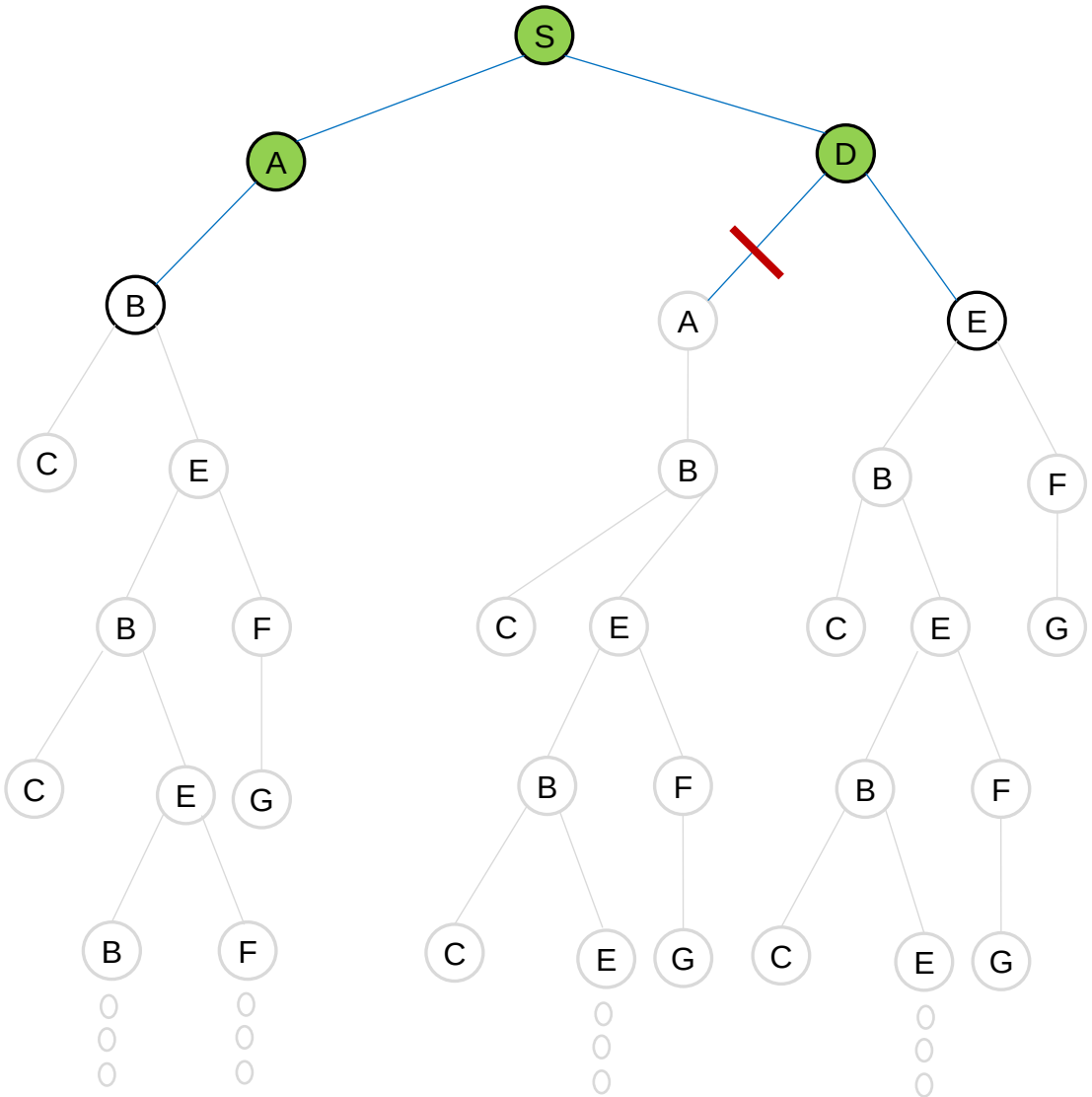
[**S**]	inserts(**S**)
[**A**,D]	pop(S), inserts(A,D)
[**D**,B]	pop(A), inserts(B)



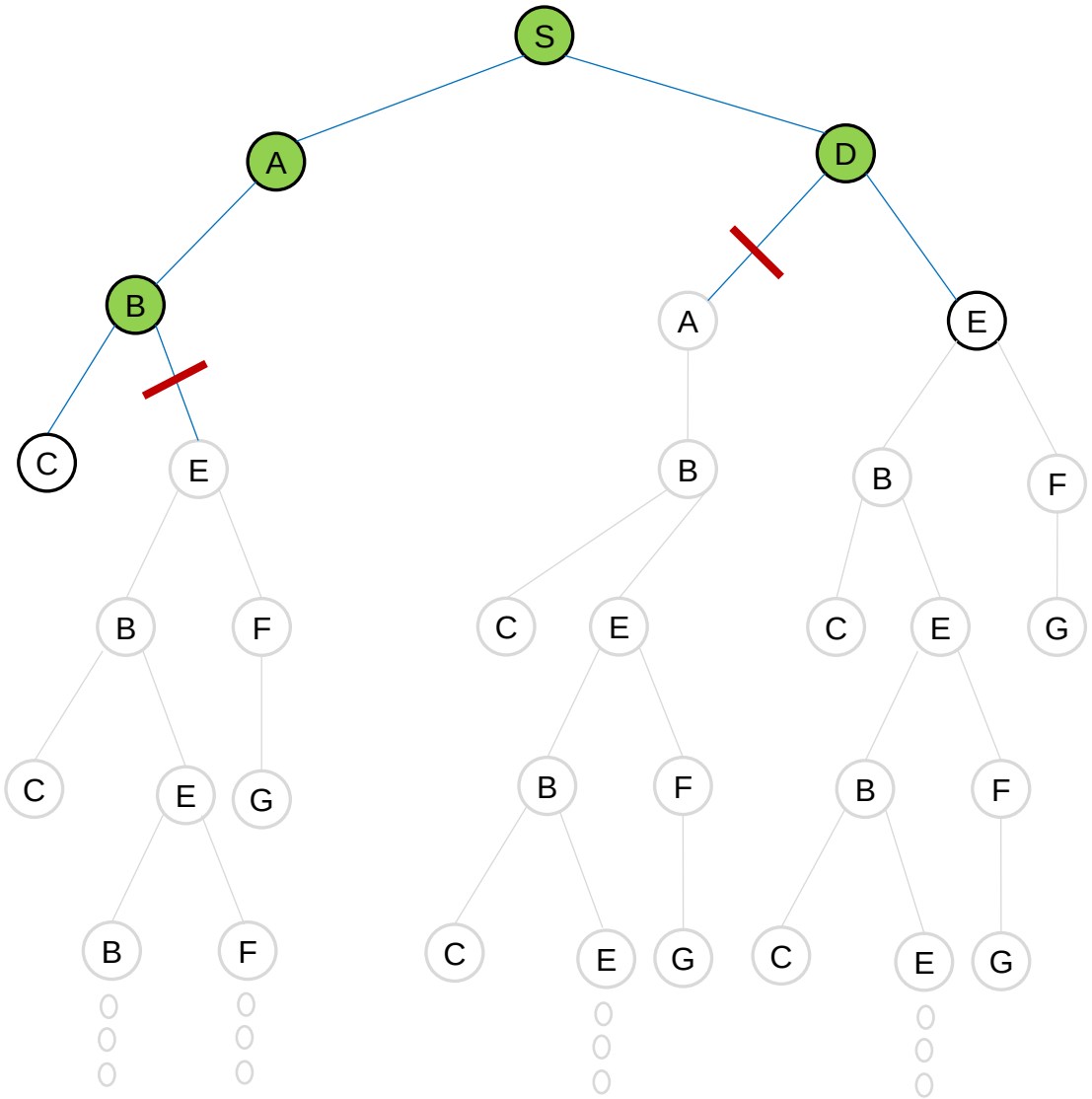
EXPLORED SET: **S, A, D,**

FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A**,D]	pop(S), inserts(A,D)
[**D**,B]	pop(A), inserts(B)
[**B**,E]	pop(D), inserts(E)



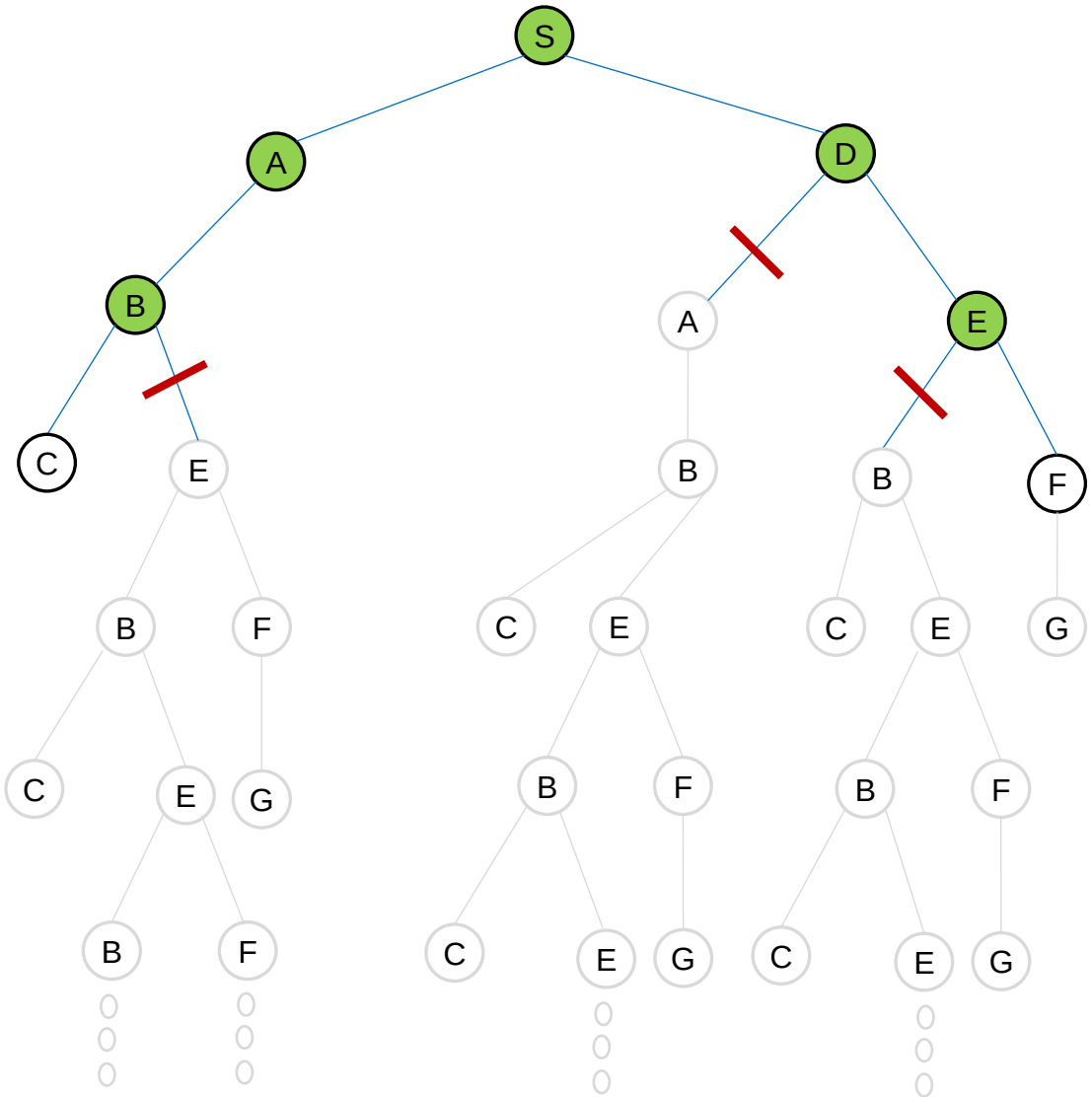
EXPLORED SET: S, A, D, B



FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A**,D]	pop(S), inserts(A,D)
[**D**,B]	pop(A), inserts(B)
[**B**,E]	pop(D), inserts(E)
[**E**,C]	pop(B), inserts(C)

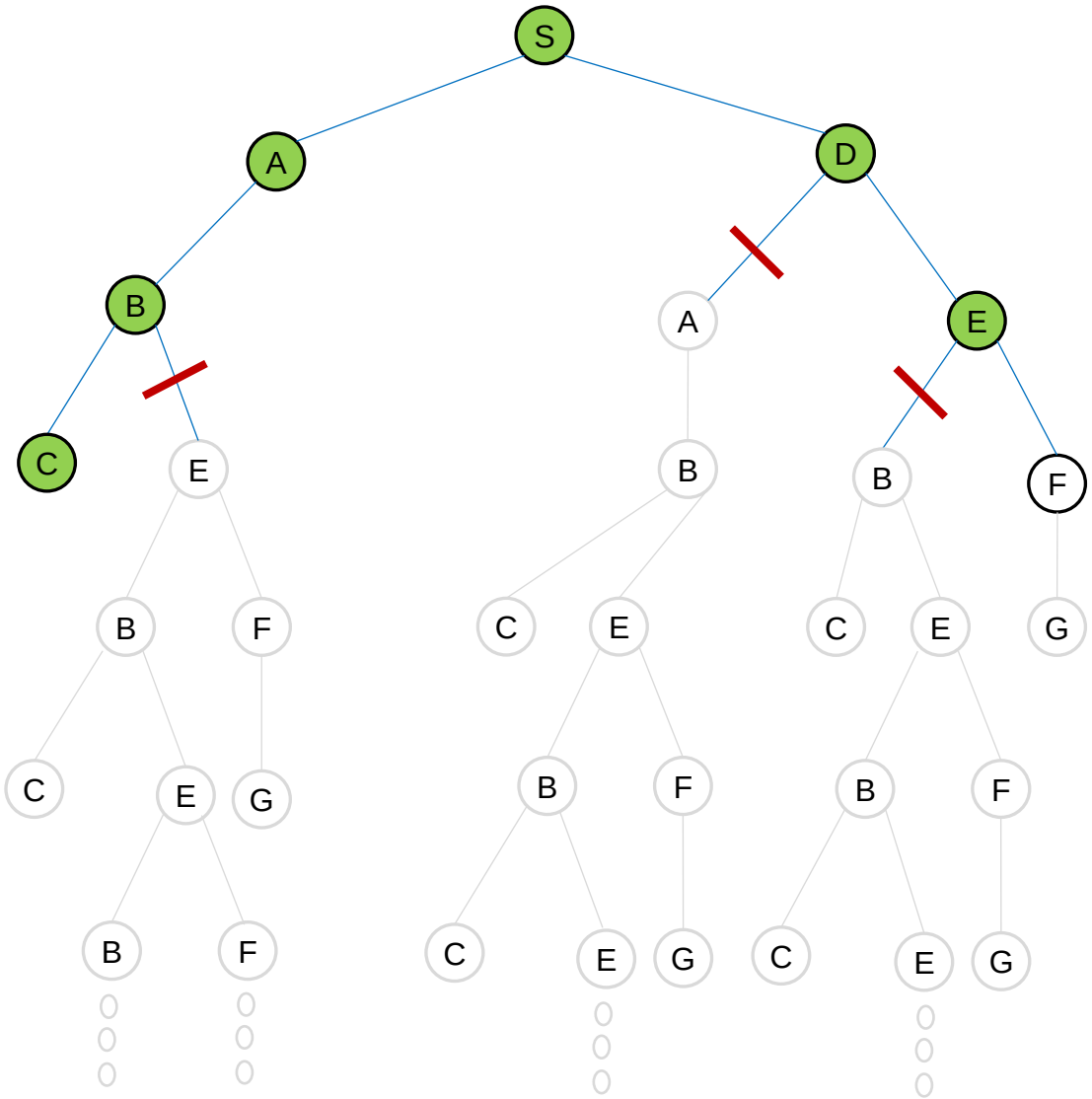
EXPLORED SET: **S, A, D, B, E**



FRONTIER (FIFO)

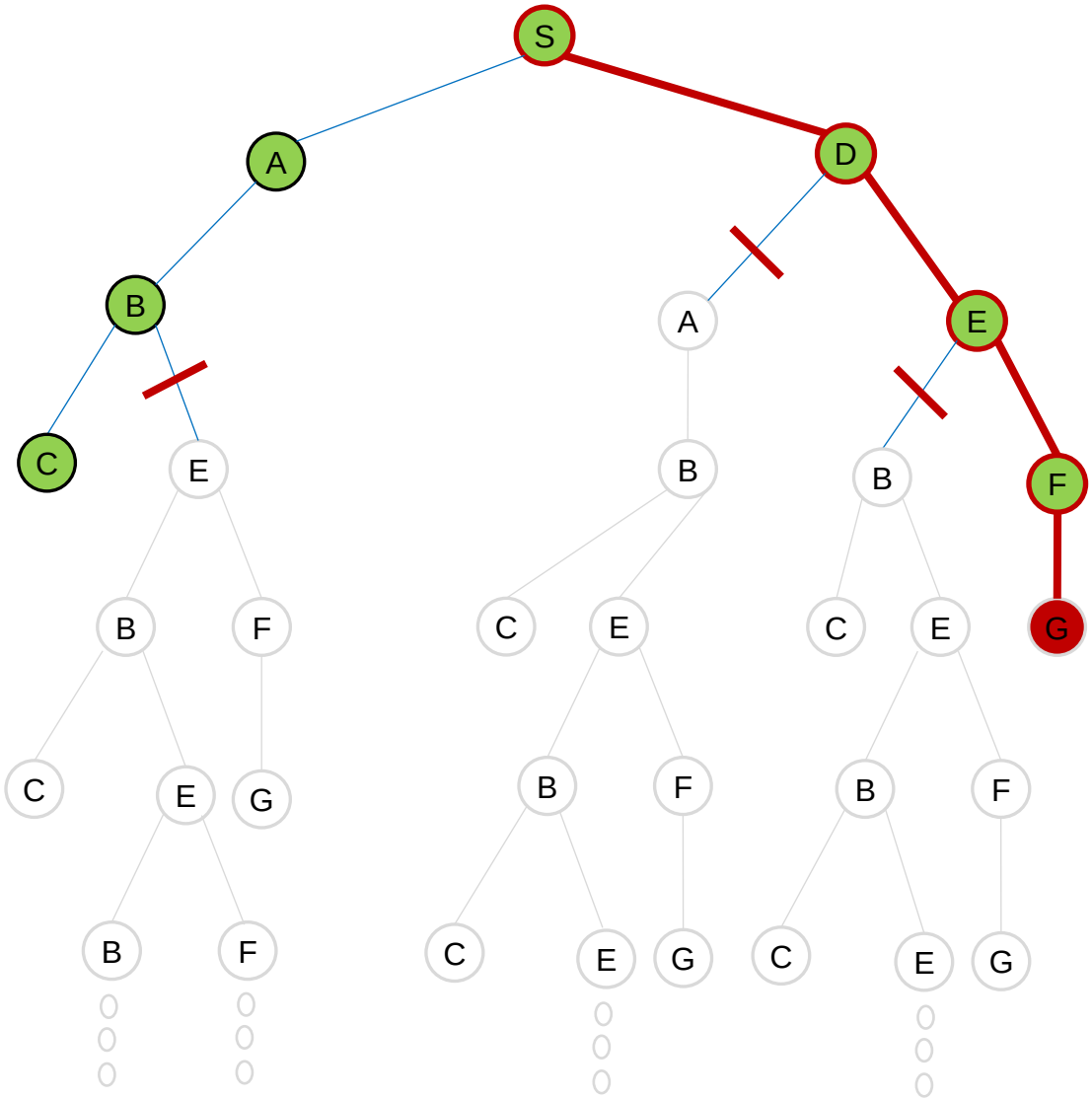
[**S**]	inserts(**S**)
[**A**,D]	pop(S), inserts(A,D)
[**D**,B]	pop(A), inserts(B)
[**B**,E]	pop(D), inserts(E)
[**E**,C]	pop(B), inserts(C)
[**C**,F]	pop(E), inserts(C)

EXPLORED SET: **S, A, D, B, E, C**



FRONTIER (FIFO)	
[**S**]	inserts(**S**)
[**A**,D]	pop(S), inserts(A,D)
[**D**,B]	pop(A), inserts(B)
[**B**,E]	pop(D), inserts(E)
[**E**,C]	pop(B), inserts(C)
[**C**,F]	pop(E), inserts(C)
[**F**]	pop(C)

EXPLORED SET: **S, A, D, B, E, C, F**



FRONTIER (FIFO)

[**S**]	inserts(**S**)
[**A**,D]	pop(S), inserts(A,D)
[**D**,B]	pop(A), inserts(B)
[**B**,E]	pop(D), inserts(E)
[**E**,C]	pop(B), inserts(C)
[**C**,F]	pop(E), inserts(C)
[**F**]	pop(C)
[ ]	pop(F)

BFS = {S,D,E,F,G}

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Breadth-first search algorithm's performance?

- It is complete if the branching factor b is finite.

- The shallowest goal node is not necessarily the optimal one. However, the achieved goal is optimal if all actions have the same cost (or more generally if the path cost is a non-decreasing function of the depth of the node).

$g(n) = g(d)$
and
$g(d)$ is a non-decreasing function for all node n

optimal

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Breadth-first search algorithm's performance?

- For the tree search, if every node has b children and the goal node is at the depth d, then (in the worst case) the total number of generated nodes is:

$$1 + b + b^2 + b^3 + \cdots + b^d = O(b^d)$$

Thus time and space complexity are $O(b^d)$.

- For the graph search, the number of the generated nodes is expected to be reduced in a state space with many redundant paths. However, the complexity does not change when considering the worst case (if there is no redundant path). Time complexity remains the same, $O(b^d)$. Additionally, there will be $O(b^{d-1})$ nodes (states) in the explored set, but space complexity $O(b^d)$ dominates it. So, the space complexity is also $O(b^d)$.

Thus time and space complexity are $O(b^d)$.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Time and memory requirements for breadth-first search

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

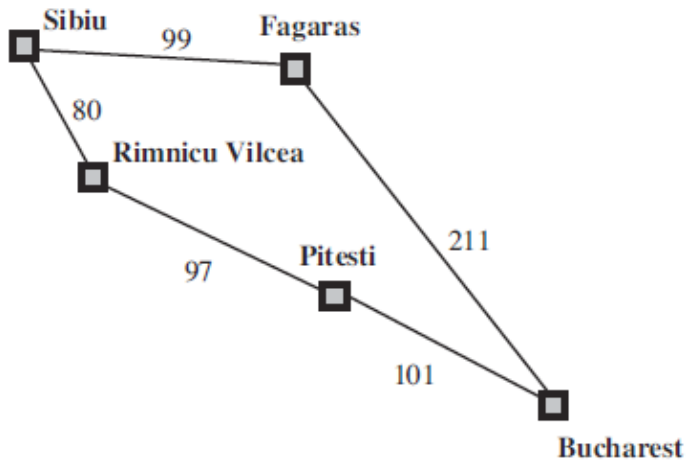
- The memory requirement is a bigger problem for breadth-first search than the execution time.
- However, time is still a major factor.

In general, exponential-complexity search problems cannot be solved by uninformed methods for large size of state spaces.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

2. Uniform-Cost Search



Part of the Romania state space, selected to illustrate uniform-cost search.

Uniform-cost search expands the node n with the lowest path cost $g(n)$. This is done by storing the frontier as a priority queue ordered by $g(n)$.

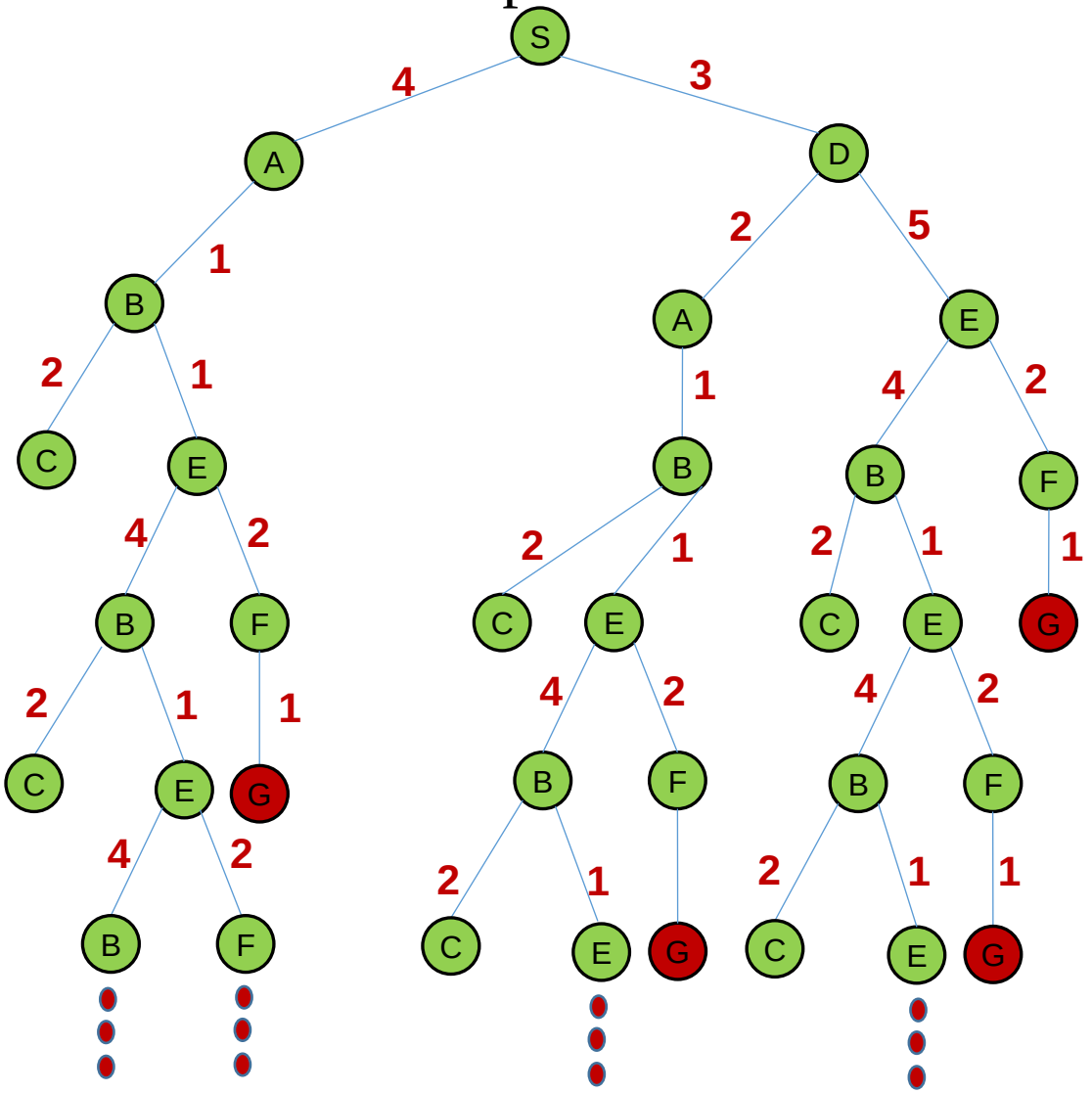
Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The tree-search algorithm for uniform-cost search is given below.

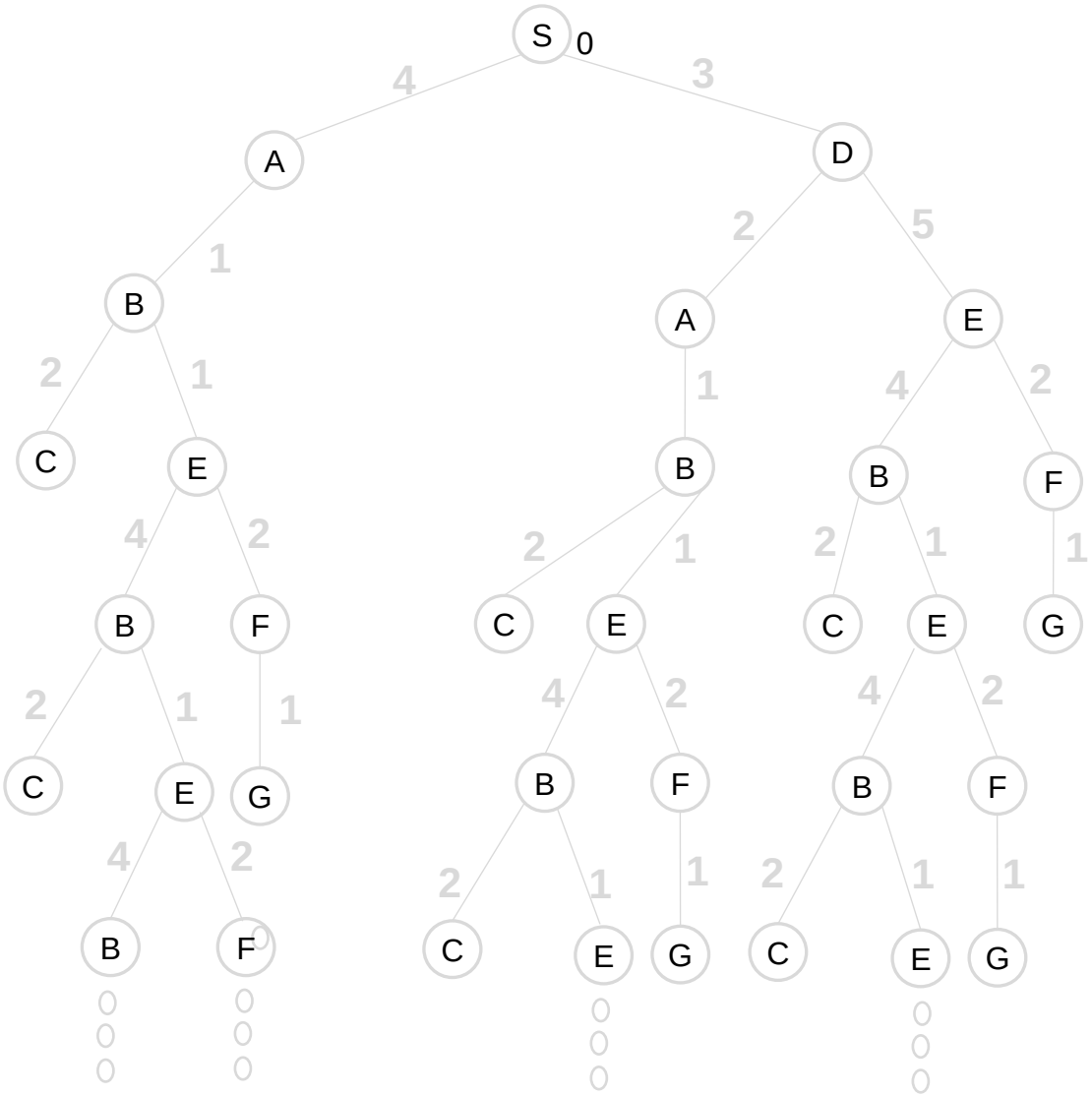
```
function UNIFORM-COST-SEARCH-TREE  (problem) returns a solution, or failure
   - node ← a node with STATE = problem.INITIAL-STATE(),
                                PATH-COST = 0
   - frontier   ← a priority queue ordered by PATH-COST, with the root node
   - loop do
      - if EMPTY( frontier) then return failure
      - node←POP( frontier )  /* chooses the lowest-cost node in frontier*/
      - if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
      - for each action in problem.ACTIONS(node.STATE) do
            - child ←CHILD-NODE(problem, node, action)
            - frontier ←INSERT(child , frontier )
```

Example: Determine a path from the initial state to the goal state by using the tree search of the uniform-cost algorithm. Show the frontier for each step.



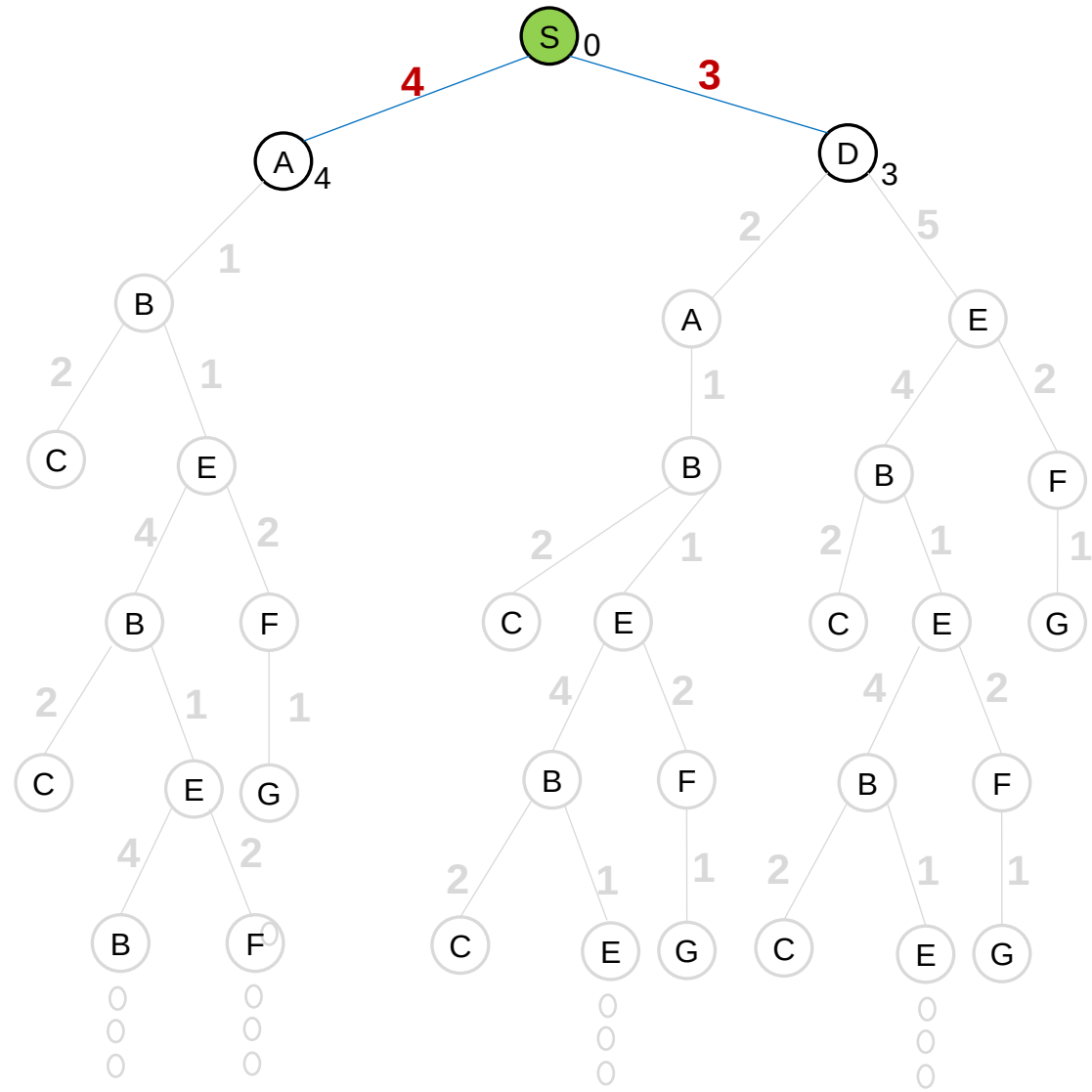
FRONTIER (Priority queue)

[S_0]



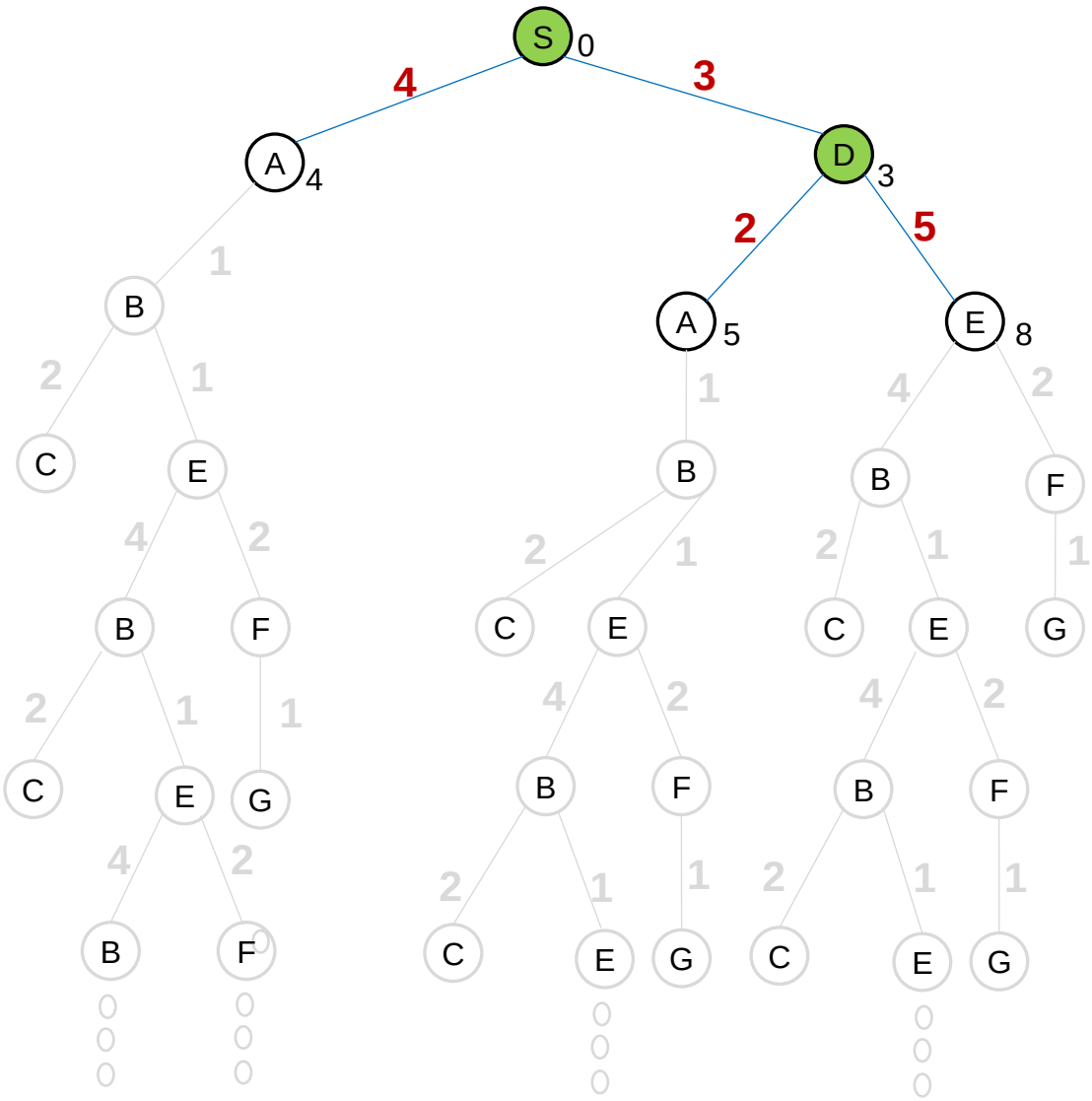
FRONTIER (Priority queue)

[S_0]
[D_3,A_4]



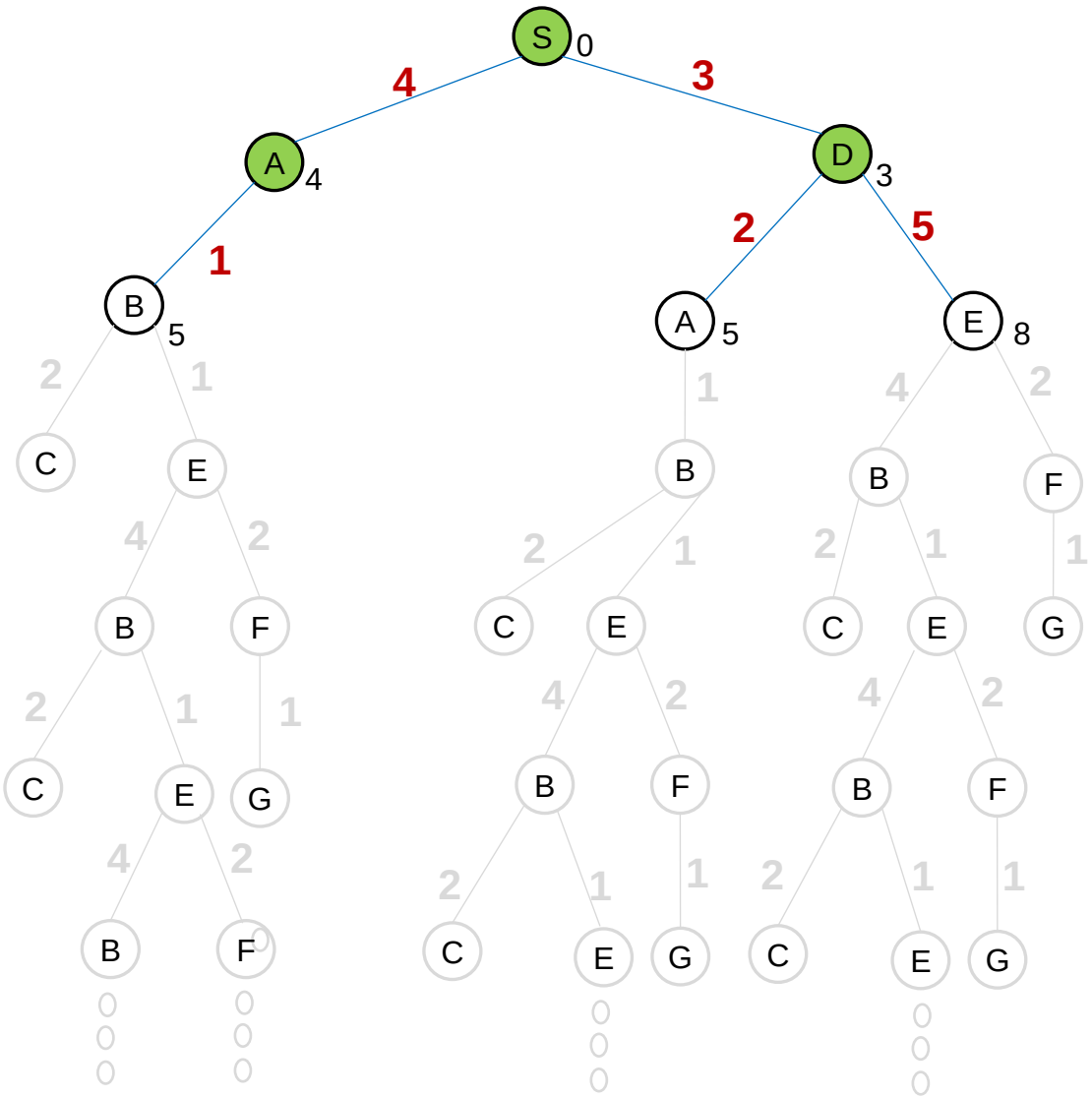
FRONTIER (Priority queue)

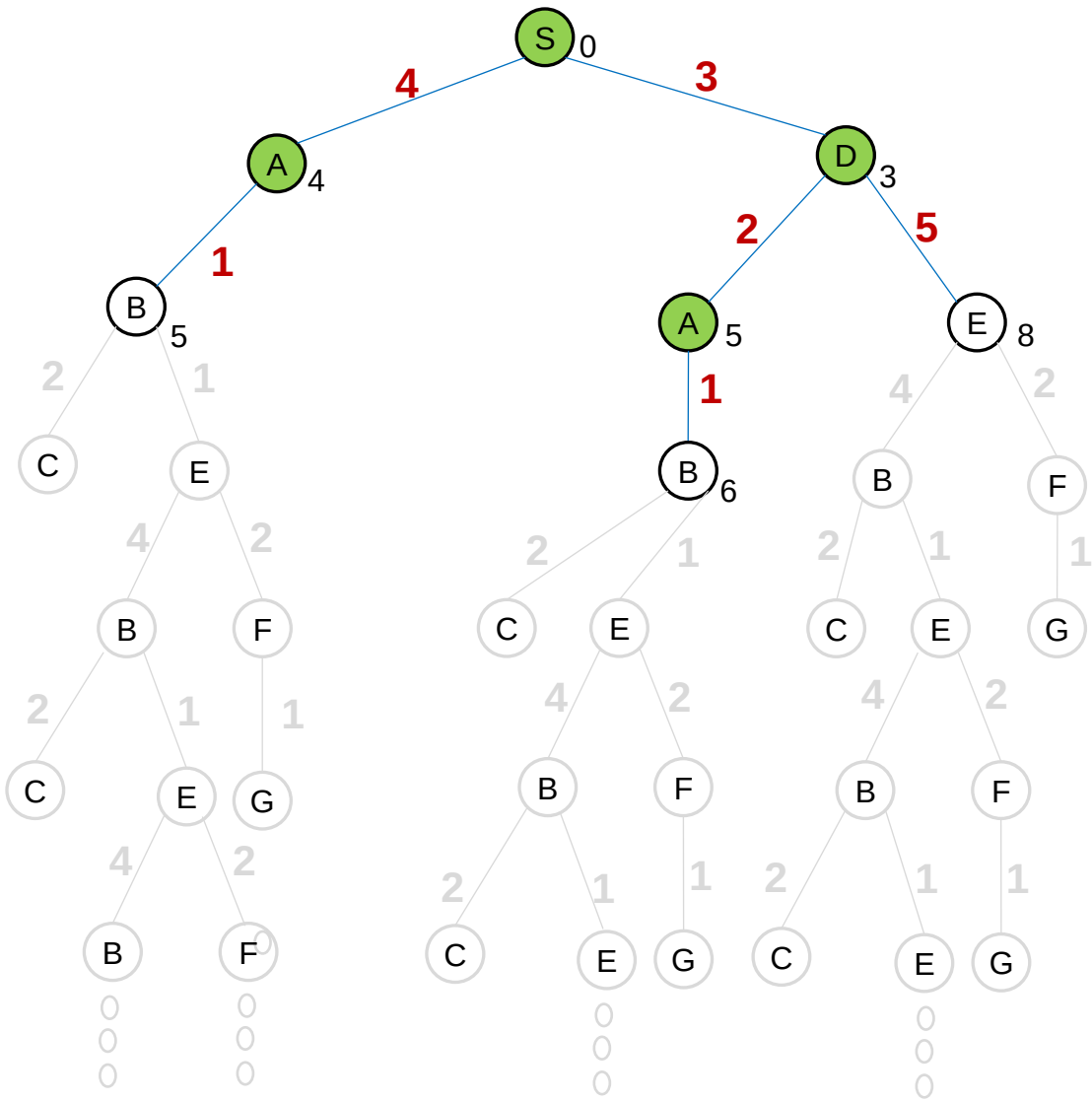
[S_0]
[D_3,A_4]
[A_4,A_5,E_8]



FRONTIER (Priority queue)

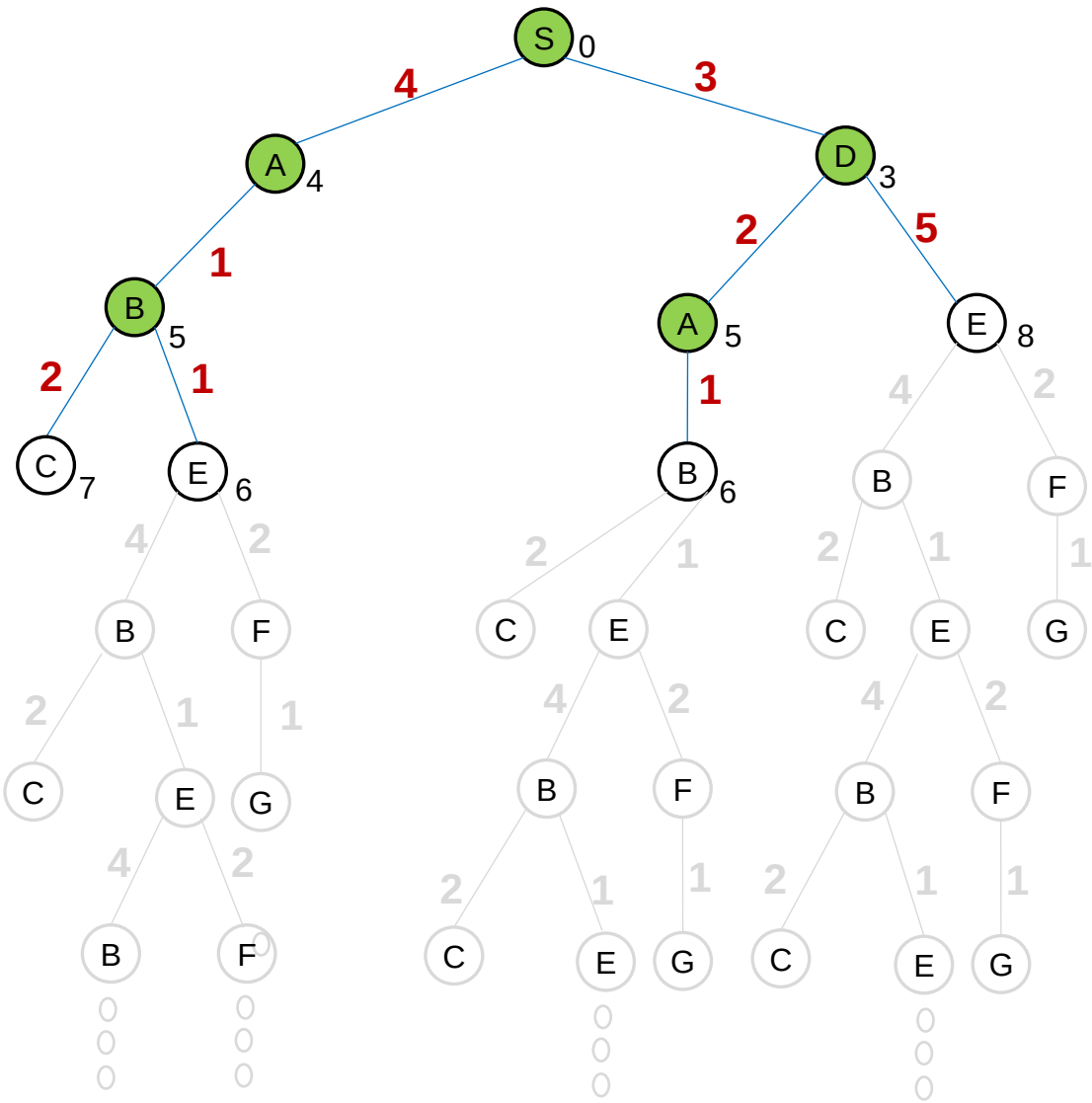
[S_0]
[D_3,A_4]
[A_4,A_5,E_8]
[A_5,B_5,E_8]





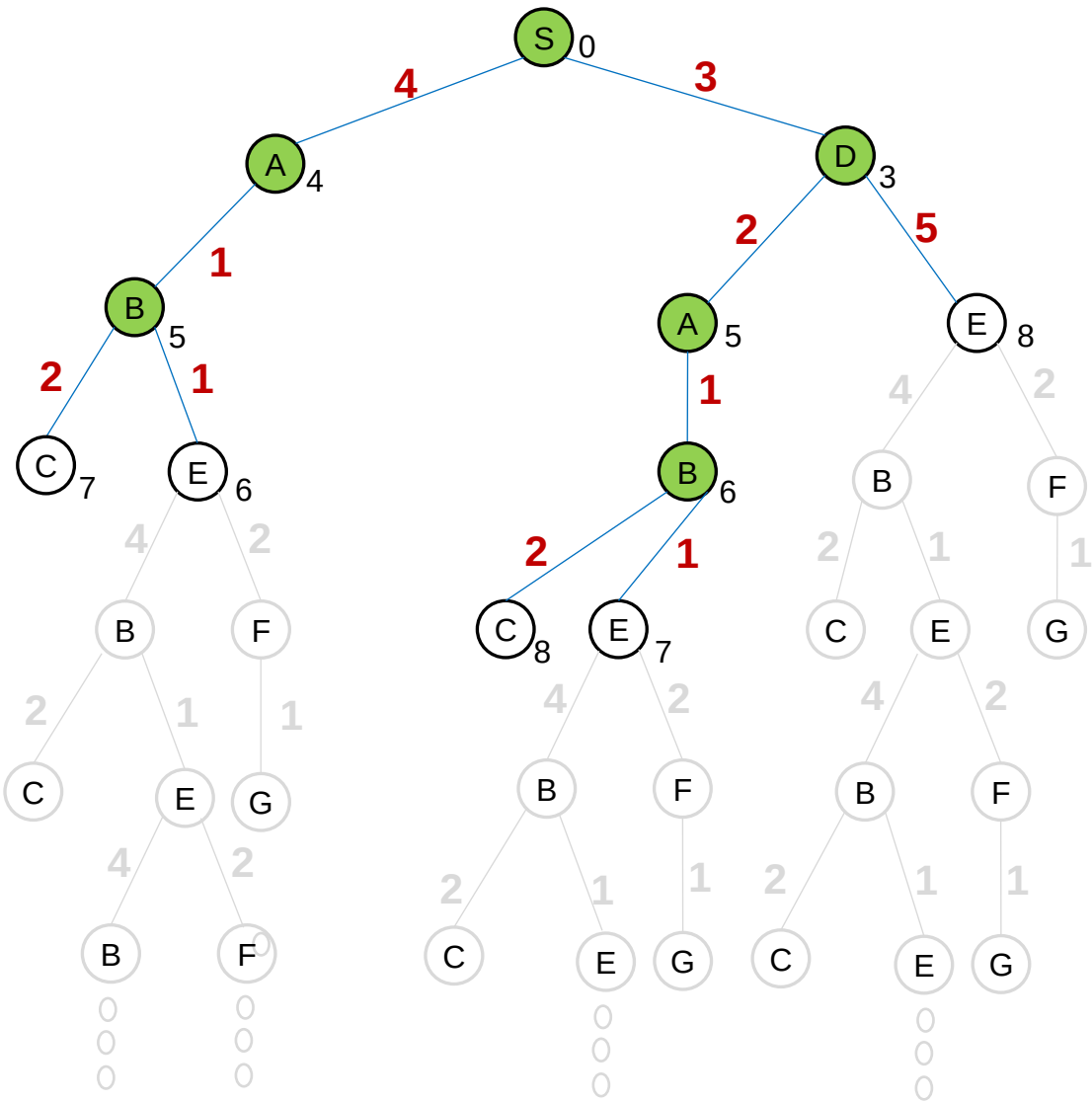
FRONTIER (Priority queue)

$[S_0]$
$[D_3, A_4]$
$[A_4, A_5, E_8]$
$[A_5, B_5, E_8]$
$[B_5, B_6, E_8]$



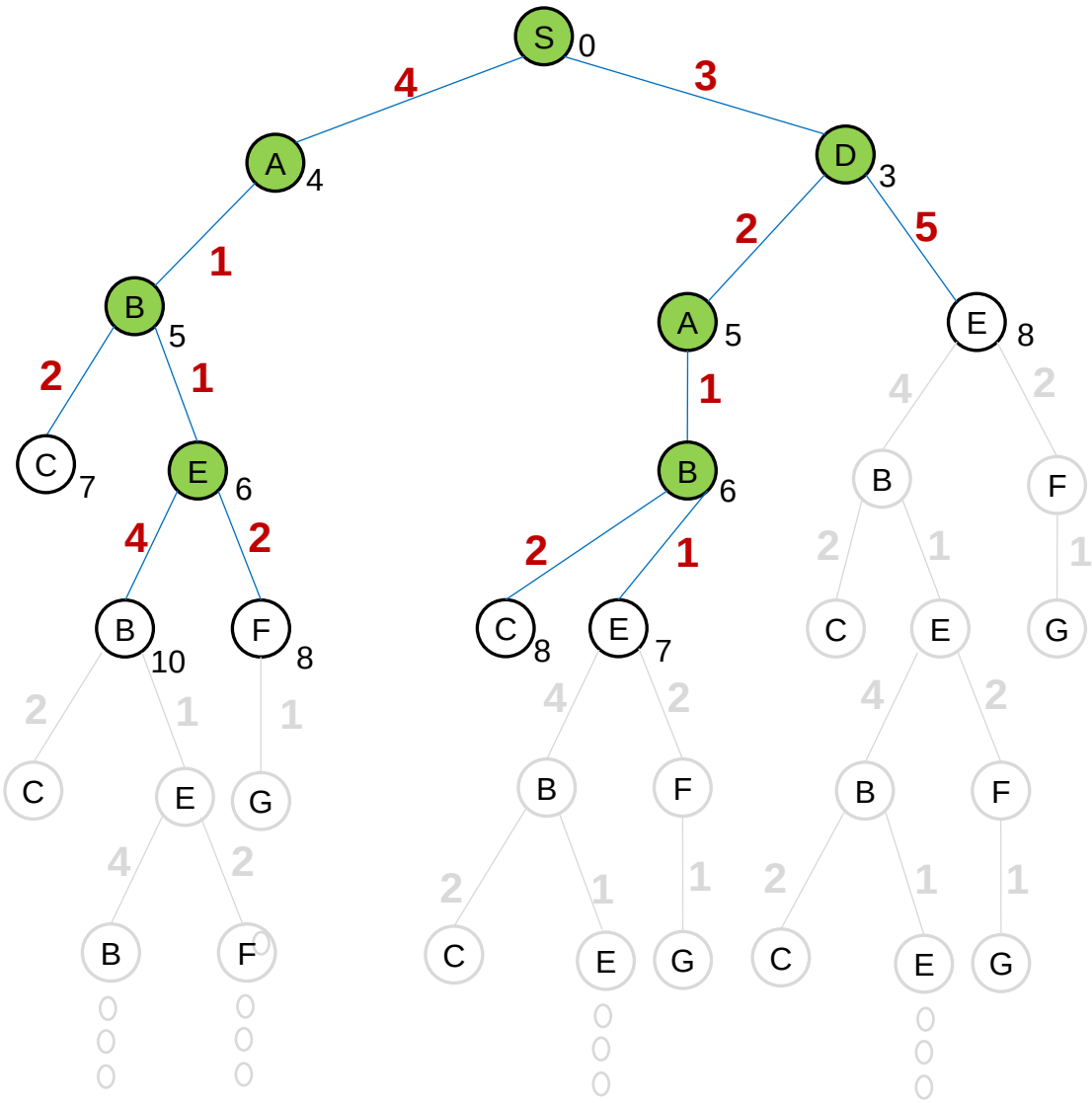
FRONTIER (Priority queue)

[$\mathbf{S_0}$]
[$\mathbf{D_3}$,A_4]
[$\mathbf{A_4}$,A_5,E_8]
[$\mathbf{A_5}$,B_5,E_8]
[$\mathbf{B_5}$,B_6,E_8]
[$\mathbf{B_6}$,E_6,C_7,E_8]



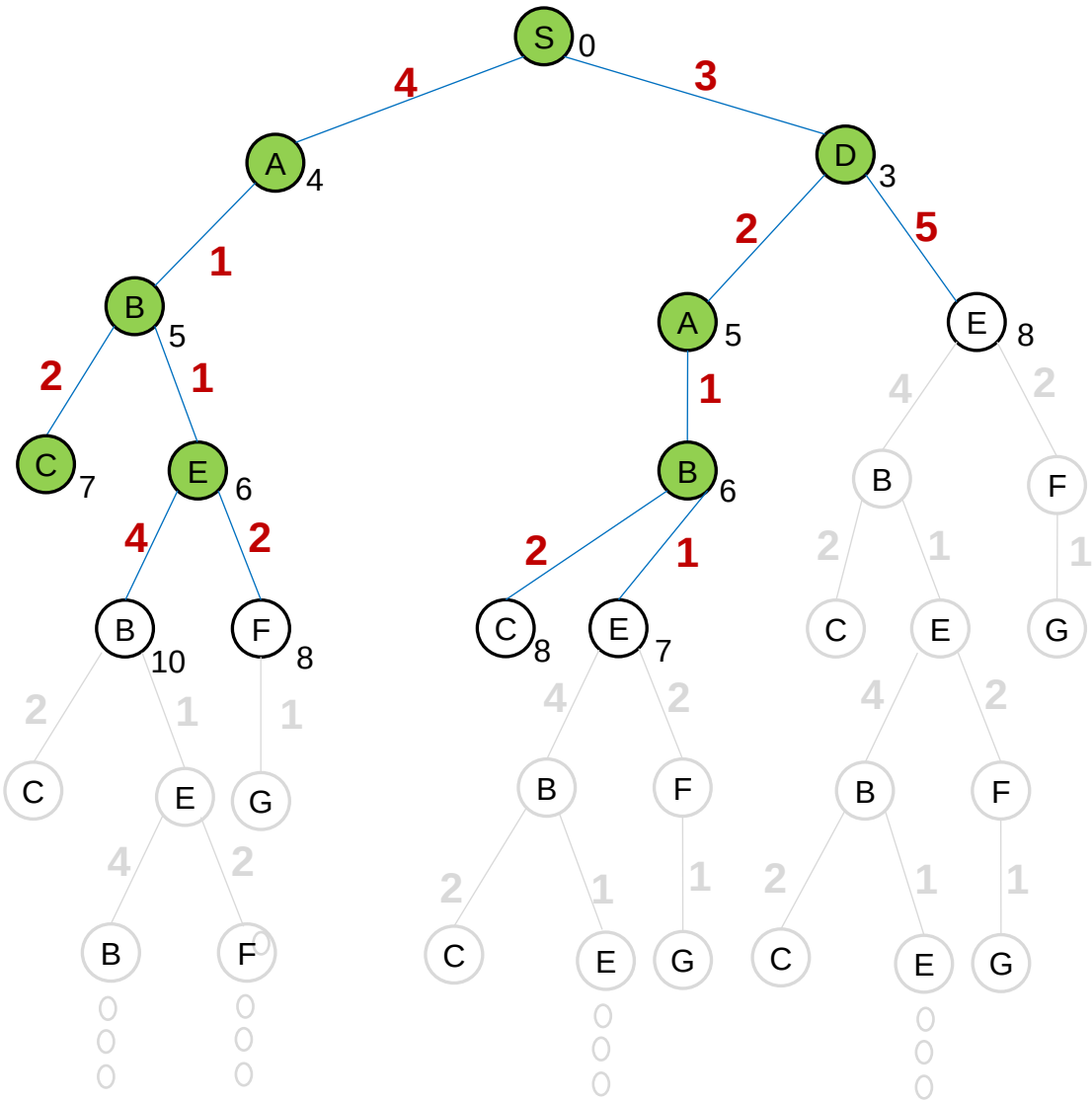
FRONTIER (Priority queue)

[S_0]
[D_3,A_4]
[A_4,A_5,E_8]
[A_5,B_5,E_8]
[B_5,B_6,E_8]
[B_6,E_6,C_7,E_8]
[E_6,C_7,E_7,E_8,C_8]



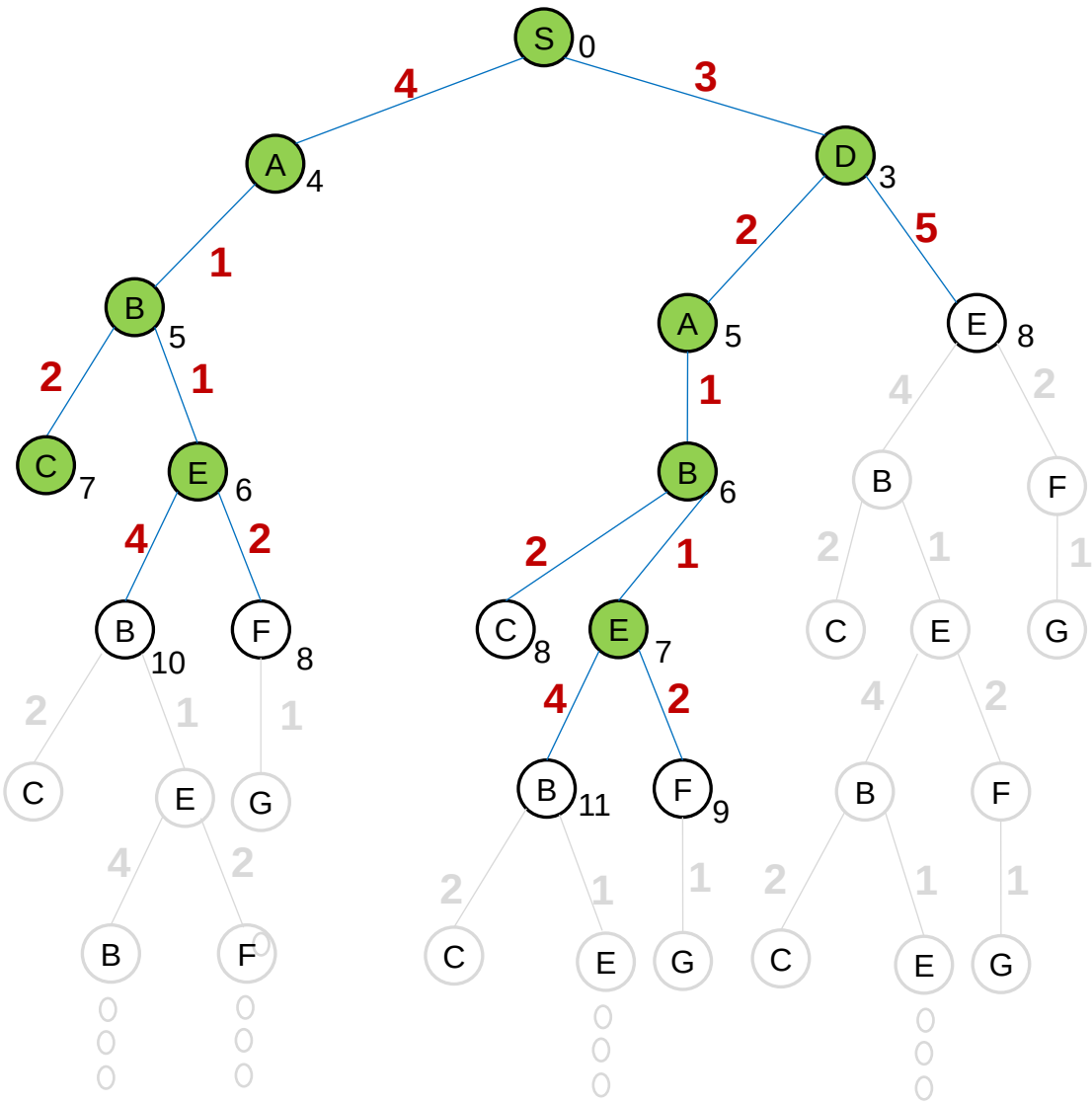
FRONTIER (Priority queue)

[S_0]
[D_3,A_4]
[A_4,A_5,E_8]
[A_5,B_5,E_8]
[B_5,B_6,E_8]
[B_6,E_6,C_7,E_8]
[E_6,C_7,E_7,E_8,C_8]
[C_7,E_7,E_8,C_8,F_8,B_{10}]



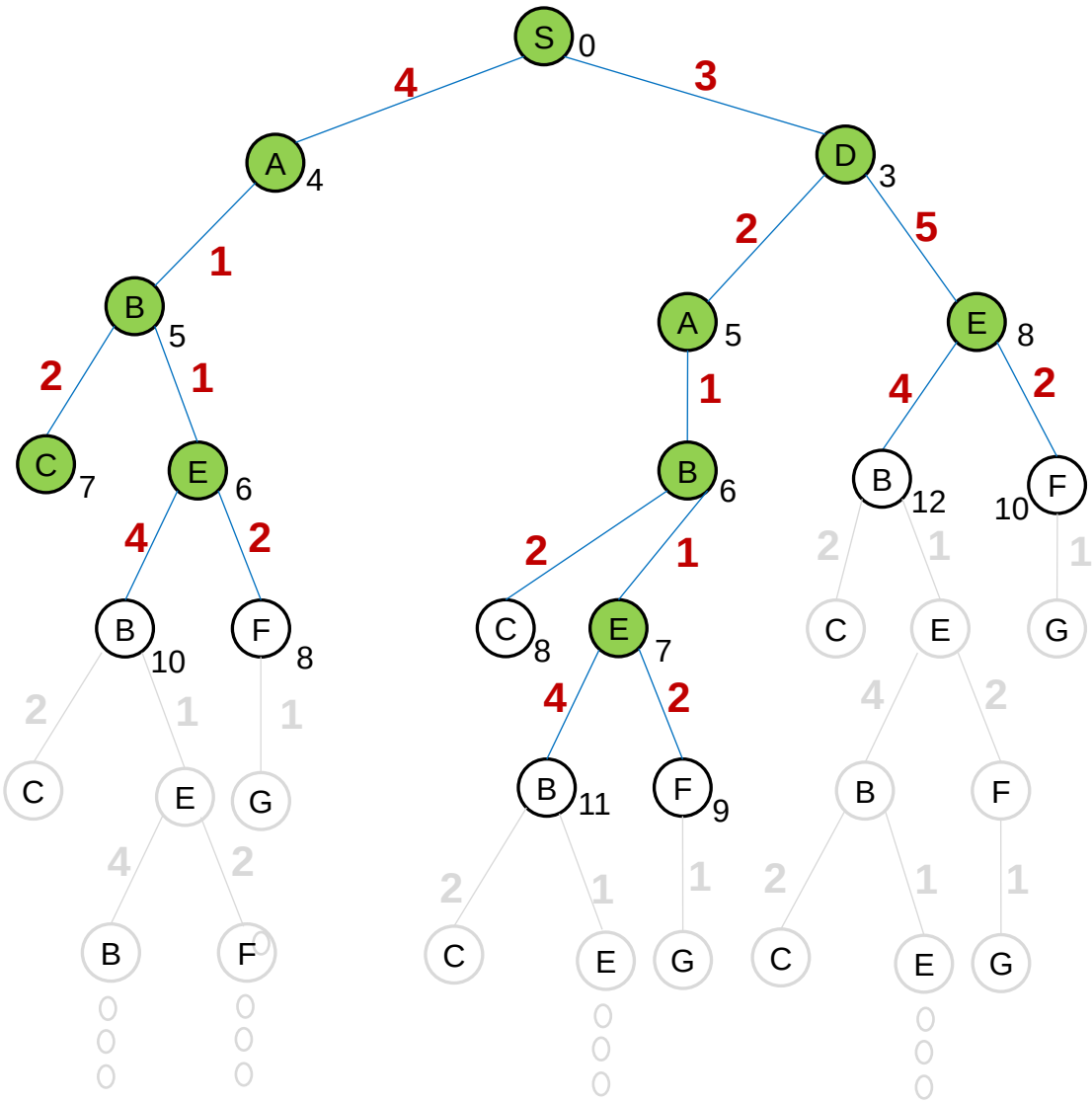
FRONTIER (Priority queue)

[S_0]
[D_3,A_4]
[A_4,A_5,E_8]
[A_5,B_5,E_8]
[B_5,B_6,E_8]
[B_6,E_6,C_7,E_8]
[E_6,C_7,E_7,E_8,C_8]
[C_7,E_7,E_8,C_8,F_8,B_{10}]
[E_7,E_8,C_8,F_8,B_{10}]



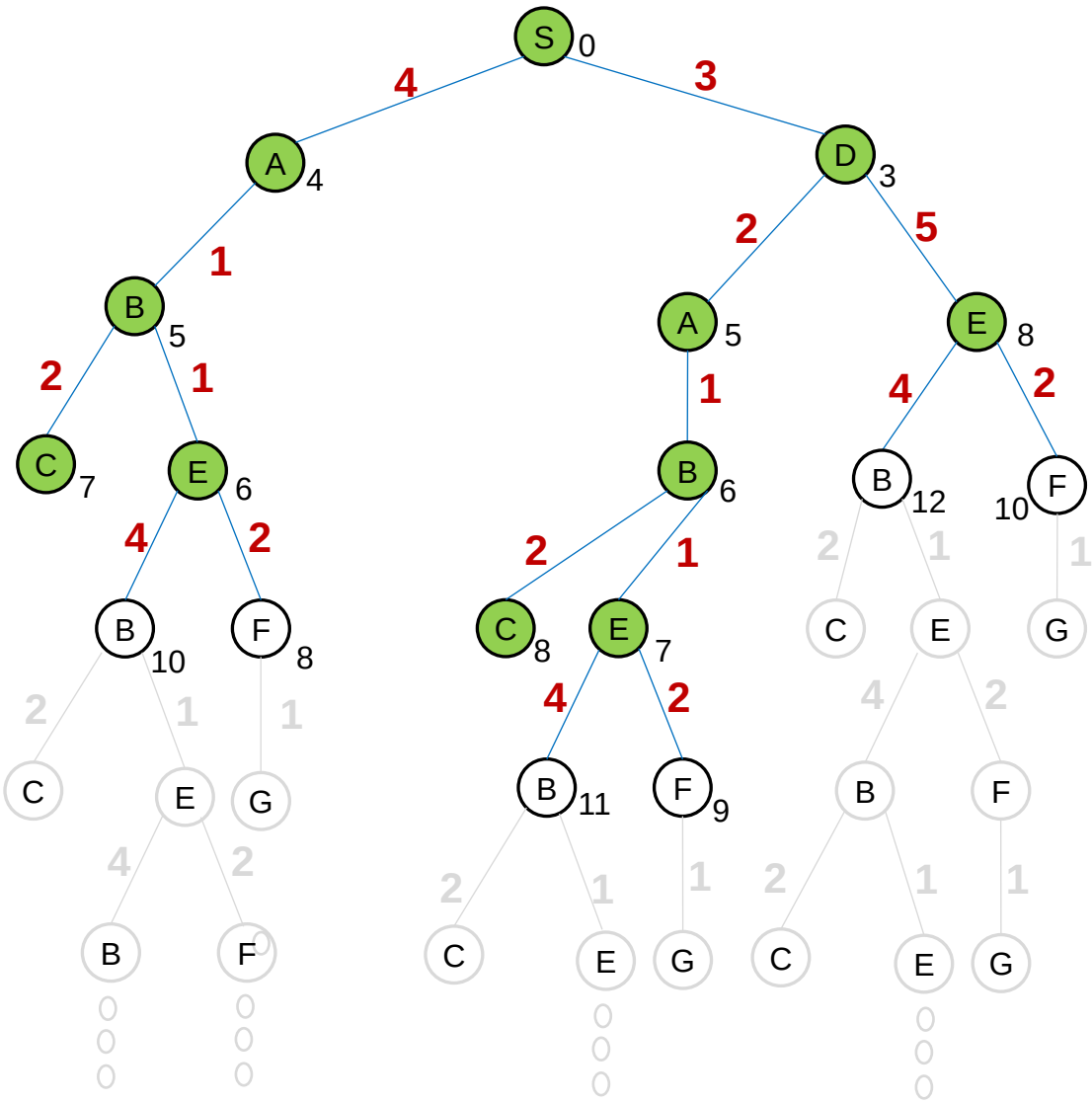
FRONTIER (Priority queue)

$[S_0]$
$[D_3, A_4]$
$[A_4, A_5, E_8]$
$[A_5, B_5, E_8]$
$[B_5, B_6, E_8]$
$[B_6, E_6, C_7, E_8]$
$[E_6, C_7, E_7, E_8, C_8]$
$[C_7, E_7, E_8, C_8, F_8, B_{10}]$
$[E_7, E_8, C_8, F_8, B_{10}]$
$[E_8, C_8, F_8, F_9, B_{10}, B_{11}]$



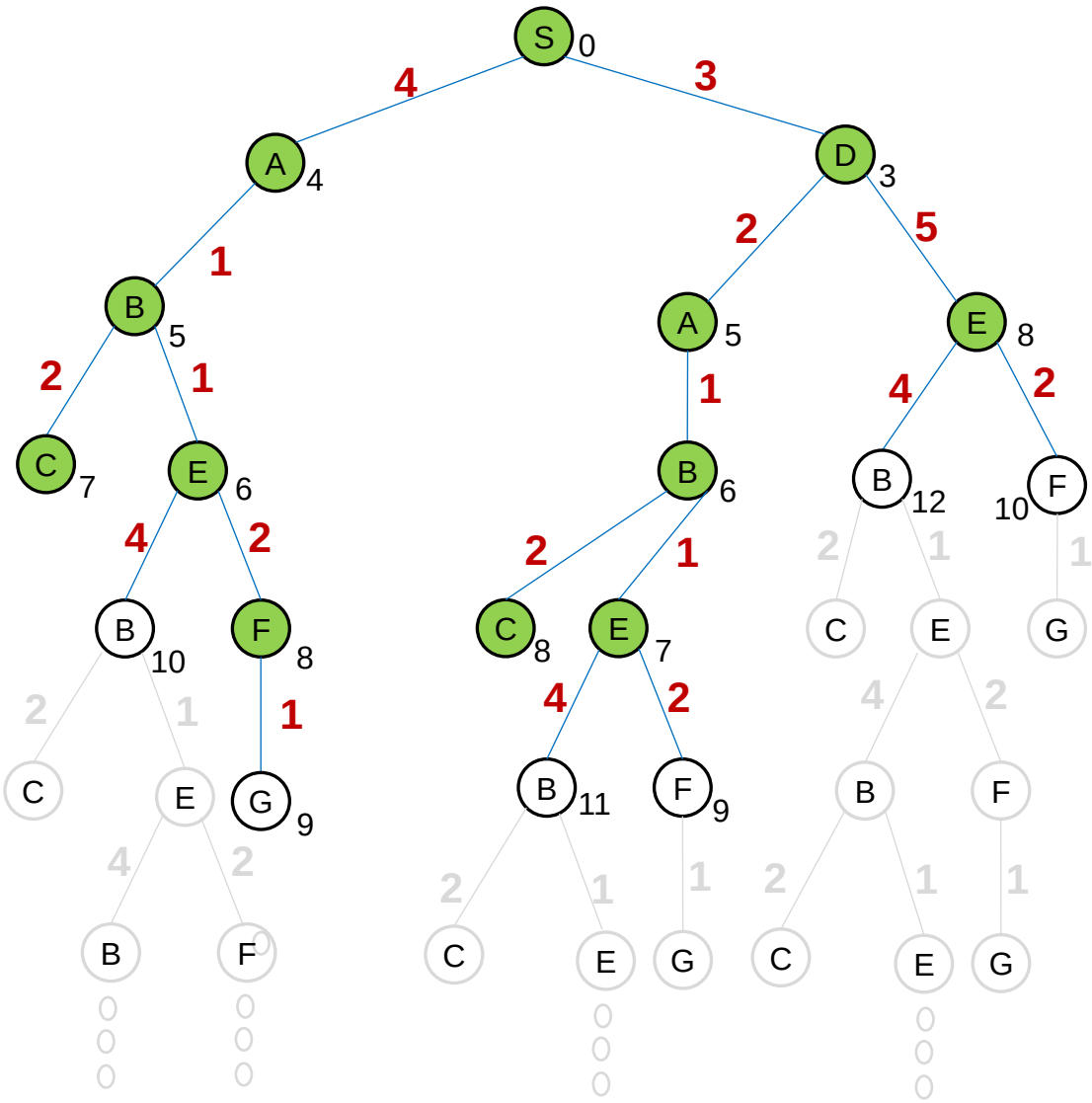
FRONTIER (Priority queue)

[S_0]
[D_3,A_4]
[A_4,A_5,E_8]
[A_5,B_5,E_8]
[B_5,B_6,E_8]
[B_6,E_6,C_7,E_8]
[E_6,C_7,E_7,E_8,C_8]
[C_7,E_7,E_8,C_8,F_8,B_{10}]
[E_7,E_8,C_8,F_8,B_{10}]
[E_8,C_8,F_8,F_9,B_{10},B_{11}]
[C_8,F_8,F_9,B_{10},F_{10},B_{11},B_{12}]



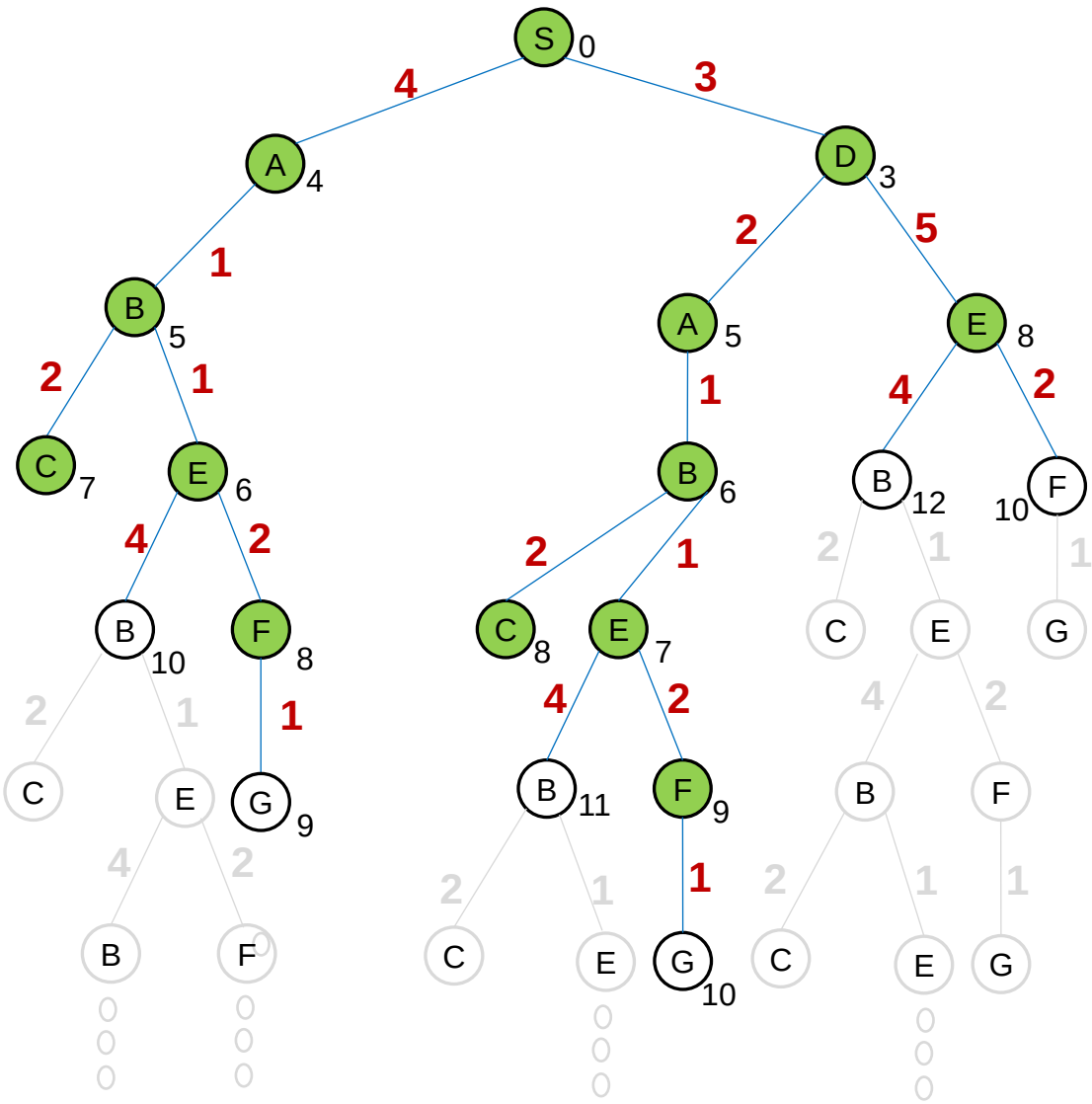
FRONTIER (Priority queue)

[S_0]
[D_3,A_4]
[A_4,A_5,E_8]
[A_5,B_5,E_8]
[B_5,B_6,E_8]
[B_6,E_6,C_7,E_8]
[E_6,C_7,E_7,E_8,C_8]
[C_7,E_7,E_8,C_8,F_8,B_{10}]
[E_7,E_8,C_8,F_8,B_{10}]
[E_8,C_8,F_8,F_9,B_{10},B_{11}]
[C_8,F_8,F_9,B_{10},F_{10},B_{11},B_{12}]
[F_8,F_9,B_{10},F_{10},B_{11},B_{12}]



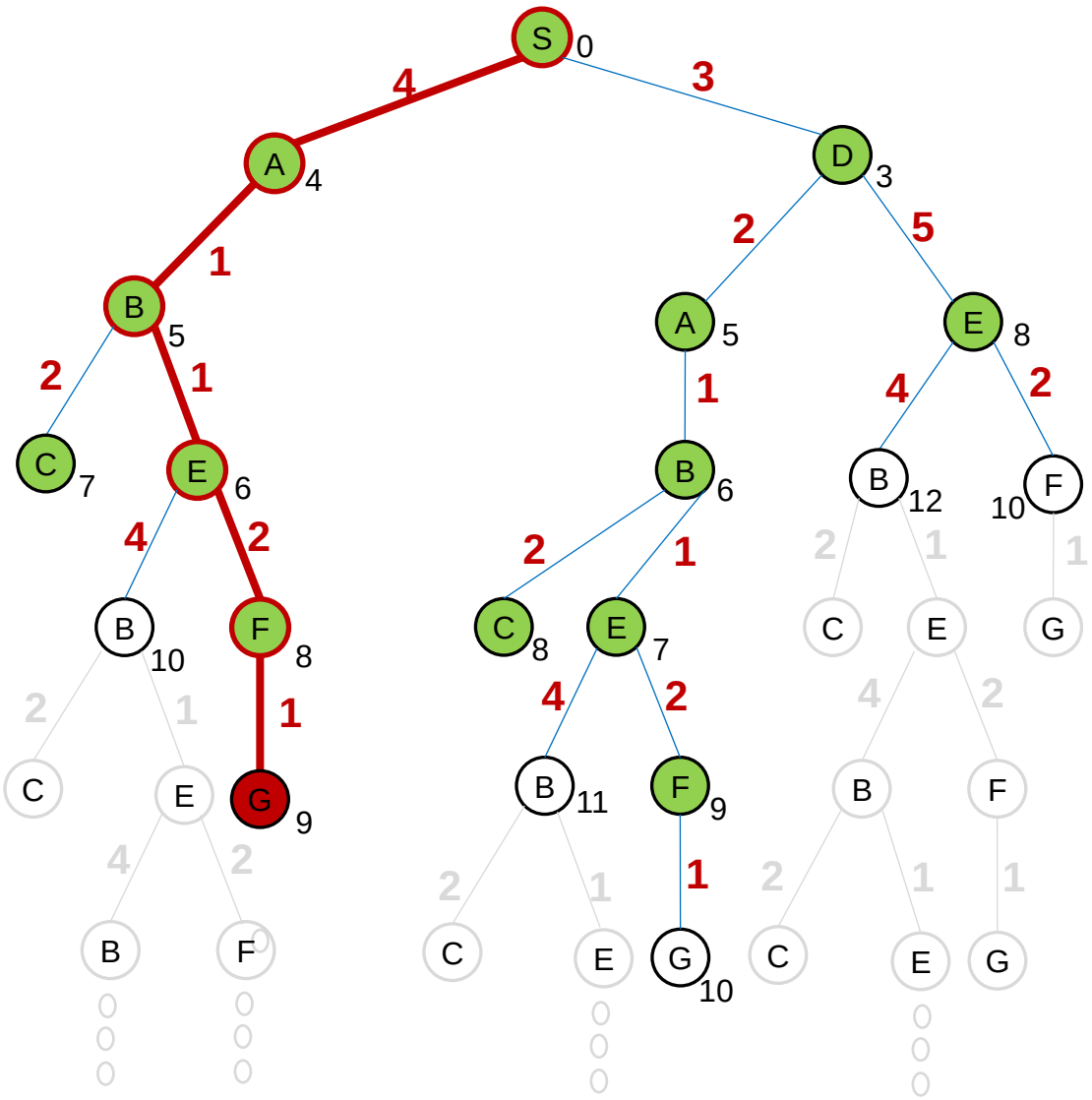
FRONTIER (Priority queue)

$[S_0]$
$[D_3, A_4]$
$[A_4, A_5, E_8]$
$[A_5, B_5, E_8]$
$[B_5, B_6, E_8]$
$[B_6, E_6, C_7, E_8]$
$[E_6, C_7, E_7, E_8, C_8]$
$[C_7, E_7, E_8, C_8, F_8, B_{10}]$
$[E_7, E_8, C_8, F_8, B_{10}]$
$[E_8, C_8, F_8, F_9, B_{10}, B_{11}]$
$[C_8, F_8, F_9, B_{10}, F_{10}, B_{11}, B_{12}]$
$[F_8, F_9, B_{10}, F_{10}, B_{11}, B_{12}]$
$[F_9, G_9, B_{10}, F_{10}, B_{11}, B_{12}]$



FRONTIER (Priority queue)

[S_0]
[D_3,A_4]
[A_4,A_5,E_8]
[A_5,B_5,E_8]
[B_5,B_6,E_8]
[B_6,E_6,C_7,E_8]
[E_6,C_7,E_7,E_8,C_8]
[C_7,E_7,E_8,C_8,F_8,B_{10}]
[E_7,E_8,C_8,F_8,B_{10}]
[E_8,C_8,F_8,F_9,B_{10},B_{11}]
[C_8,F_8,F_9,B_{10},F_{10},B_{11},B_{12}]
[F_8,F_9,B_{10},F_{10},B_{11},B_{12}]
[F_9,G_9,B_{10},F_{10},B_{11},B_{12}]
[G_9,B_{10},F_{10},G_{10},B_{11},B_{12}]



FRONTIER (Priority queue)

$[S_0]$
$[D_3, A_4]$
$[A_4, A_5, E_8]$
$[A_5, B_5, E_8]$
$[B_5, B_6, E_8]$
$[B_6, E_6, C_7, E_8]$
$[E_6, C_7, E_7, E_8, C_8]$
$[C_7, E_7, E_8, C_8, F_8, B_{10}]$
$[E_7, E_8, C_8, F_8, B_{10}]$
$[E_8, C_8, F_8, F_9, B_{10}, B_{11}]$
$[C_8, F_8, F_9, B_{10}, F_{10}, B_{11}, B_{12}]$
$[F_8, F_9, B_{10}, F_{10}, B_{11}, B_{12}]$
$[F_9, G_9, B_{10}, F_{10}, B_{11}, B_{12}]$
$[G_9, B_{10}, F_{10}, G_{10}, B_{11}, B_{12}]$
$[B_{10}, F_{10}, G_{10}, B_{11}, B_{12}]$

UCS={S,A,B,E,F,G}

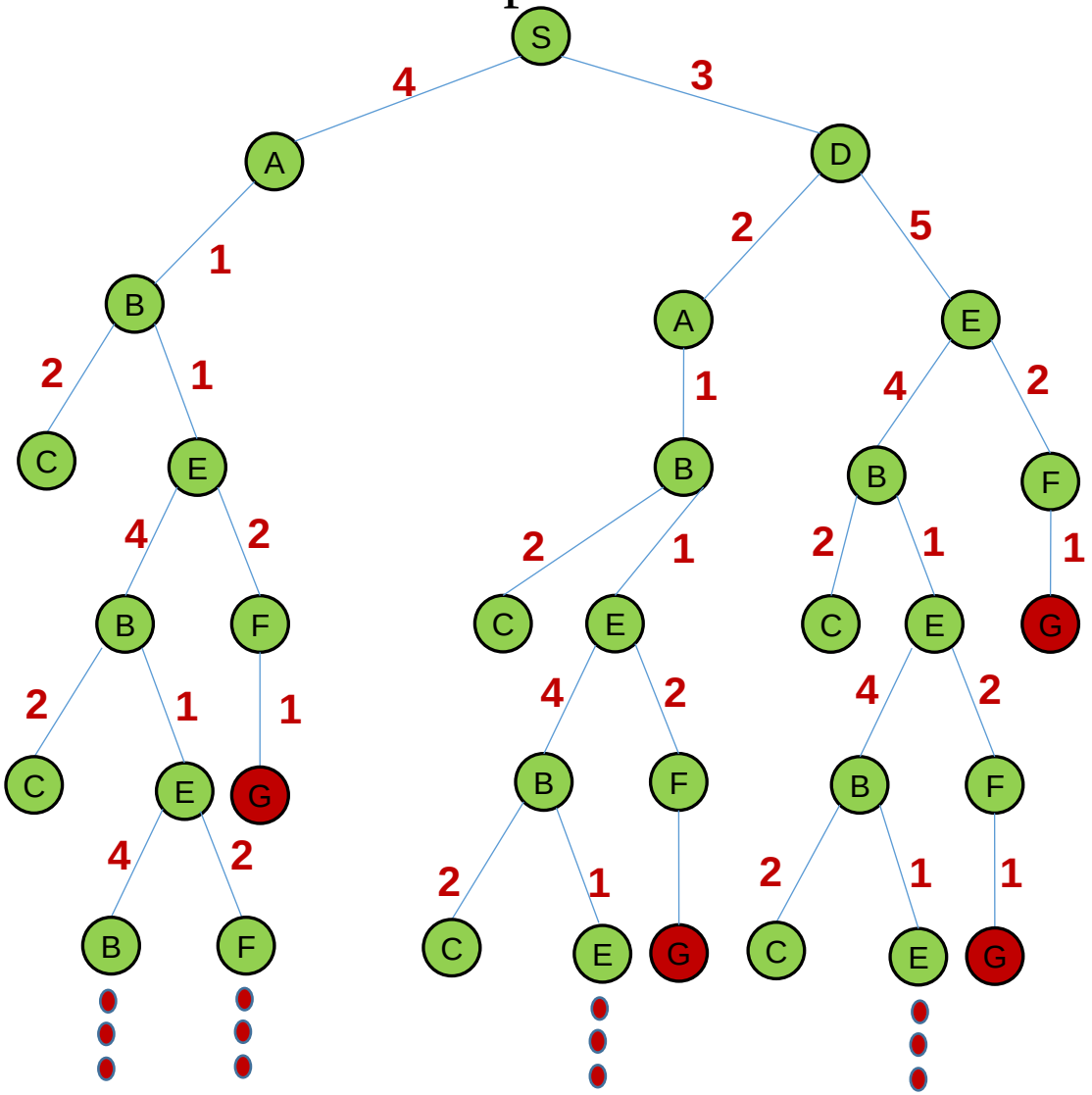
Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The graph-search algorithm for uniform-cost search is given below.

```
function UNIFORM-COST-SEARCH-GRAPH  (problem) returns a solution, or failure
  - node ← a node with STATE = problem.INITIAL-STATE(),
                                PATH-COST = 0
  - frontier   ← a priority queue ordered by PATH-COST, with the root node
  - explored ← an empty set
  - loop do
     - if EMPTY( frontier) then return failure
     - node←POP( frontier )  /* chooses the lowest-cost node in frontier*/
     - if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
     - add node.STATE to explored set
     - for each action in problem.ACTIONS(node.STATE) do
          - child ←CHILD-NODE(problem, node, action)
          - if child .STATE is not in explored or frontier then
               - frontier ←INSERT(child , frontier )
           else if child.STATE is in frontier with higher PATH-COST then
               - replace that frontier node with child
```

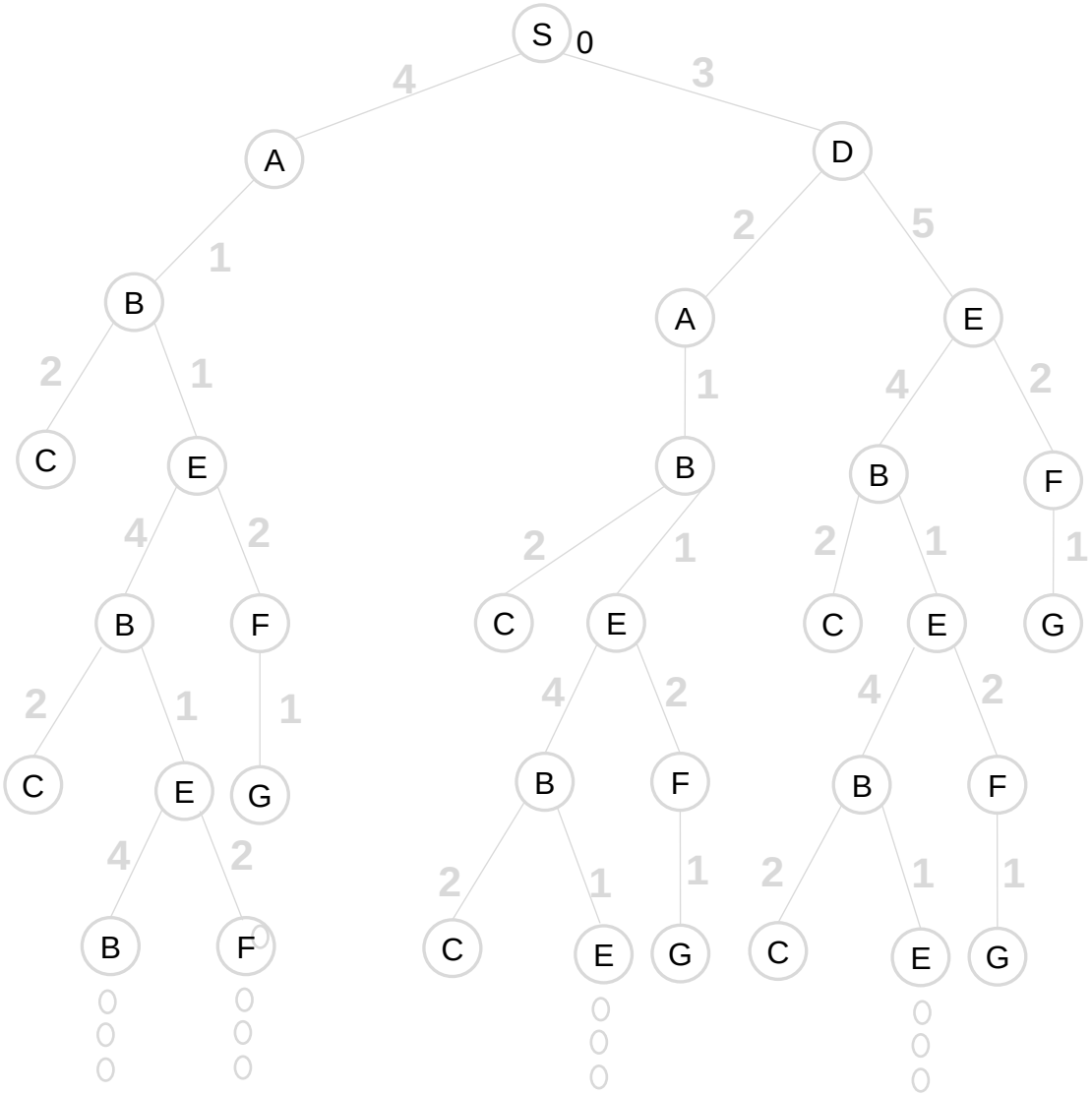
Example: Determine a path from the initial state to the goal state by using the graph search of the uniform-cost algorithm. Show the frontier for each step.



EXPLORED SET:

FRONTIER (Priority queue)

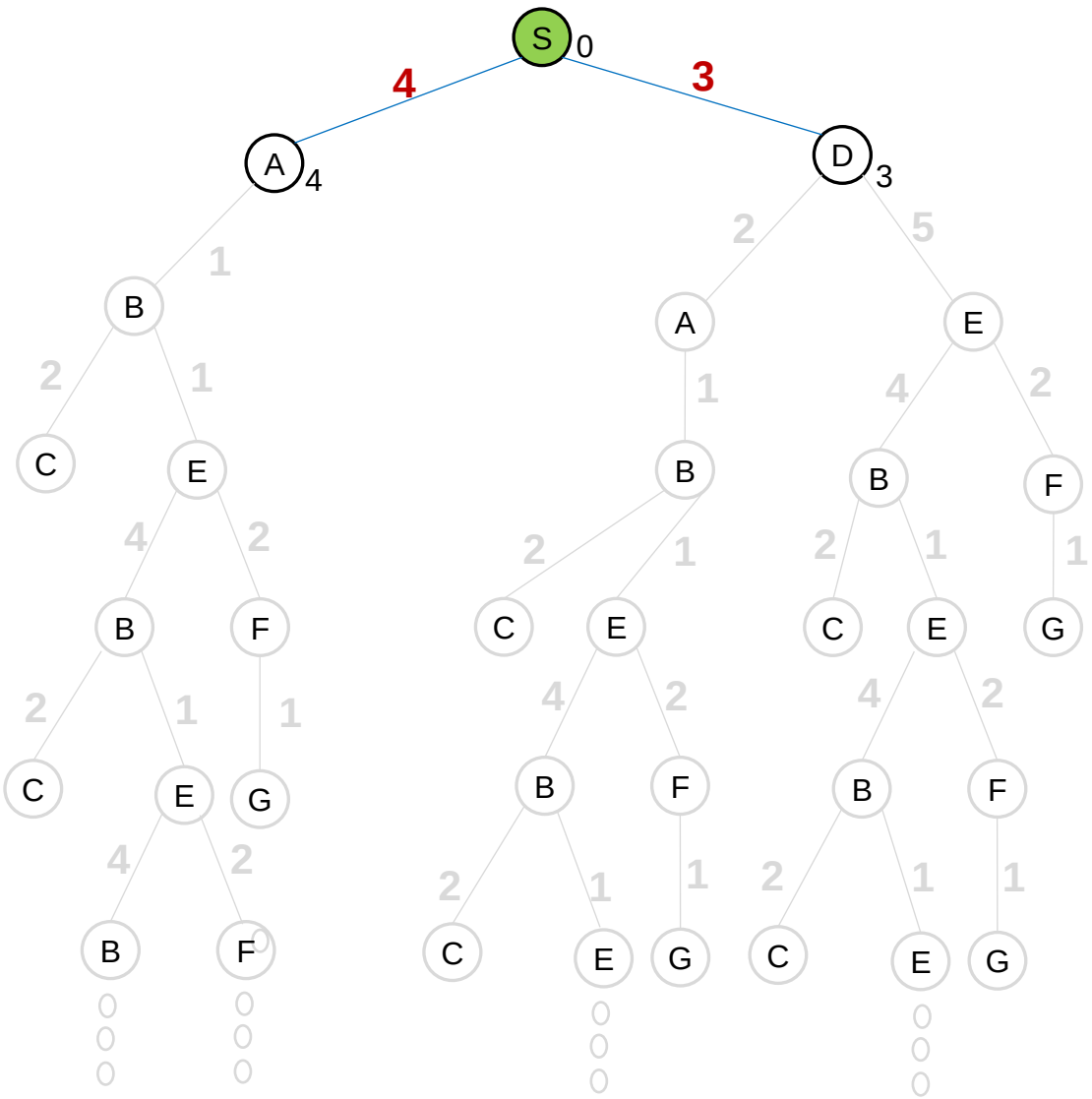
[S_0]



EXPLORED SET: S

FRONTIER (Priority queue)

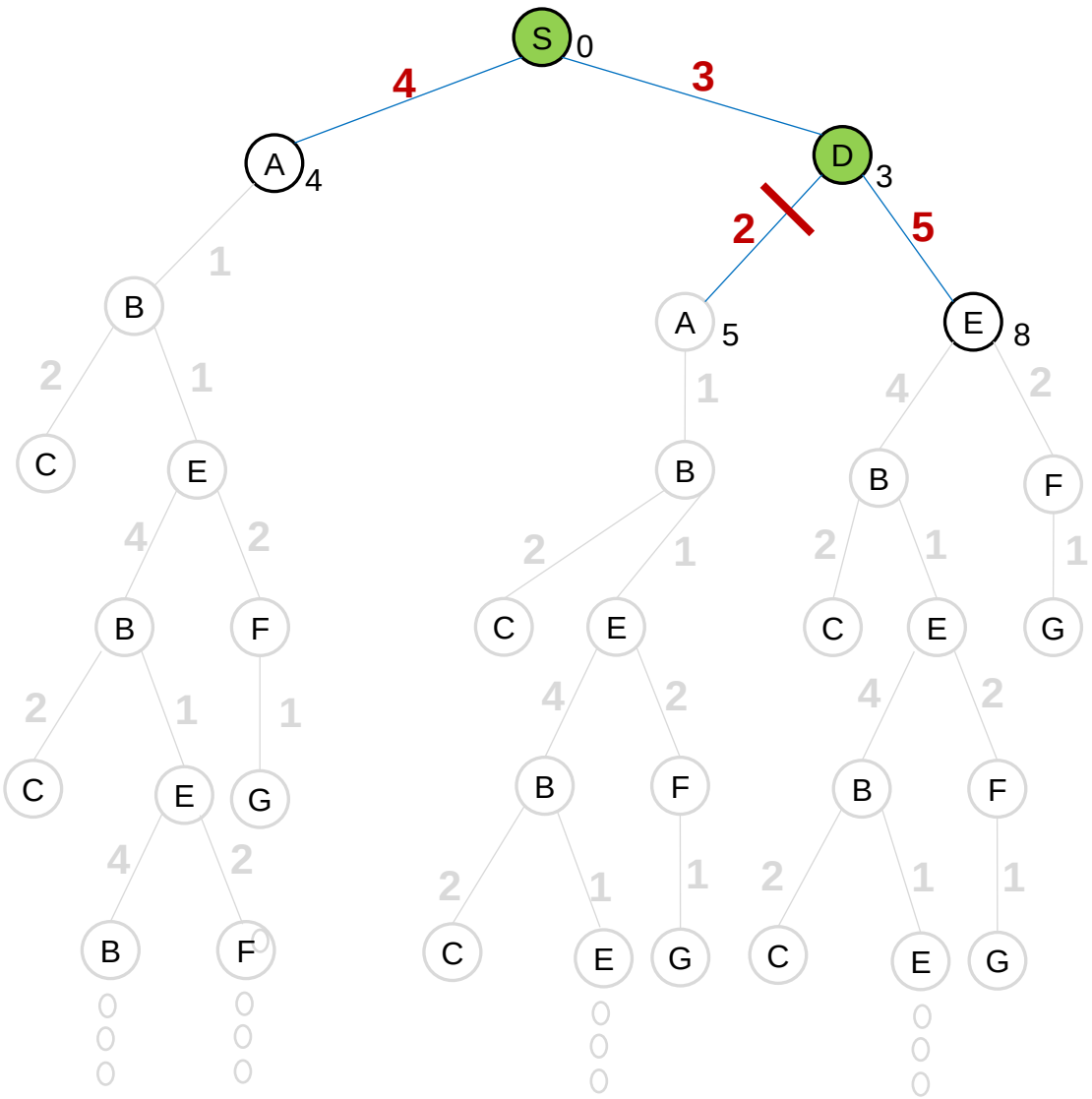
$[S_0]$
$[D_3, A_4]$



EXPLORED SET: S, D

FRONTIER (Priority queue)

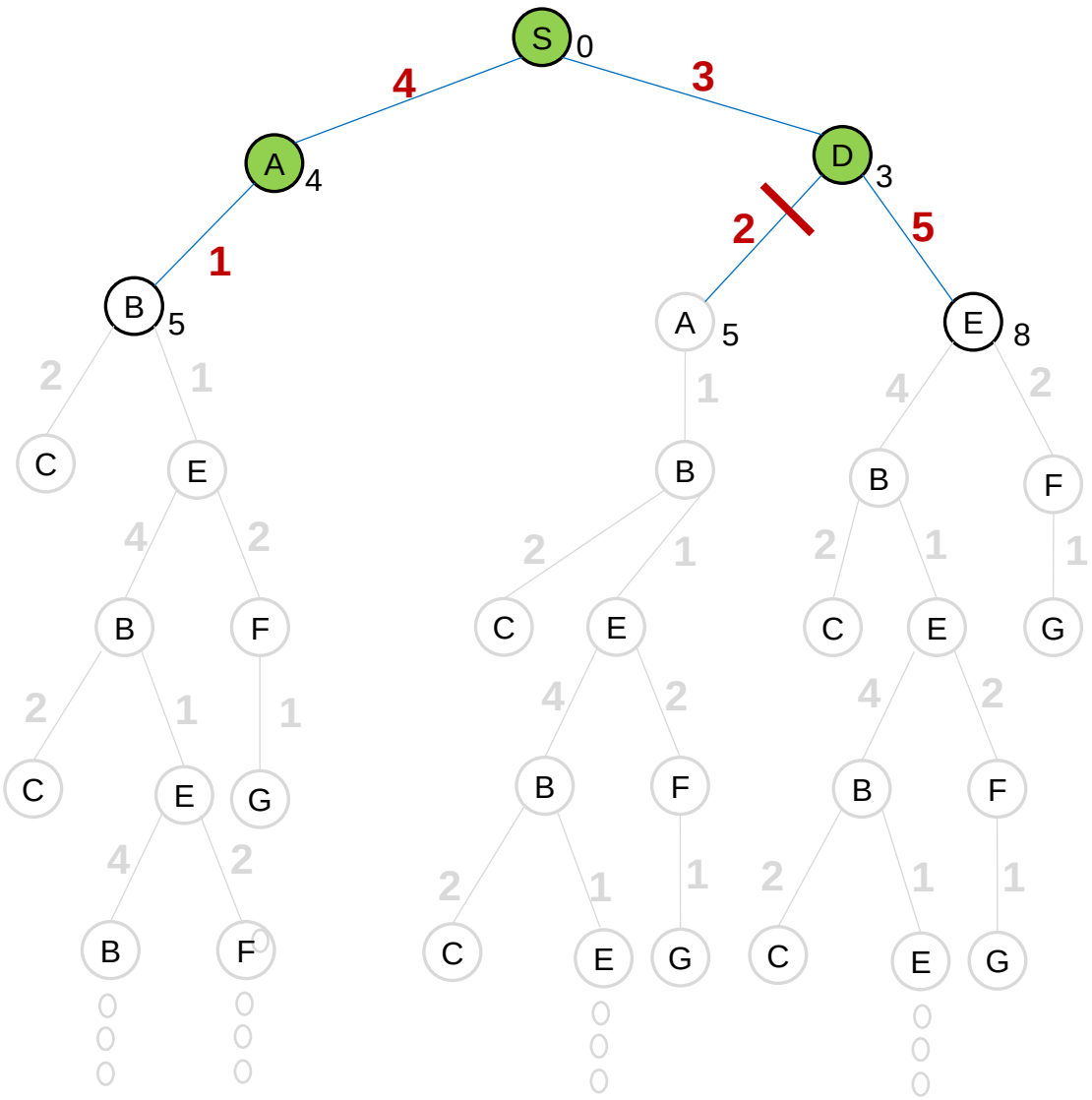
[S_0]
[D_3,A_4]
[A_4,E_8]



EXPLORED SET: S, D, A

FRONTIER (Priority queue)

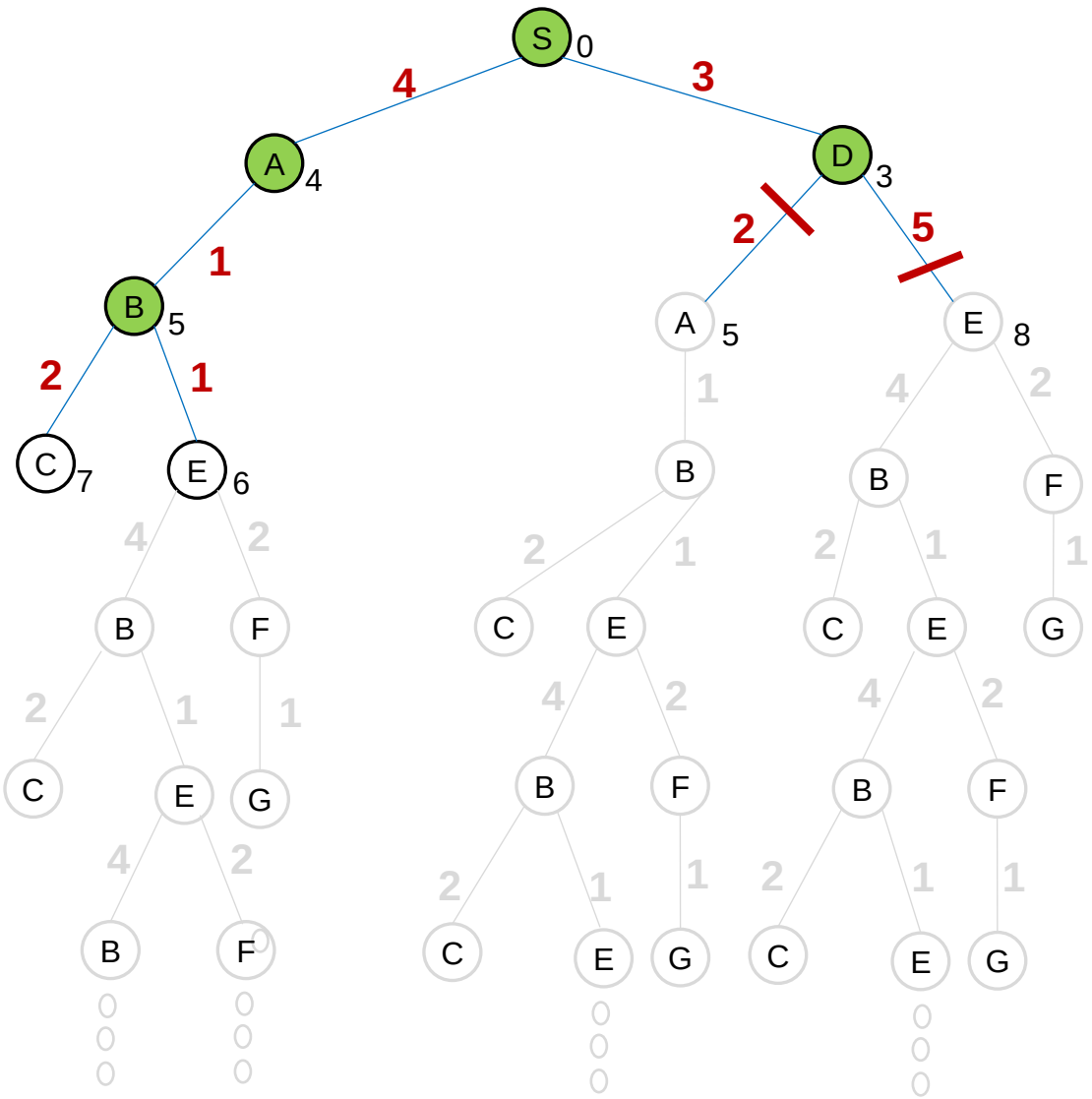
[S_0]
[D_3,A_4]
[A_4,E_8]
[B_5,E_8]



EXPLORED SET: S, D, A, B

FRONTIER (Priority queue)

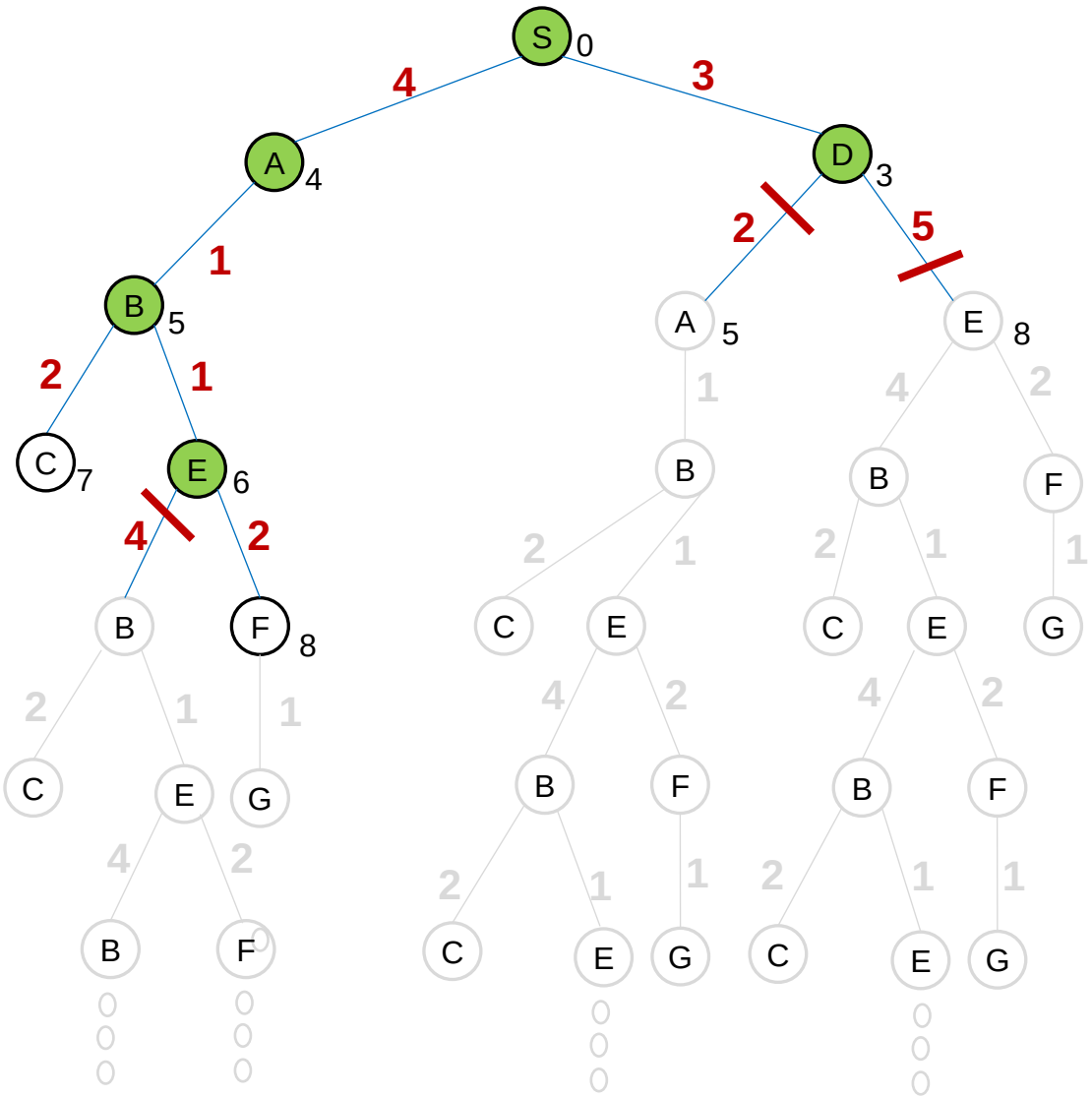
[S_0]
[D_3,A_4]
[A_4,E_8]
[B_5,E_8]
[E_6,C_7]



EXPLORED SET: S, D, A, B, E

FRONTIER (Priority queue)

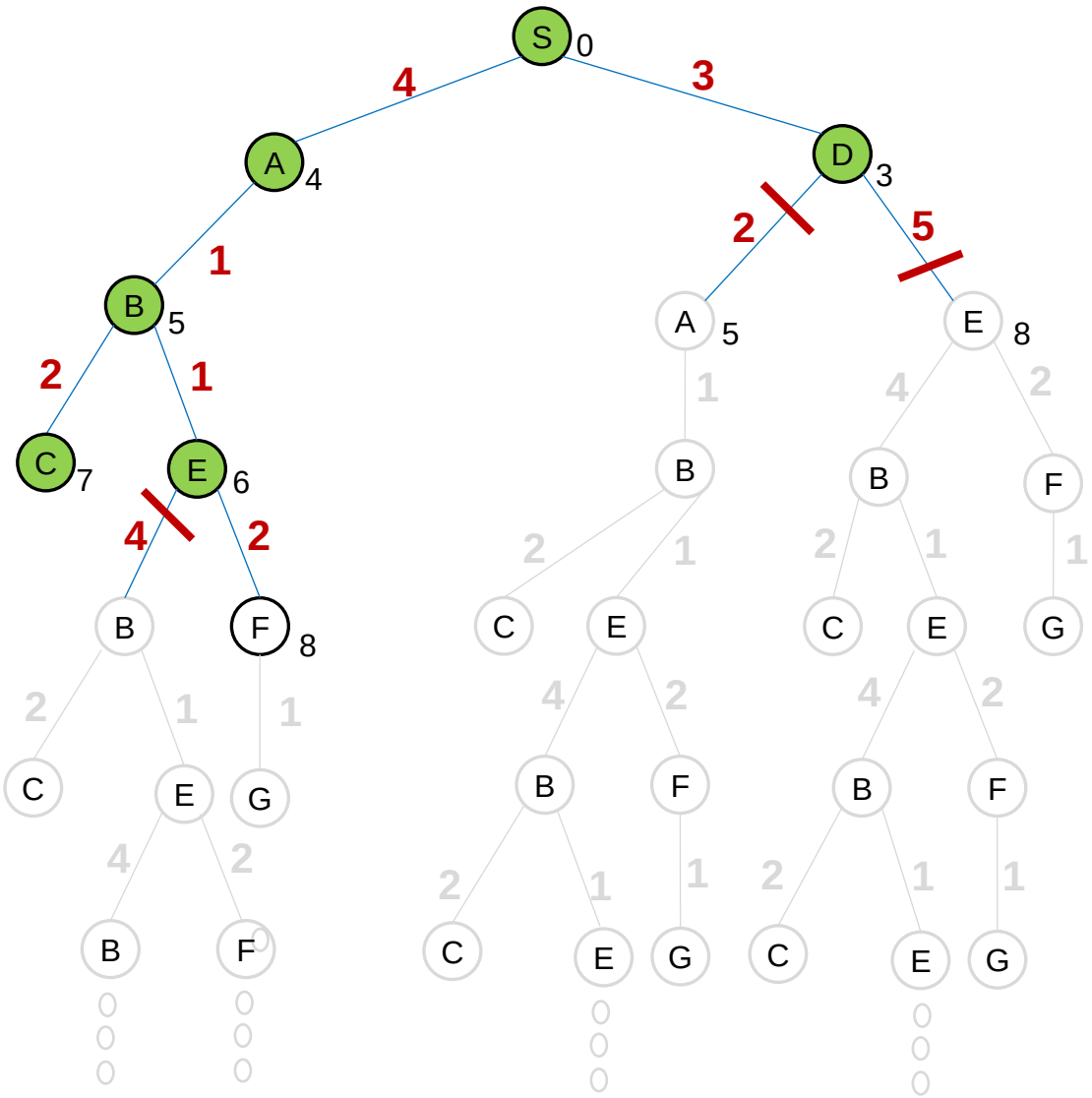
[S_0]
[D_3,A_4]
[A_4,E_8]
[B_5,E_8]
[E_6,C_7]
[C_7,F_8]



EXPLORED SET: S, D, A, B, E, C

FRONTIER (Priority queue)

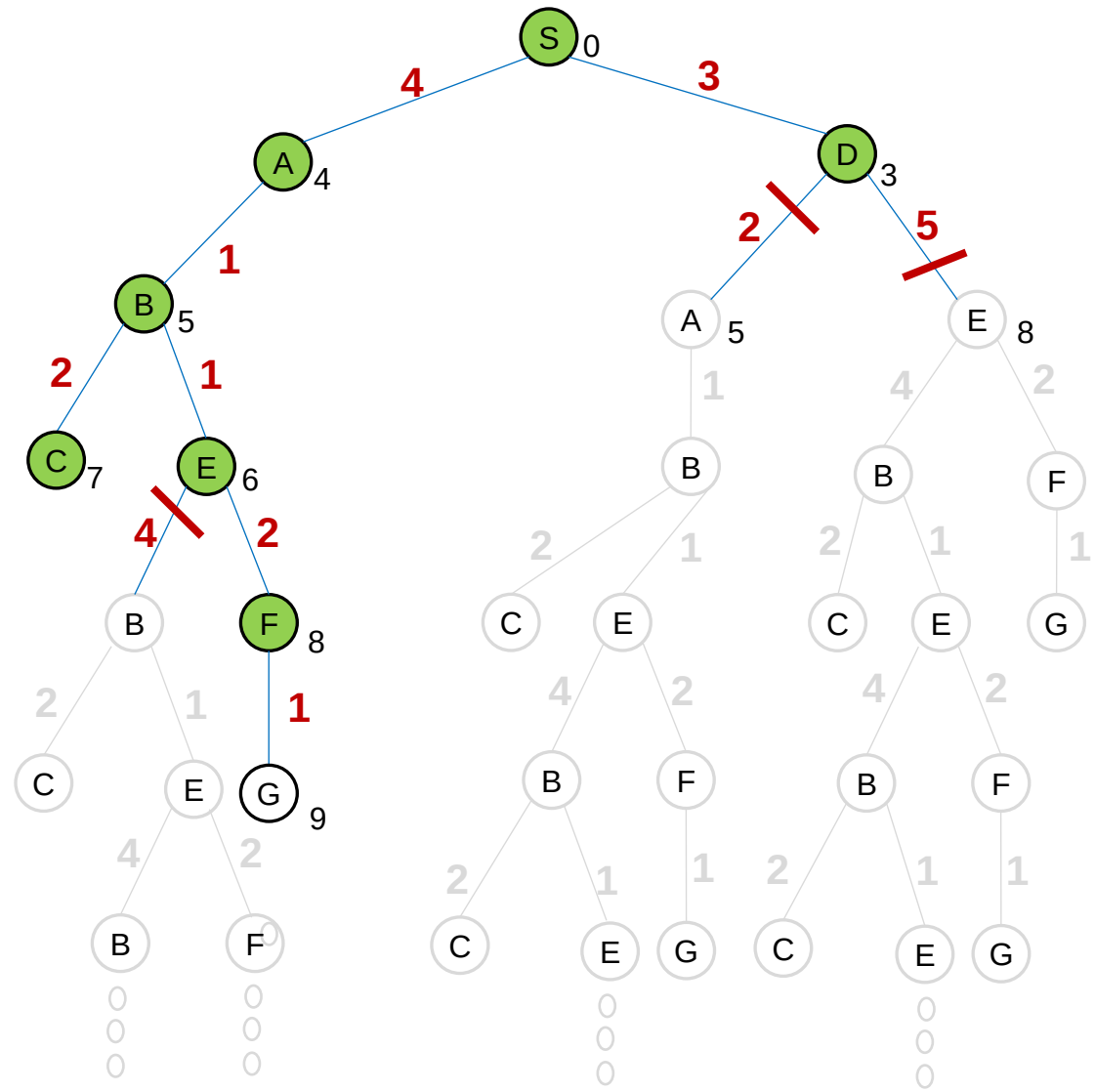
[S_0]
[D_3,A_4]
[A_4,E_8]
[B_5,E_8]
[E_6,C_7]
[C_7,F_8]
[F_8]



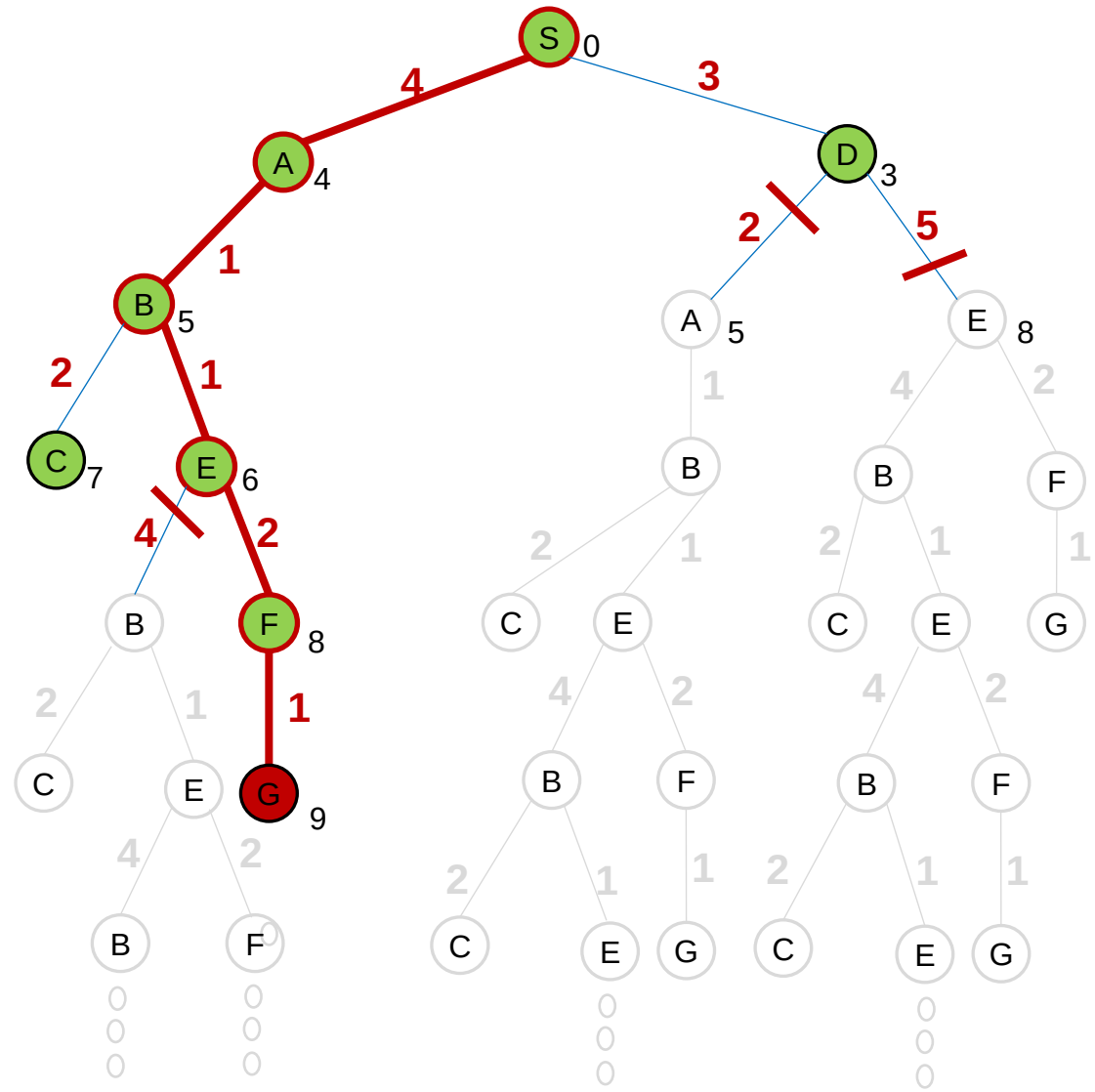
EXPLORED SET: S, D, A, B, E, C, F

FRONTIER (Priority queue)

[S_0]
[D_3,A_4]
[A_4,E_8]
[B_5,E_8]
[E_6,C_7]
[C_7,F_8]
[F_8]
[G_9]



EXPLORED SET: S, D, A, B, E, C, F



FRONTIER (Priority queue)

[S_0]
[D_3,A_4]
[A_4,E_8]
[B_5,E_8]
[E_6,C_7]
[C_7,F_8]
[F_8]
[G_9]
[ ]

UCS={S,A,B,E,F,G}

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Uniform-cost search algorithm's performance?

- Completeness is guaranteed provided the cost of every step exceeds some small positive constant ε. Otherwise, it may get stuck in an infinite loop. Also b should be finite.
- Uniform-cost search is optimal in general.
- Let C^* be the cost of the optimal solution and assume that every action costs at least ε. Then the algorithm's worst-case time and space complexity is:

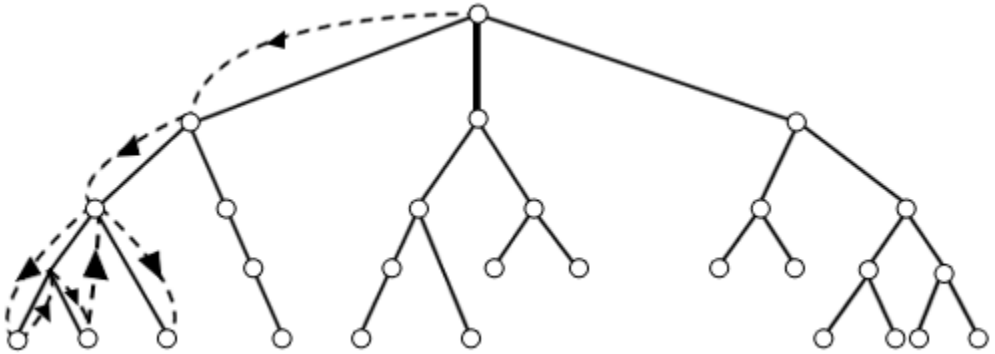
$$O\left(b^{1+\lfloor C^*/\varepsilon \rfloor}\right)$$

which can be much greater than $O\left(b^d\right)$.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

3. Depth-First Search

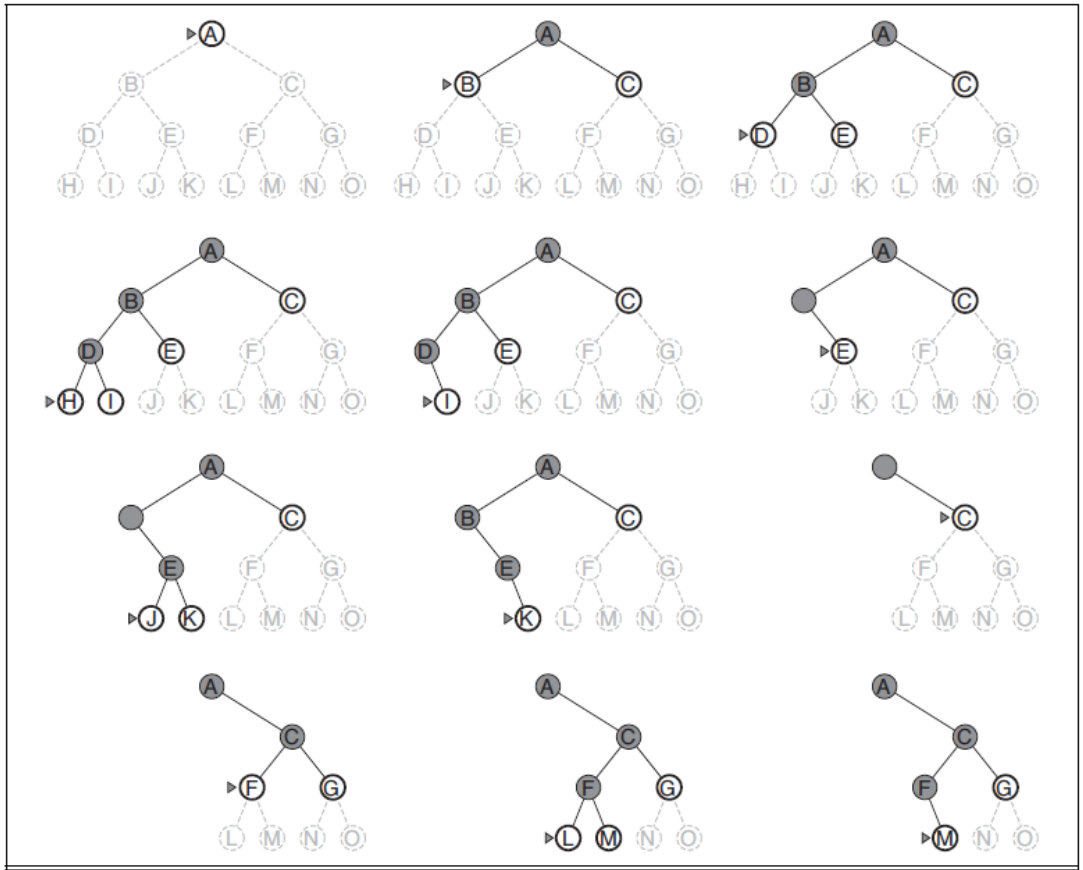


Depth-first search always expands the deepest node in the current frontier of the search tree.

We use a LIFO queue for determining the search sequence of nodes.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Depth-first search on a binary tree. The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory. Nodes at depth 3 have no successors and M is the only goal node.

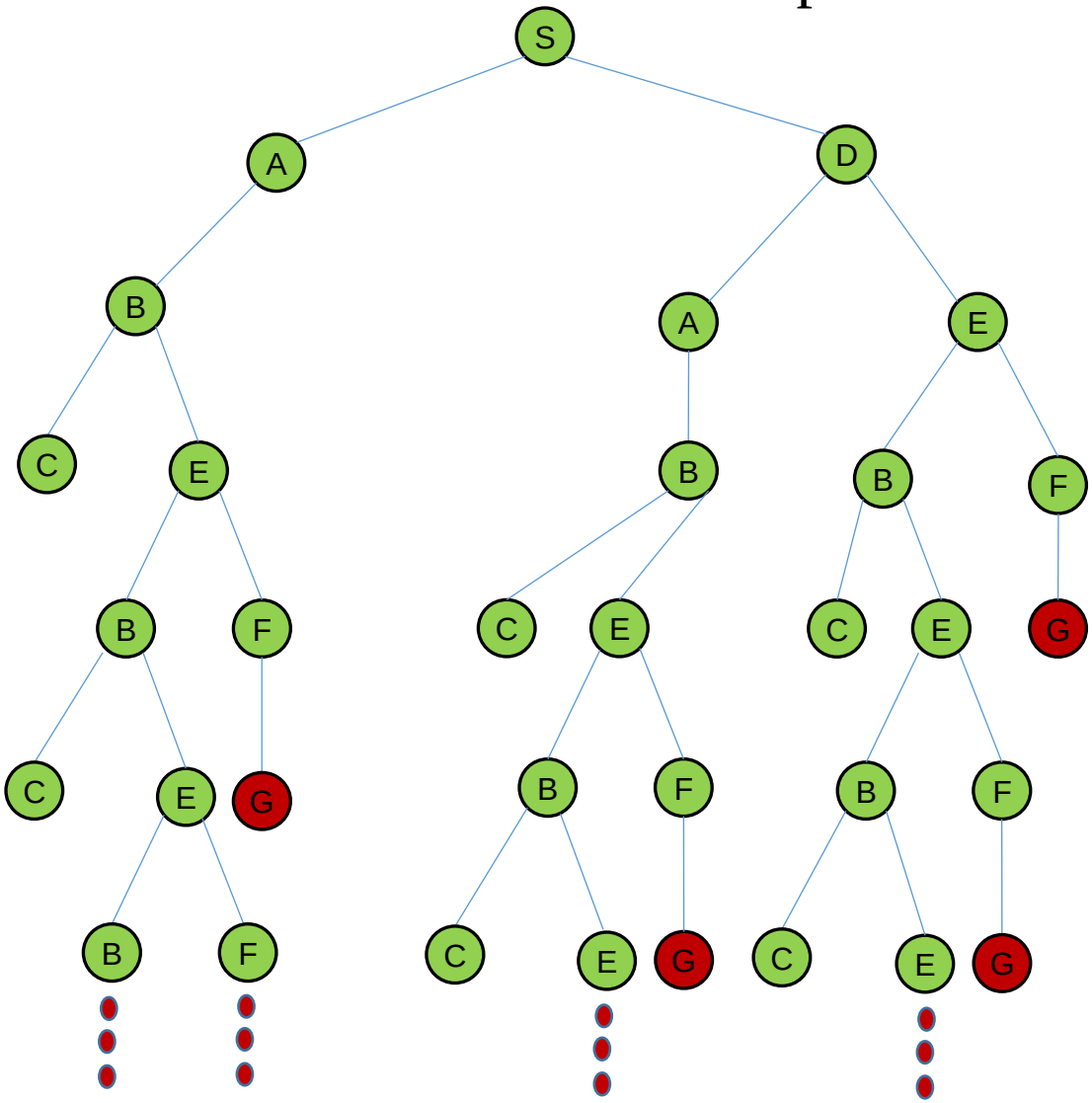
Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The tree-search algorithm for depth-first search is given below.

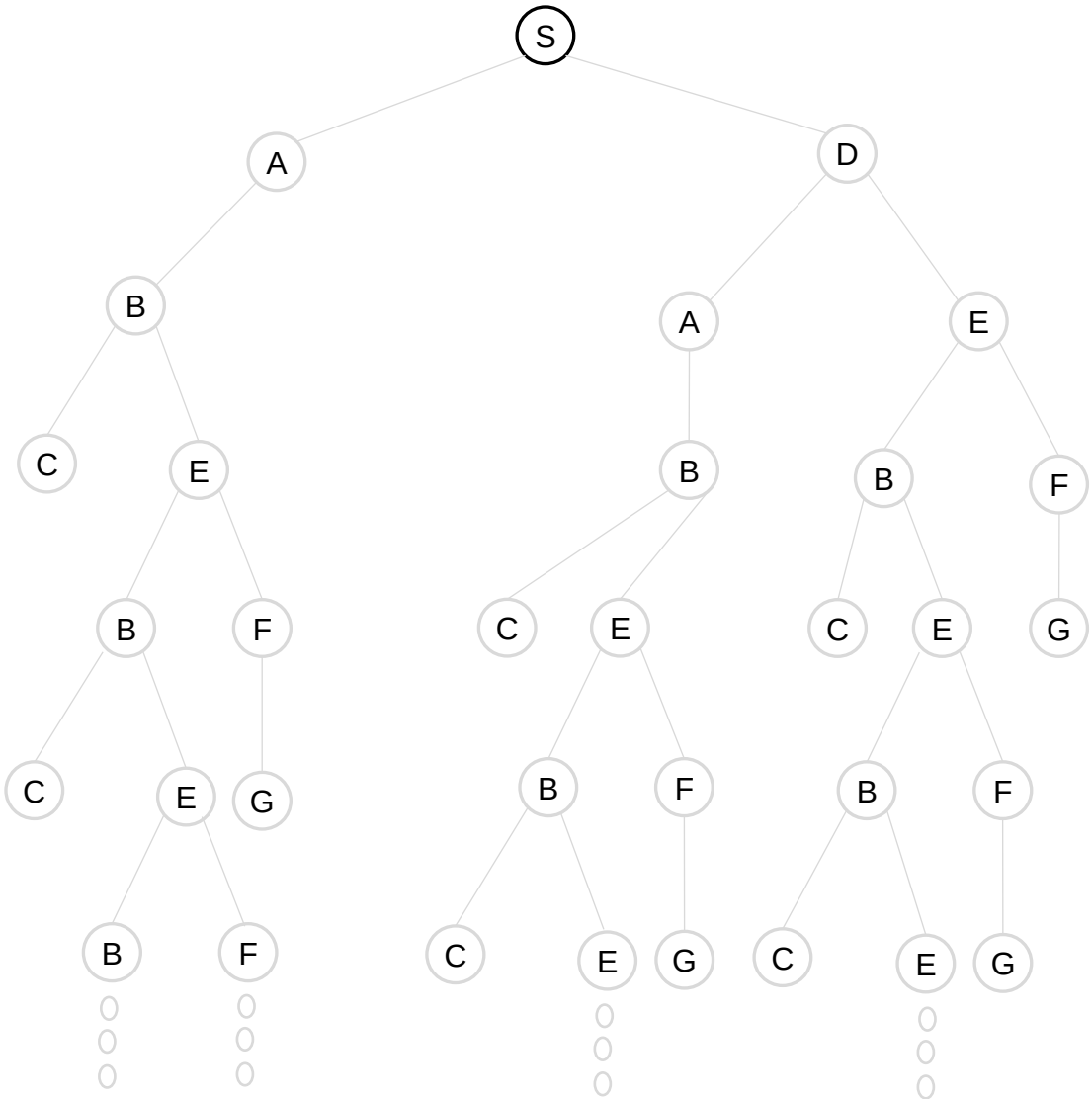
```
function DEPTH-FIRST-SEARCH-TREE(problem) returns a solution, or failure
  - node ← a node with STATE = problem.INITIAL-STATE(),
                                PATH-COST = 0
  - if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  - frontier  ← a LIFO queue with the root node
  - loop do
     - if EMPTY( frontier) then return failure
     - node←POP( frontier )  /* chooses the deepest node in frontier */
     - for each action in problem.ACTIONS(node.STATE) do
          - child ←CHILD-NODE(problem, node, action)
          - if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
          - frontier ←INSERT(child , frontier )
```

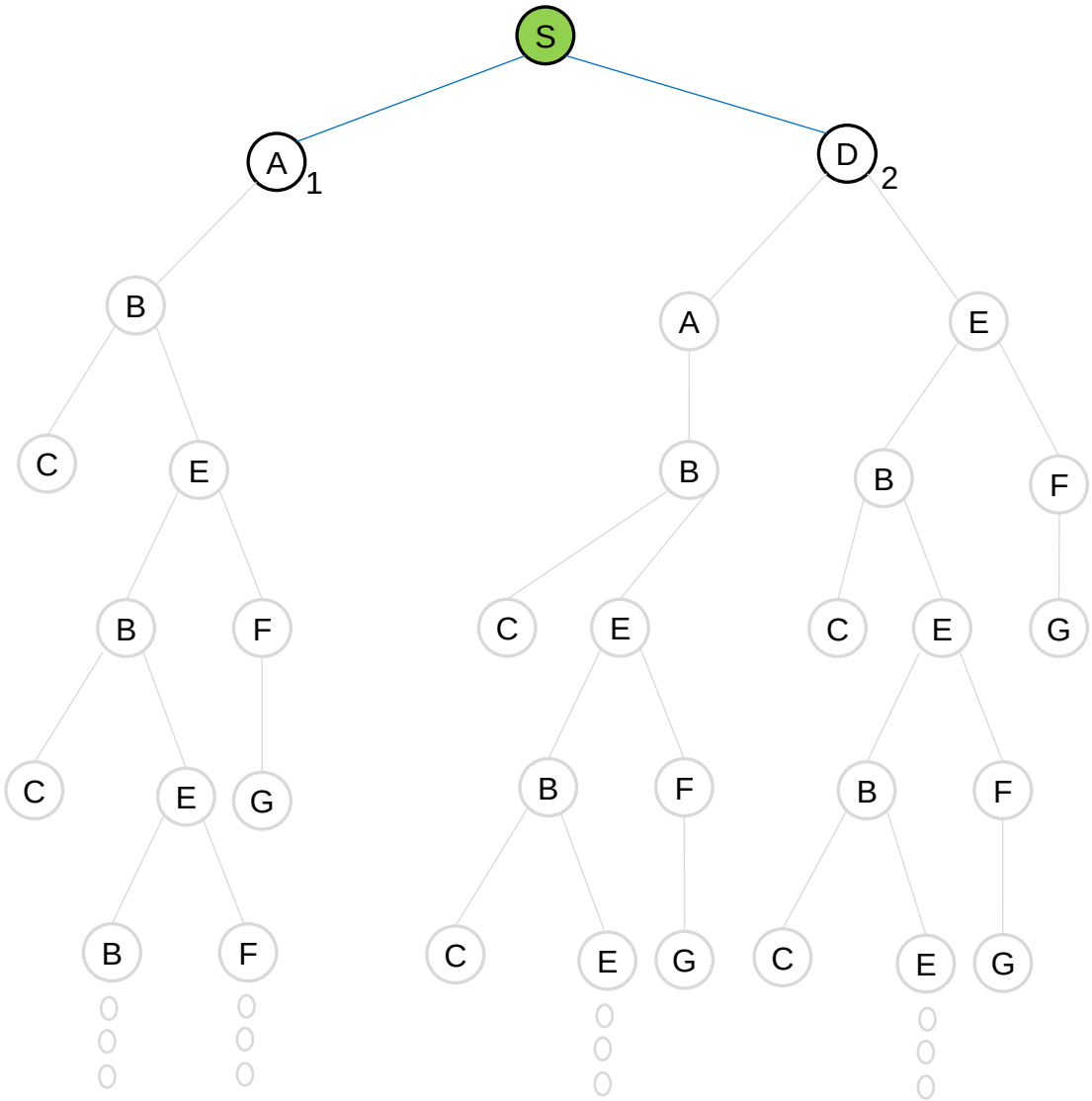
Example: Determine a path from the initial state to the goal state by using the tree search of the depth-first search algorithm. Show the frontier for each step.



FRONTIER (LIFO)

[**S**]

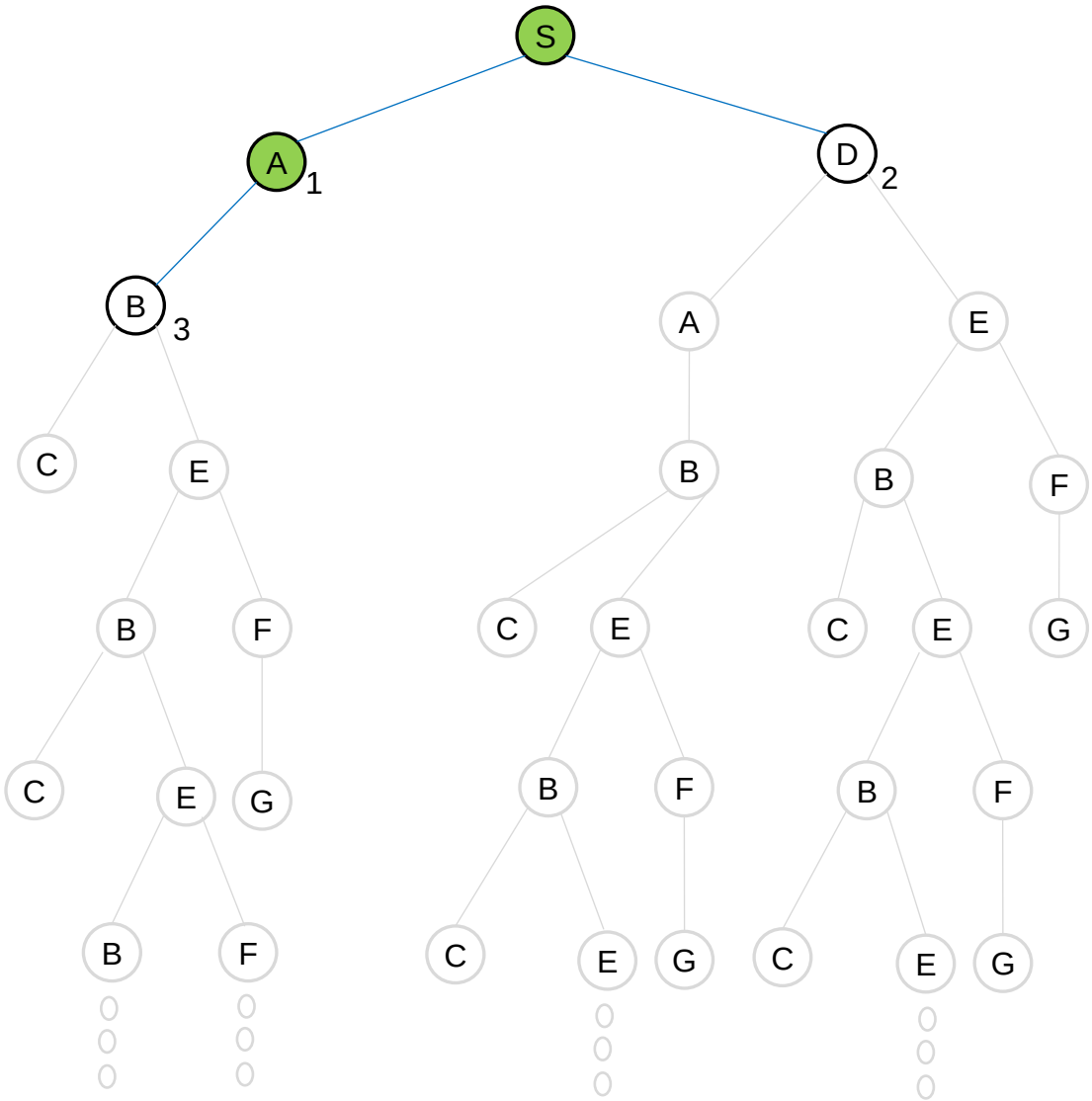




FRONTIER (LIFO)

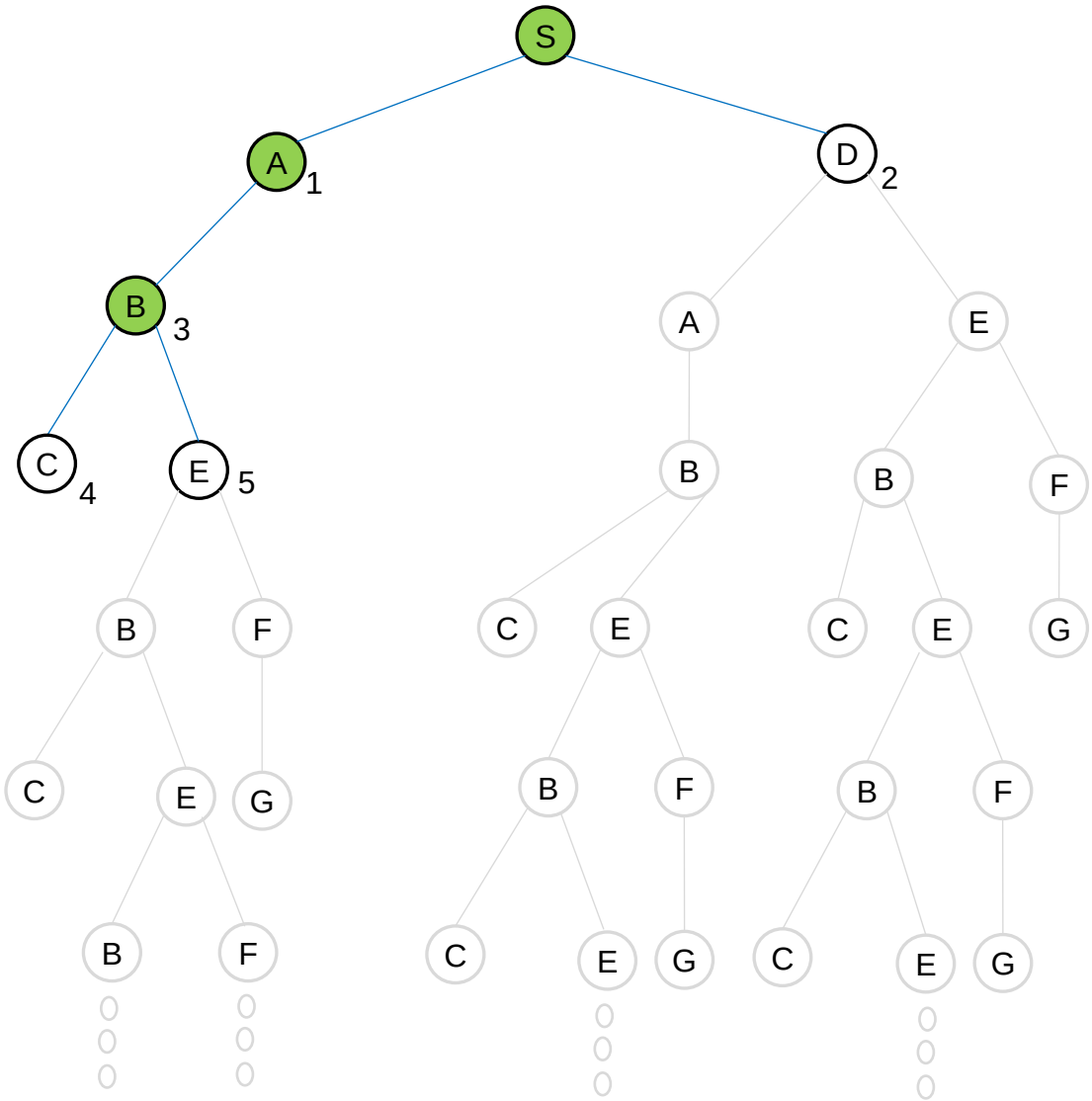
[**S**]

[**A_1**,D_2]



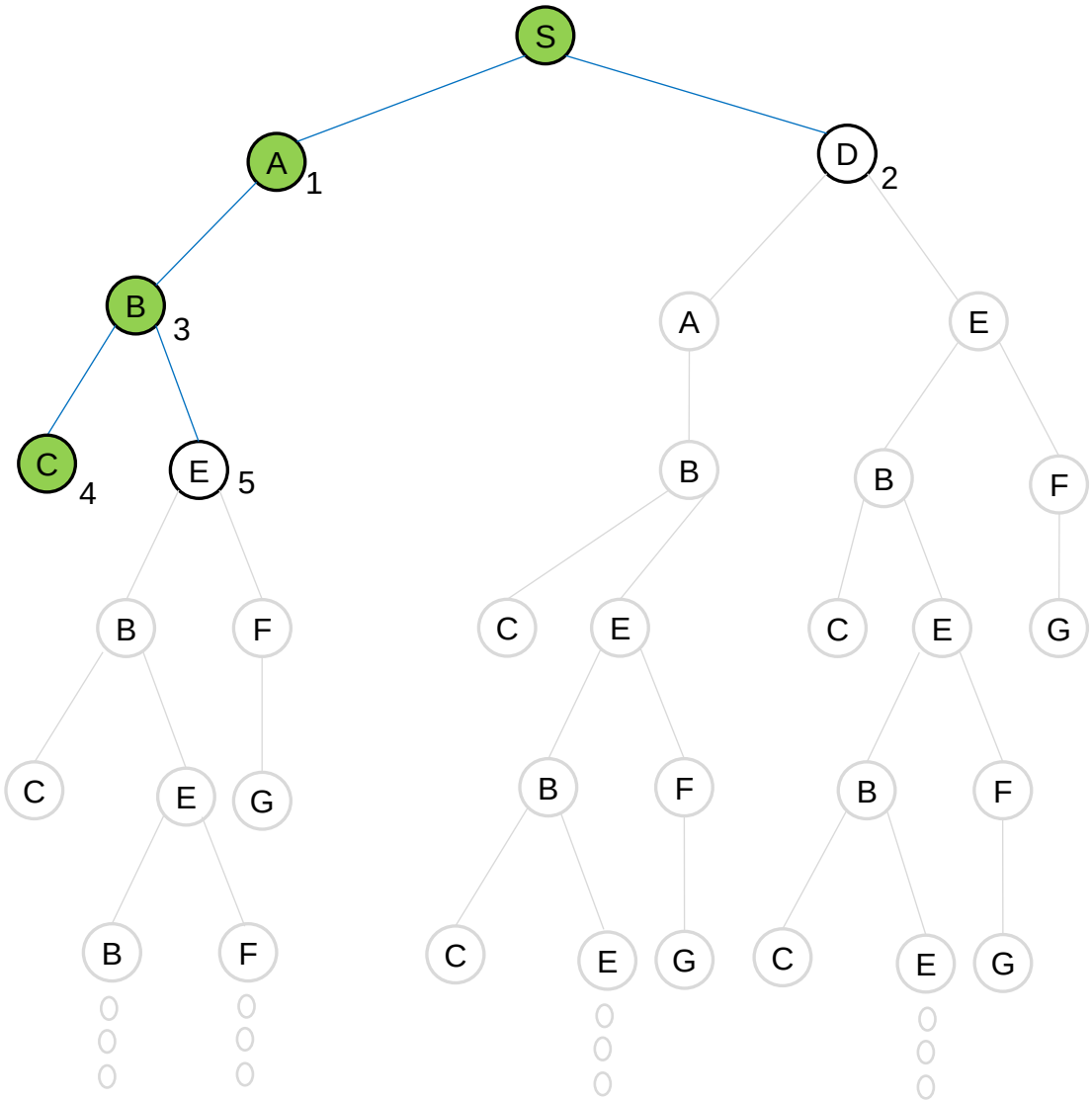
FRONTIER (LIFO)

[**S**]
[**A_1**,D_2]
[**B_3**,D_2]



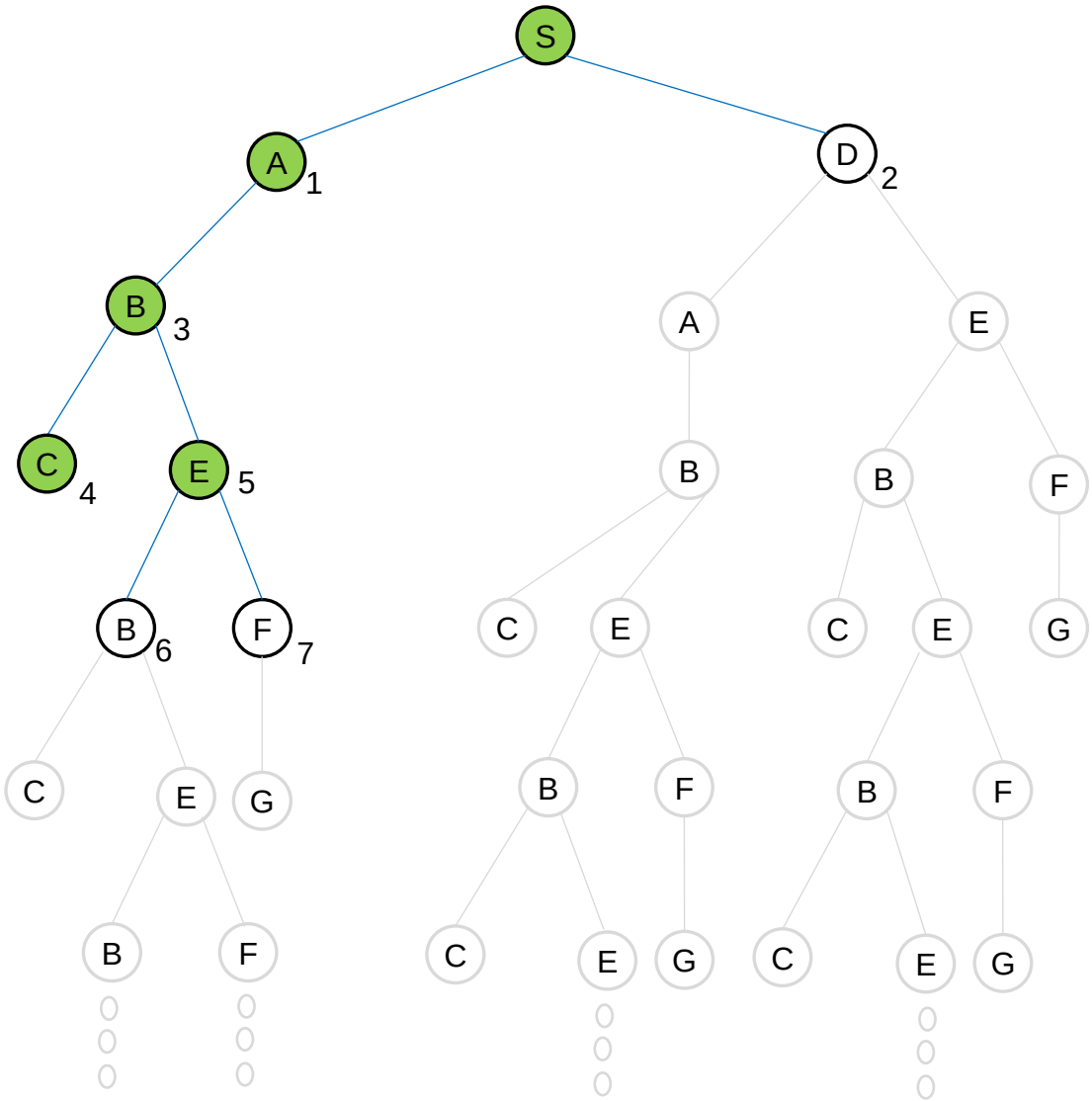
FRONTIER (LIFO)

[**S**]
[**A_1**,D_2]
[**B_3**,D_2]
[**C_4**,E_5,D_2]



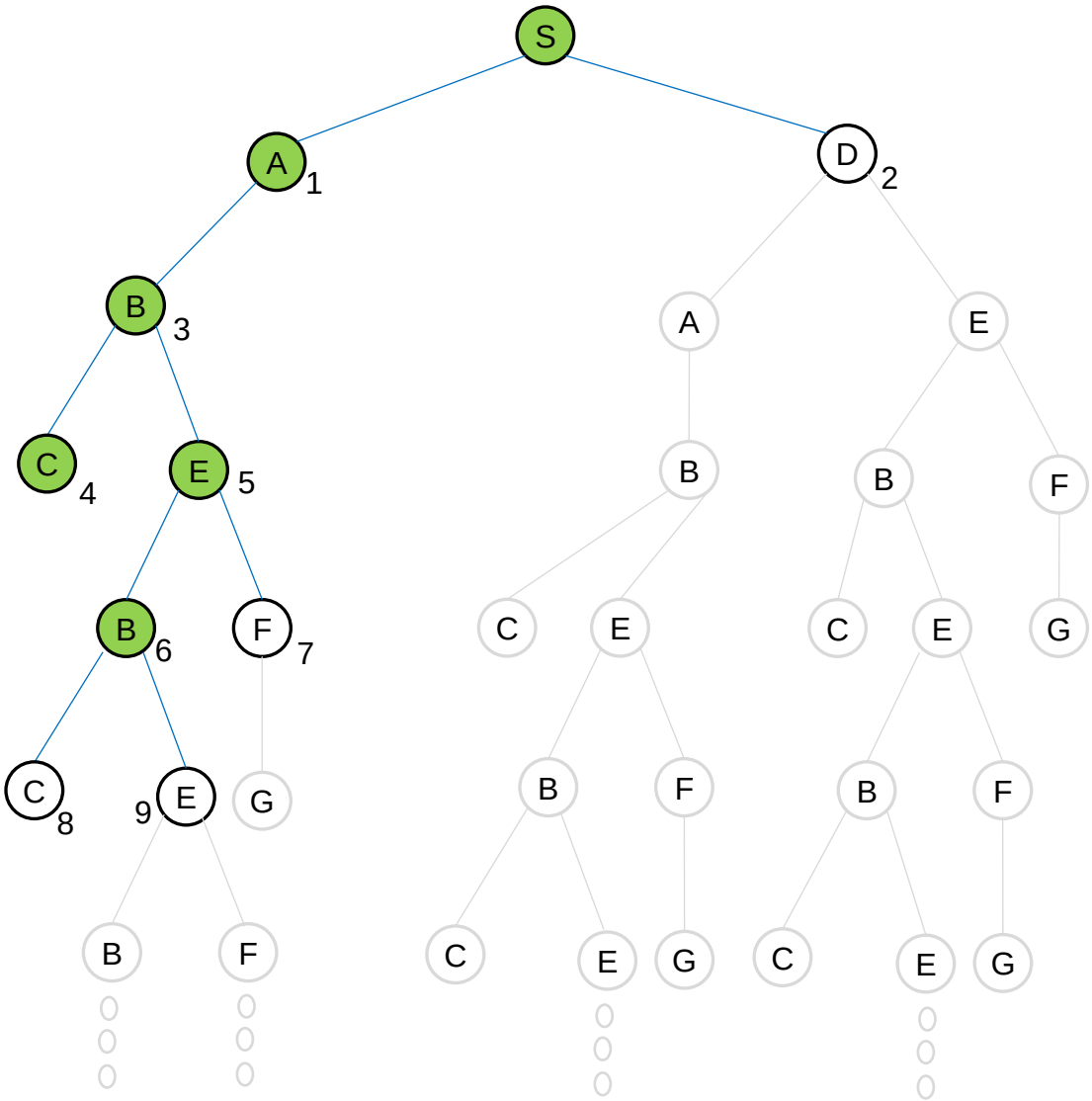
FRONTIER (LIFO)

[**S**]
[**A_1**,D_2]
[**B_3**,D_2]
[**C_4**,E_5,D_2]
[**E_5**,D_2]



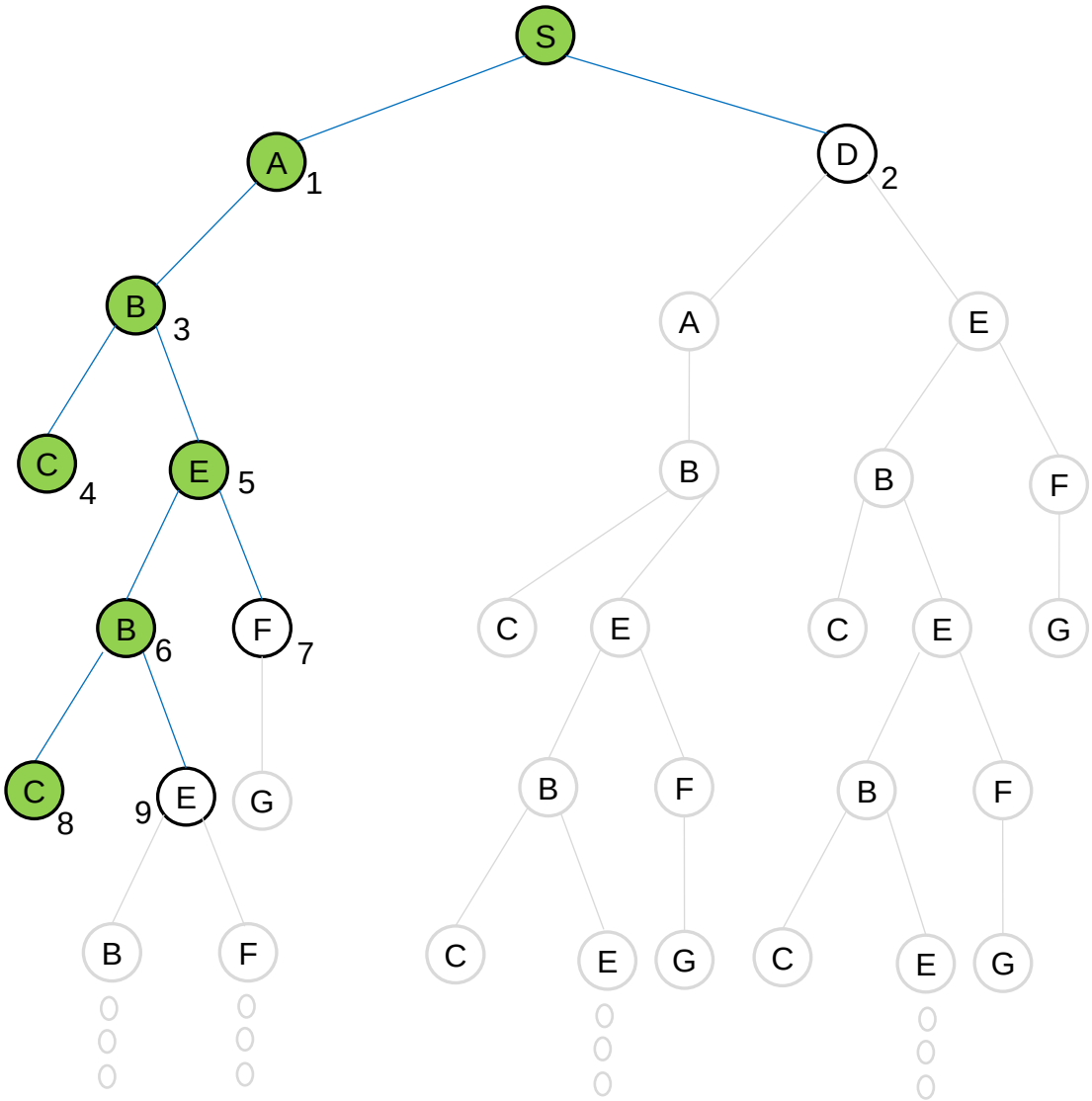
FRONTIER (LIFO)

[**S**]
[**A_1**,D_2]
[**B_3**,D_2]
[**C_4**,E_5,D_2]
[**E_5**,D_2]
[**B_6**,F_7,D_2]



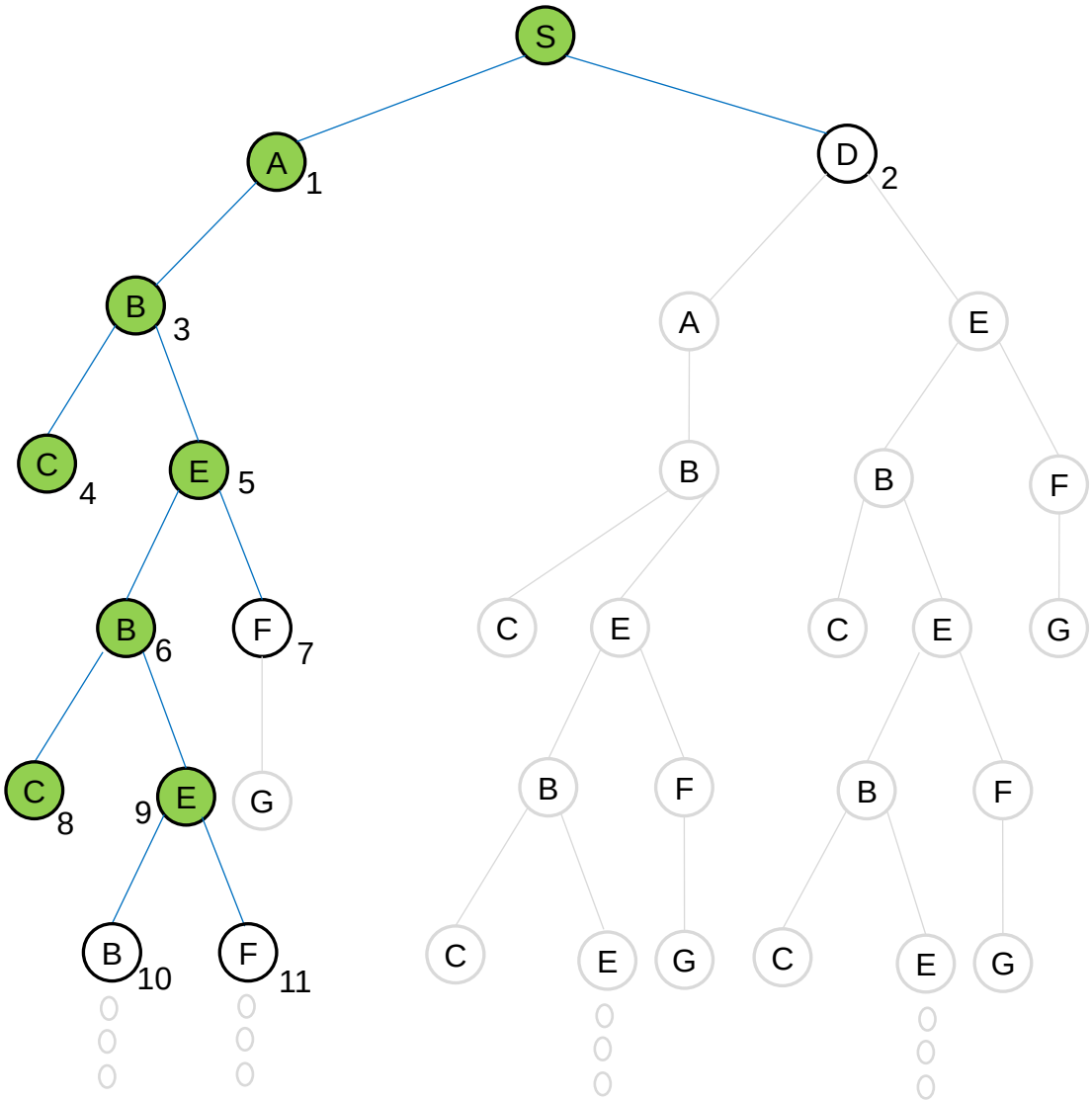
FRONTIER (LIFO)

[**S**]
[**A_1**,D_2]
[**B_3**,D_2]
[**C_4**,E_5,D_2]
[**E_5**,D_2]
[**B_6**,F_7,D_2]
[**C_8**,E_9,F_7,D_2]



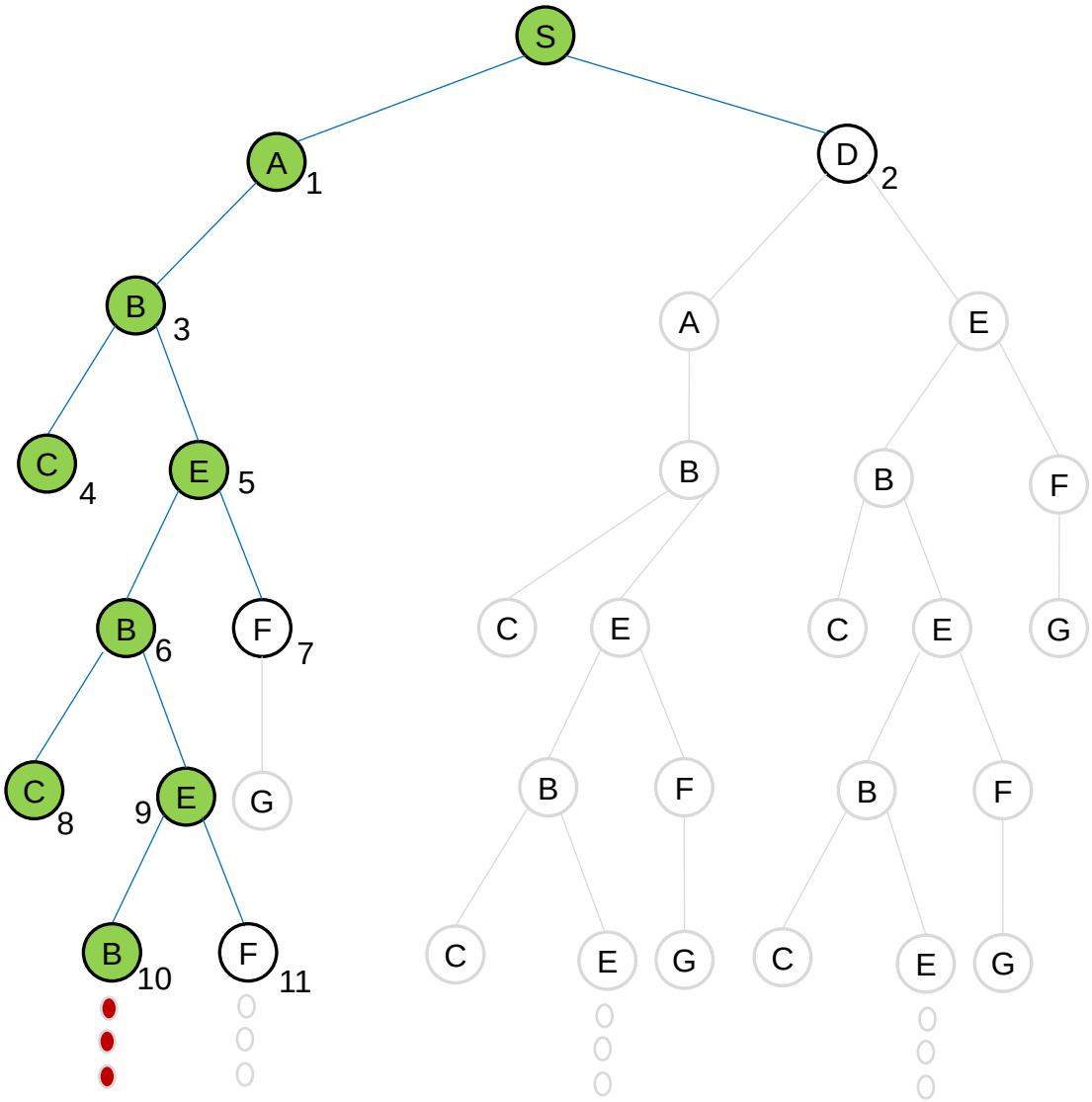
FRONTIER (LIFO)

[**S**]
[**A_1**,D_2]
[**B_3**,D_2]
[**C_4**,E_5,D_2]
[**E_5**,D_2]
[**B_6**,F_7,D_2]
[**C_8**,E_9,F_7,D_2]
[**E_9**,F_7,D_2]



FRONTIER (LIFO)

[**S**]
[**A_1**,D_2]
[**B_3**,D_2]
[**C_4**,E_5,D_2]
[**E_5**,D_2]
[**B_6**,F_7,D_2]
[**C_8**,E_9,F_7,D_2]
[**E_9**,F_7,D_2]
[**B_{10}**,F_{11}, F_7,D_2]



FRONTIER (LIFO)

[**S**]
[$\mathbf{A_1}$,D_2]
[$\mathbf{B_3}$,D_2]
[$\mathbf{C_4}$,E_5,D_2]
[$\mathbf{E_5}$,D_2]
[$\mathbf{B_6}$,F_7,D_2]
[$\mathbf{C_8}$,E_9,F_7,D_2]
[$\mathbf{E_9}$,F_7,D_2]
[$\mathbf{B_{10}}$,F_{11}, F_7,D_2]

Infinite Loop!

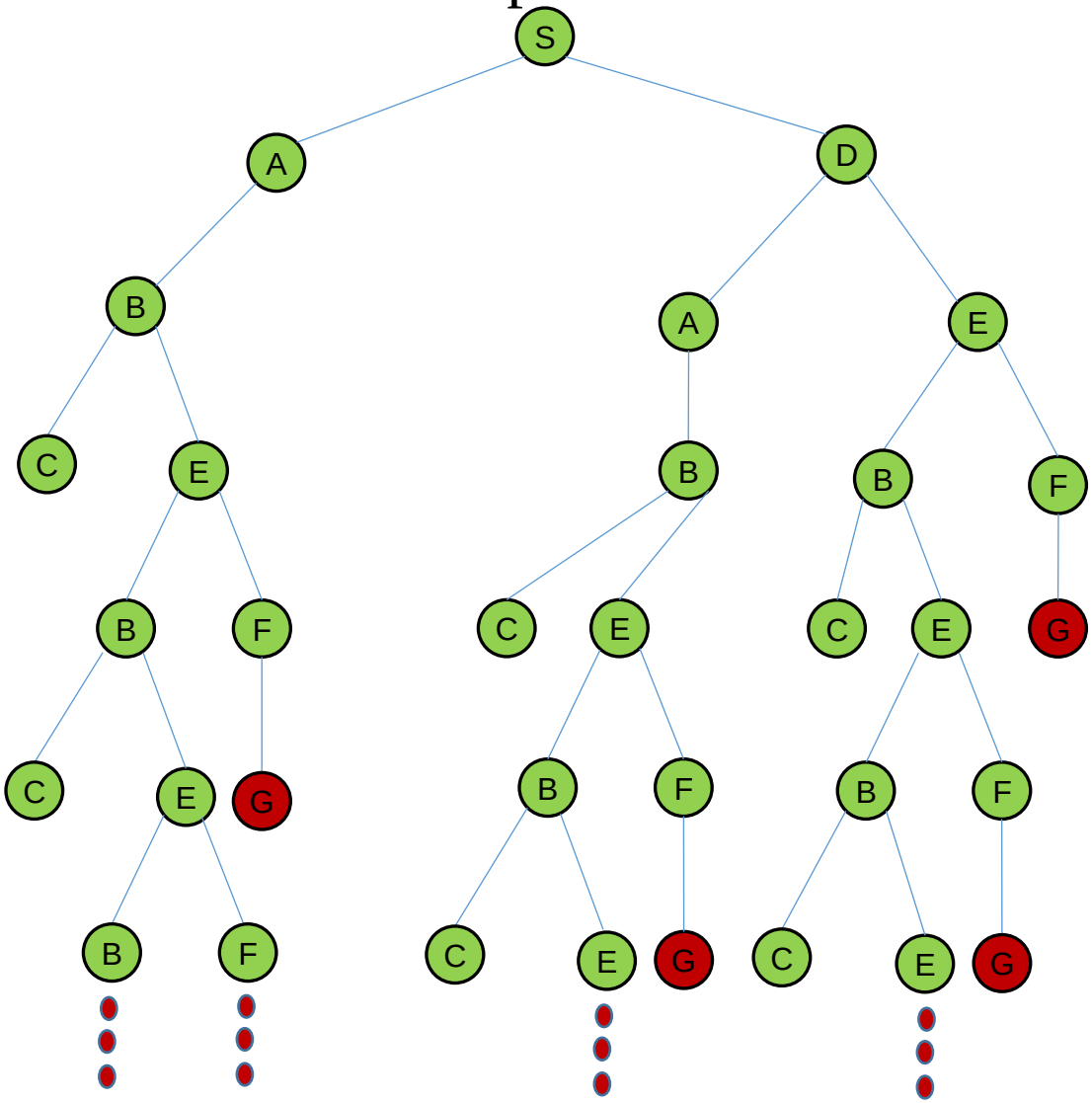
Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The graph-search algorithm for depth-first search is given below.

```
function DEPTH-FIRST-SEARCH-GRAPH(problem) returns a solution, or failure
  - node ← a node with STATE = problem.INITIAL-STATE(),
                               PATH-COST = 0
  - if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  - frontier  ← a LIFO queue with the root node
  - explored ← an empty set
  - loop do
     - if EMPTY( frontier) then return failure
     - node←POP( frontier )  /* chooses the deepest node in frontier */
     - add node.STATE to explored
     - for each action in problem.ACTIONS(node.STATE) do
          - child ←CHILD-NODE(problem, node, action)
          - if child.STATE is not in explored or frontier then
               - if problem.GOAL-TEST(child .STATE) then return SOLUTION(child )
               - frontier ←INSERT(child , frontier )
```

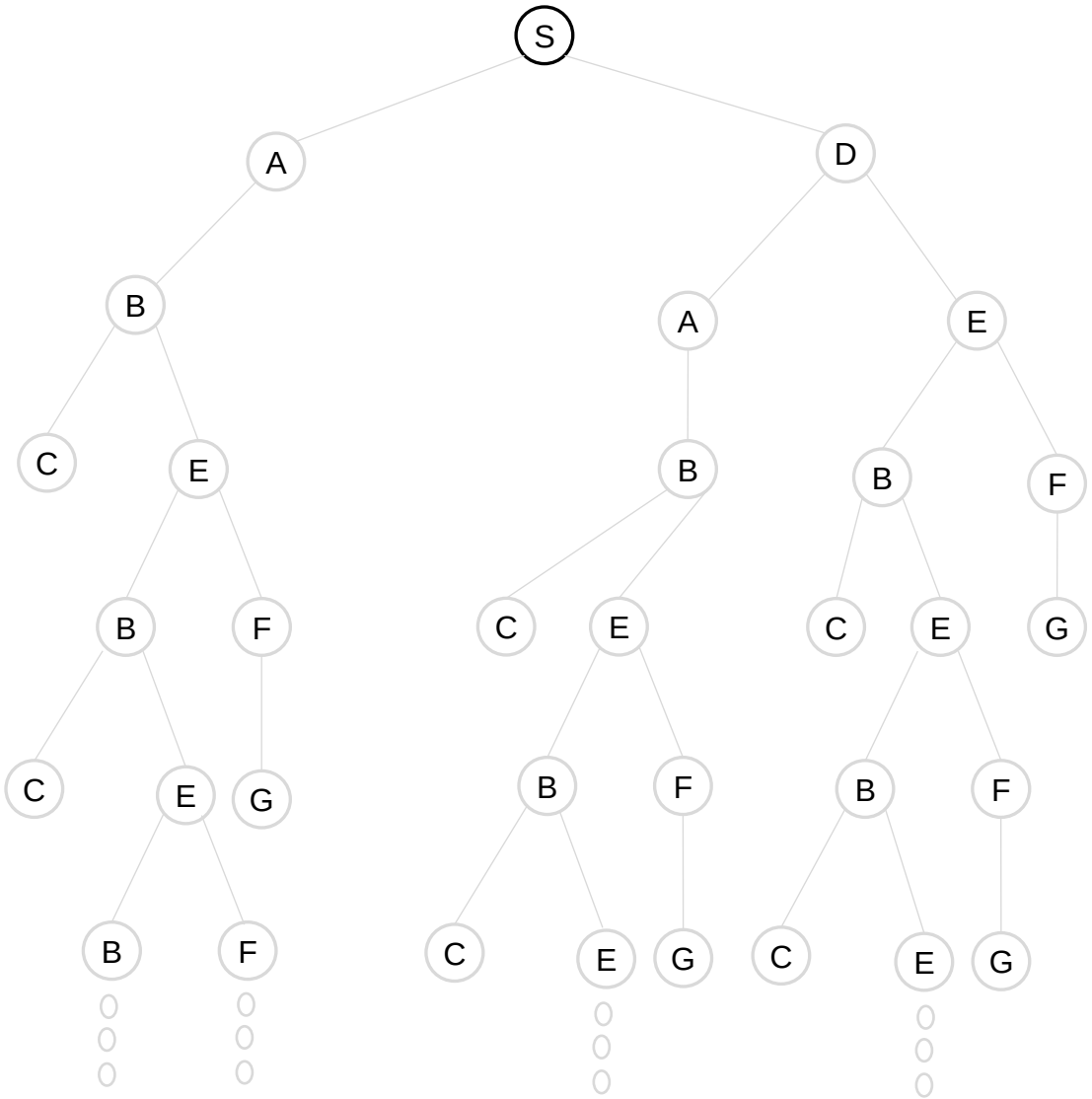
Example: Determine a path from the initial state to the goal state by the graph search of the depth-first search algorithm. Show the frontier for each step.



EXPLORED SET:

FRONTIER (LIFO)

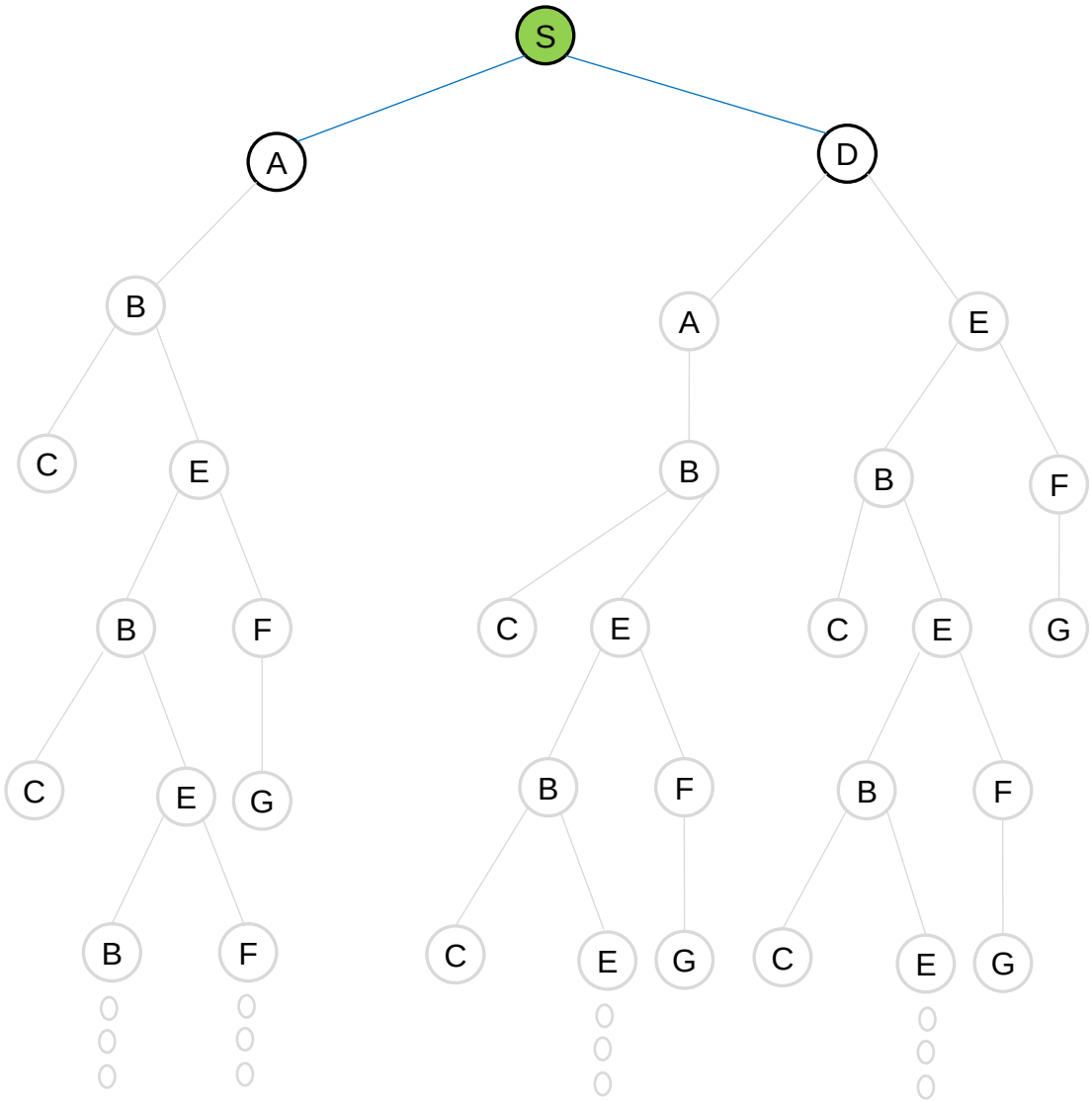
[S]



EXPLORED SET: S

FRONTIER (LIFO)

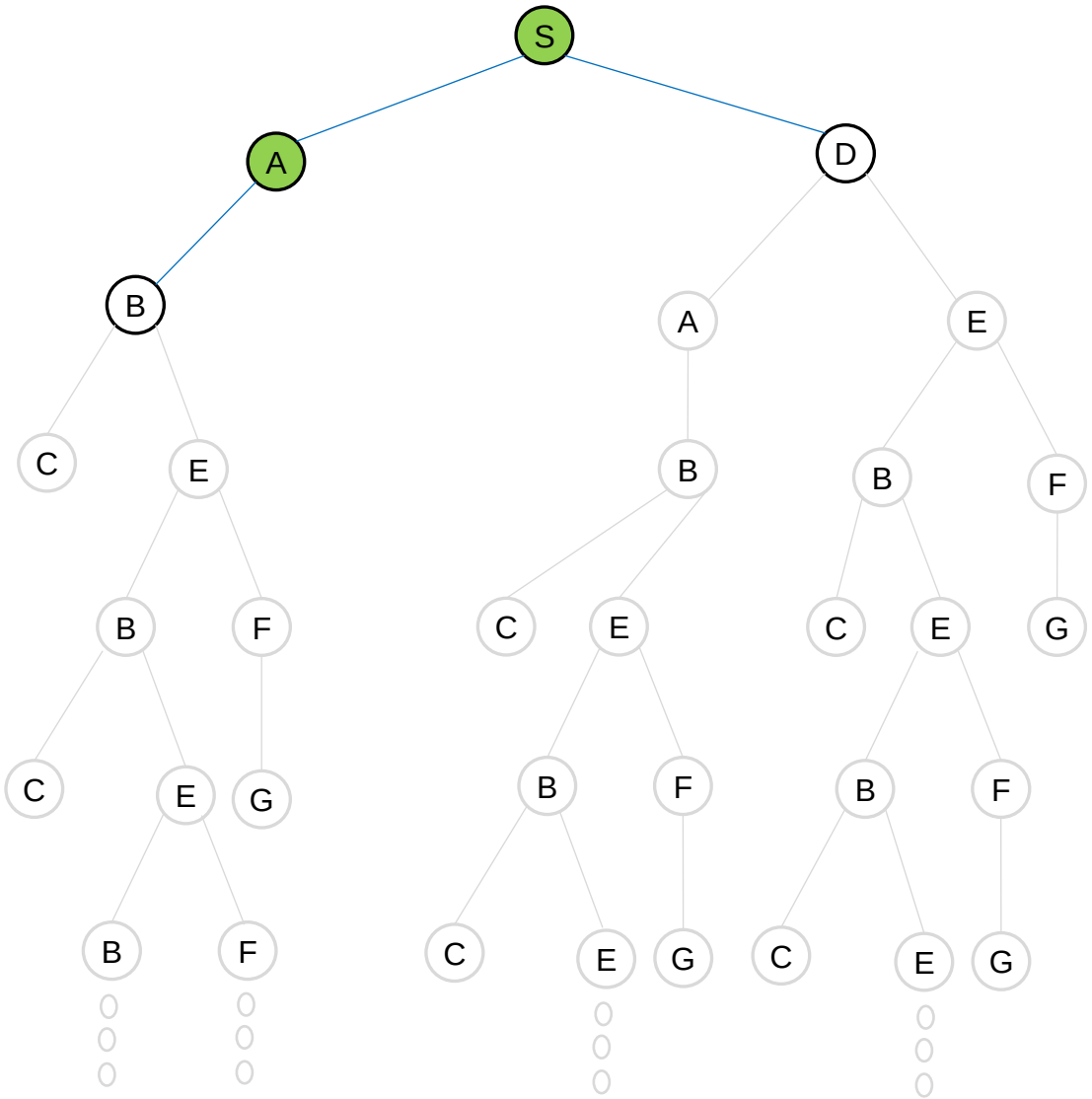
[S]
[A,D]



EXPLORED SET: S, A

FRONTIER (LIFO)

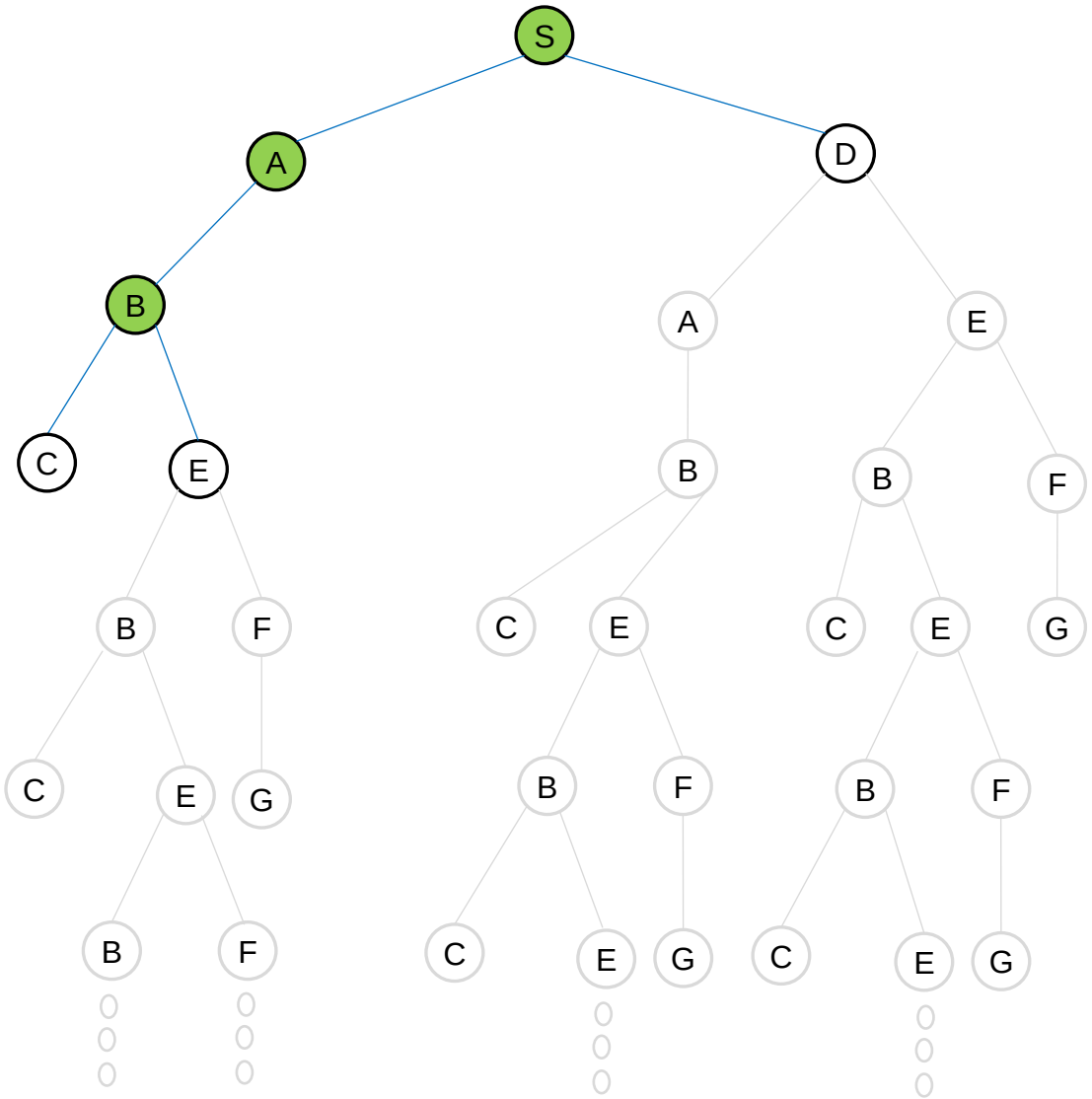
[S]
[A,D]
[B,D]



EXPLORED SET: S, A, B

FRONTIER (LIFO)

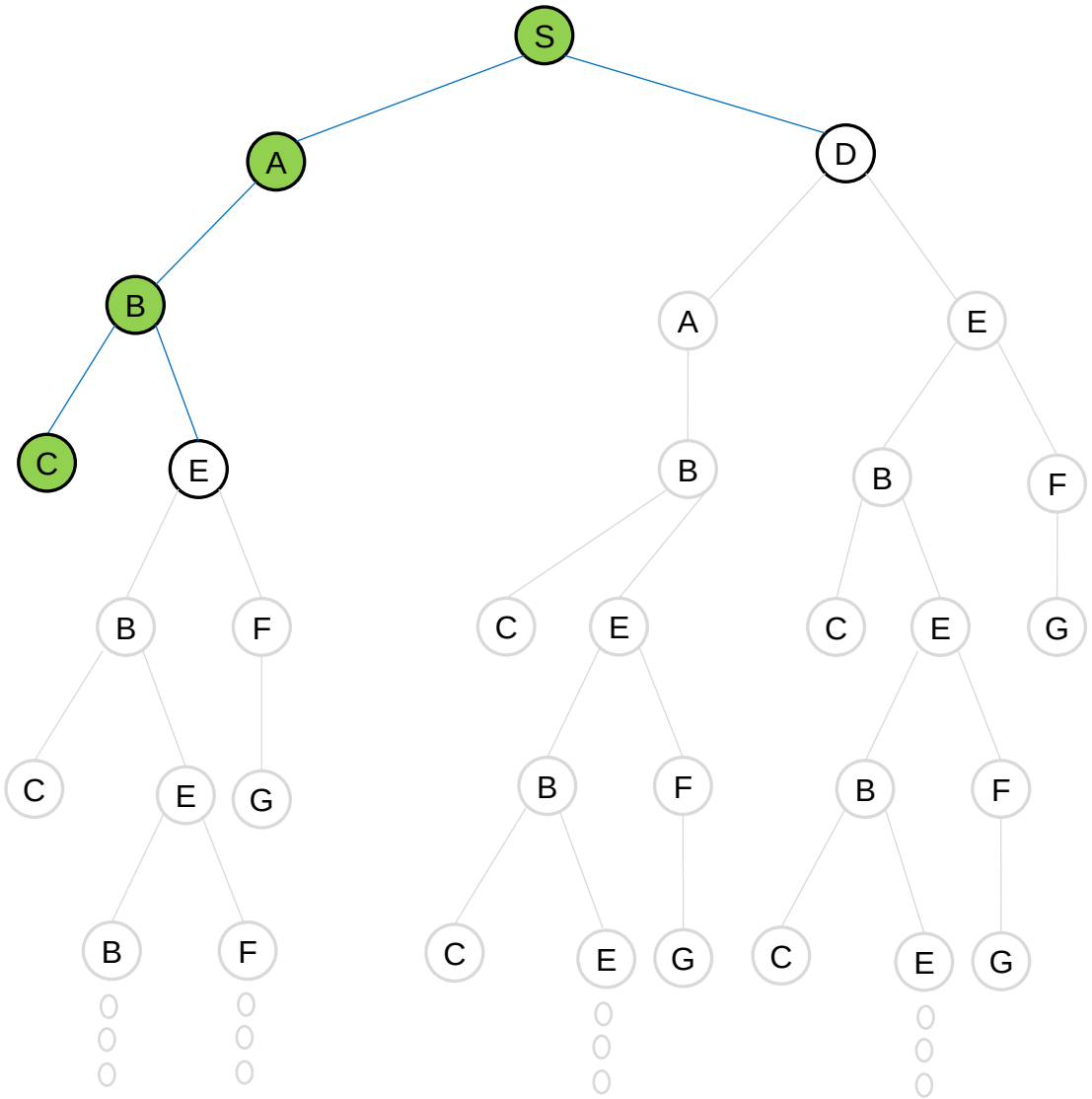
[**S**]
[**A**,D]
[**B**,D]
[**C**,E,D]



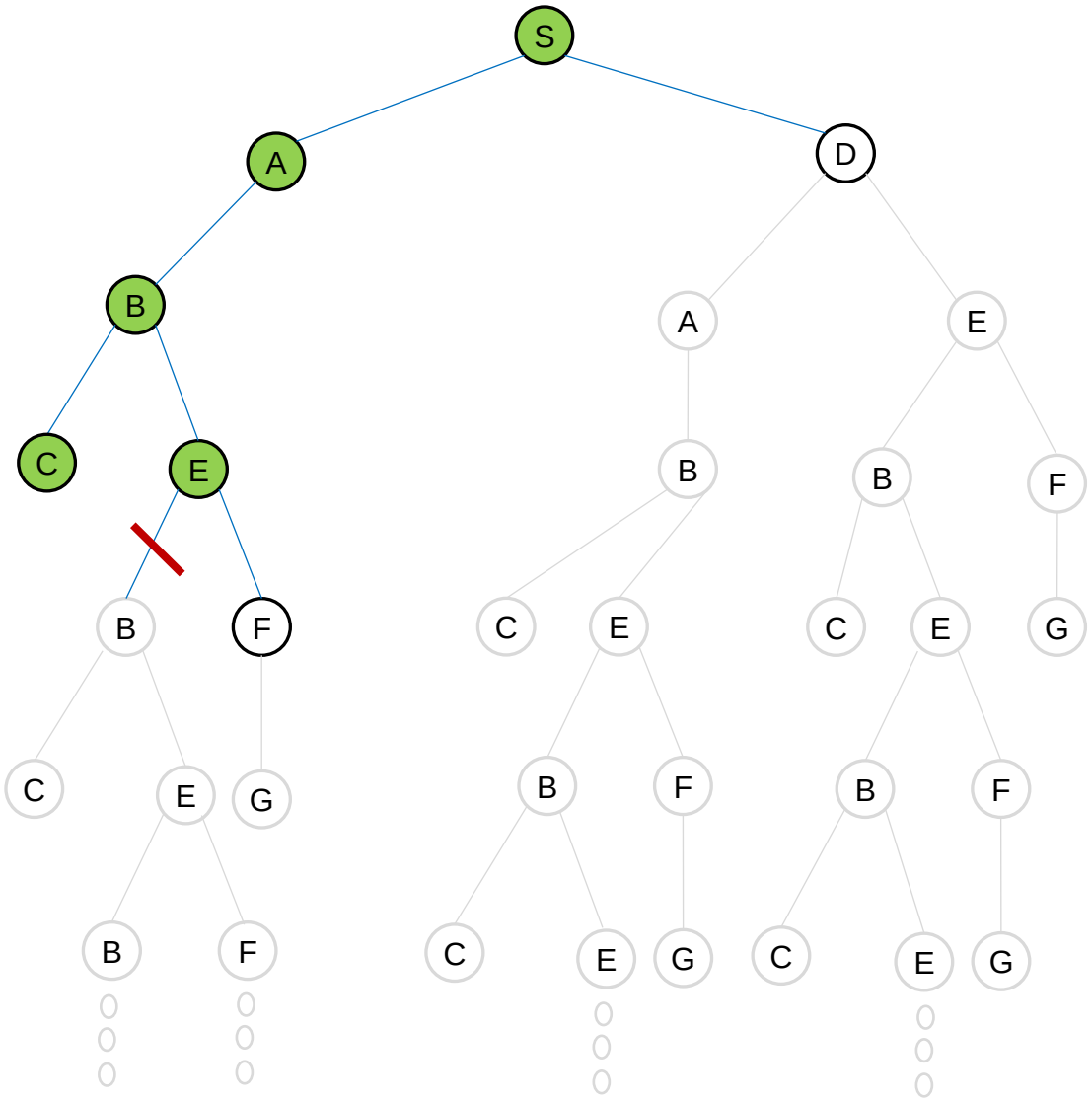
EXPLORED SET: S, A, B, C

FRONTIER (LIFO)

[**S**]
[**A**,D]
[**B**,D]
[**C**,E,D]
[**E**, D]



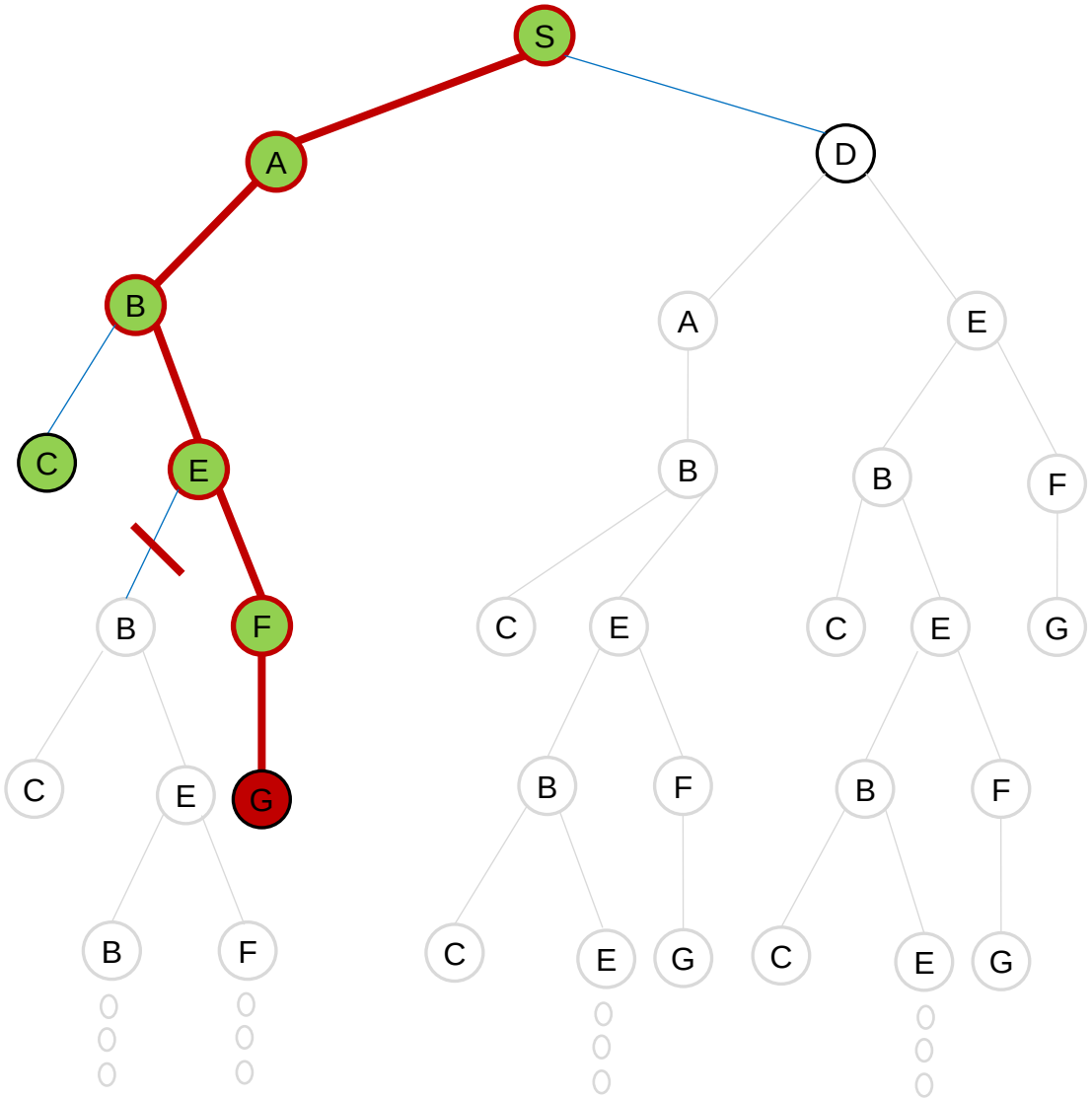
EXPLORED SET: S, A, B, C, E



FRONTIER (LIFO)

[**S**]
[**A**,D]
[**B**,D]
[**C**,E,D]
[**E**, D]
[**F**, D]

EXPLORED SET: S, A, B, C, E, F



FRONTIER (LIFO)

[**S**]
[**A**,D]
[**B**,D]
[**C**,E,D]
[**E**, D]
[**F**, D]
[D]

DFS={S,A,B,E,F,G}

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Depth-first search algorithm's performance?

- The tree-search version is not complete. On the other hand, the graph-search version, which avoids repeated states and redundant paths, is complete in finite state space. In infinite state spaces, both versions are not complete.
- Both versions are non-optimal.
- A depth-first tree search may generate all of the $O(b^m)$ nodes in the search tree, where m is the maximum depth of any node. Note that m itself can be much larger than d (the depth of the shallowest solution) and is infinite if the tree is unbounded.
- The depth-first search requires a storage of only $O(bm)$ nodes.

4. Depth-Limited Search

Depth-limited search limits the search depth as l. Thus it prevents falling into the trap for the case of infinite loops or the infinite state space.

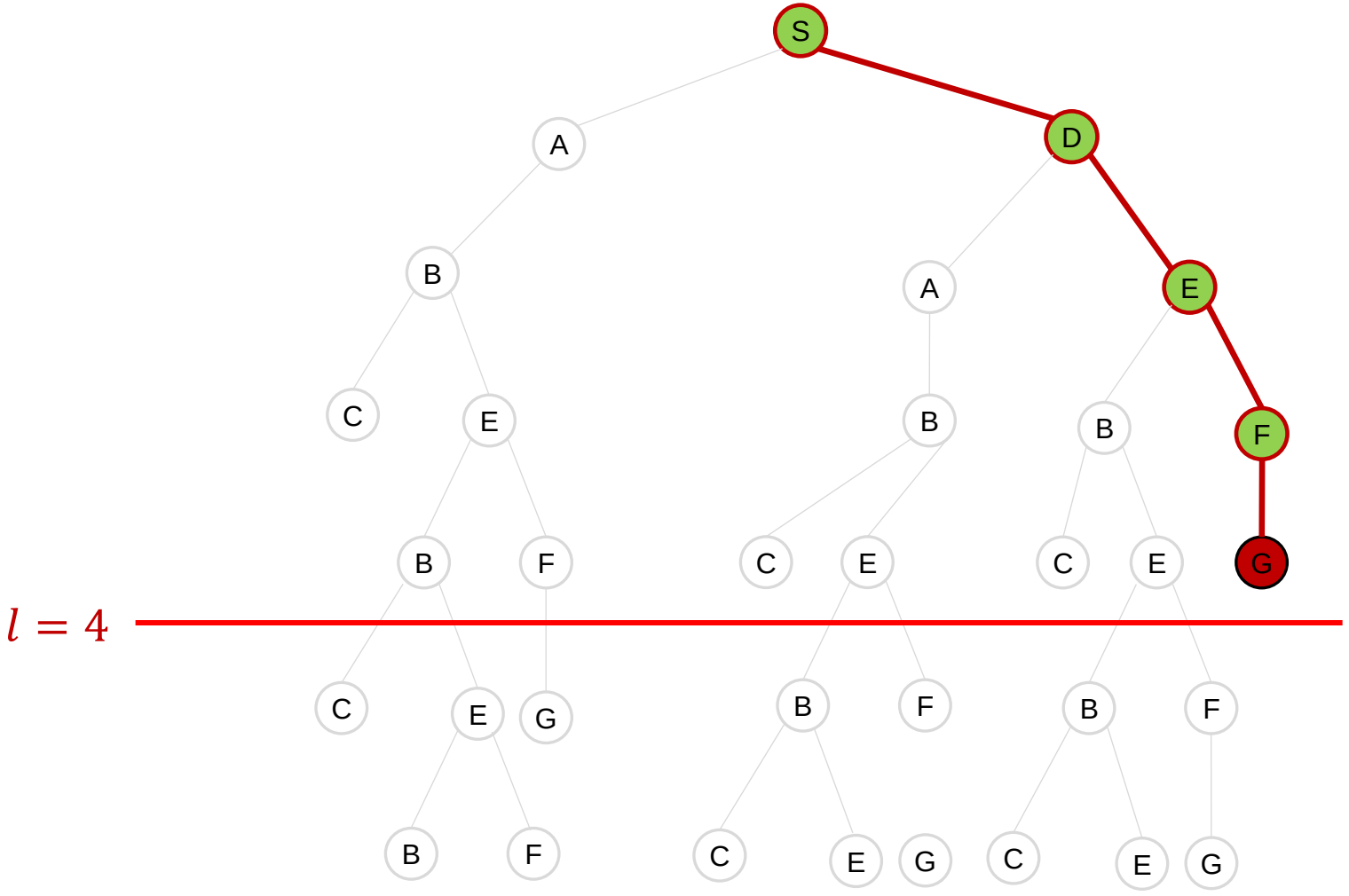
The depth limit solves the infinite-path problem. Unfortunately, it also introduces an additional source of incompleteness if we choose $l < d$.

Depth-limited search is generally non-optimal.

Its time complexity is $O(b^l)$ and its space complexity is $O(bl)$.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Depth-limited search can be implemented as a simple modification to the general tree or graph-search algorithm. Alternatively, it can be implemented as a simple recursive algorithm as shown below.

```
function DEPTH-LIMITED-SEARCH (problem, limit) returns a solution, or failure/cutoff
  - return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE()), problem, limit )

function RECURSIVE-DLS(node, problem, limit ) returns a solution, or failure/cutoff
   - if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
     else if limit = 0 then return cutoff
     else
          - cutoff occurred?←false
          - for each action in problem.ACTIONS(node.STATE) do
              - child ←CHILD-NODE(problem, node, action)
              - result ←RECURSIVE-DLS(child , problem, limit − 1)
              - if result = cutoff then cutoff occurred?←true
                else if result = failure then return result
         - if cutoff occurred? then return cutoff else return failure
```

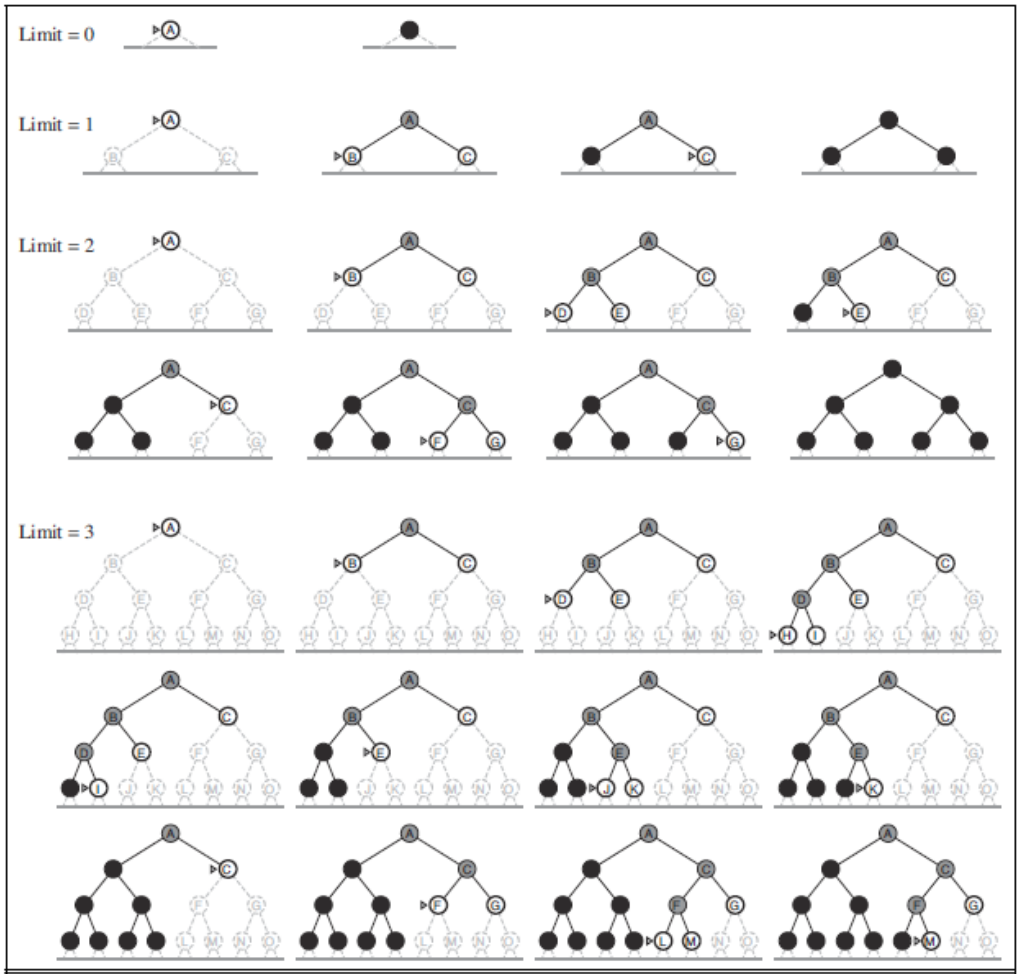
5. Iterative Deepening Depth-First Search

- Iterative deepening depth-first search apply depth-limited search iteratively. It gradually increases the limit—first 0, then 1, then 2, and so on—until a goal is found. This will occur when the depth limit reaches d, the depth of the shallowest goal node.

- Iterative deepening combines the benefits of the depth-first search and the breadth-first search.
  - Like the depth-first search, its memory requirements are modest: $O(bd)$
  - Like breadth-first search, it is complete when the branching factor b is finite; and optimal when the path cost is a non-decreasing function of the depth of the node.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Four iterations of iterative deepening search on a binary tree.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The algorithm for the iterative deepening depth-first search is given below.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  - for depth = 0 to ∞ do
      - result ←DEPTH-LIMITED-SEARCH(problem, depth)
      - if result≠ cutoff then return result
```

The iterative deepening search algorithm repeatedly applies the depth-limited search with increasing limits. It terminates when a solution is found or if the depth-limited search returns failure (no solution).

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The iterative deepening depth-first search may seem wasteful because states are generated multiple times. But, It is not too costly. The total number of nodes generated in the worst case is:

$$N(IDS) = (d)b + (d-1)b^2 + (d-2)b^3 + \cdots + (1)b^d$$

which gives a time complexity of $O(b^d)$,asymptotically the same as breadth-first search.

Ex: if $b = 10$ and $d = 5$, the numbers are

$N(IDS)$ = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450
$N(BFS)$ = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110

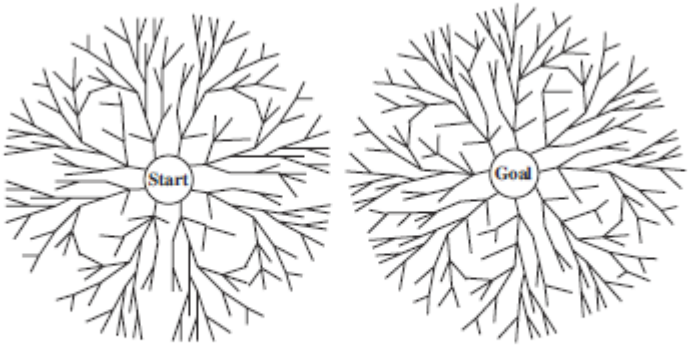
In general, iterative deepening is the preferred uninformed search method when the search space is large and the depth of the solution is not known.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

6. Bidirectional search

The idea behind the bidirectional search is to run two simultaneous searches one forward from the initial state and the other backward from the goal—hoping that the two searches meet in the middle. The motivation is that $b^{d/2} + b^{d/2}$ is much less than b^d, or in the figure, the area of the two small circles is less than the area of one big circle centered on the start and reaching to the goal.



A schematic view of a bidirectional search that is about to succeed when a branch from the start node meets a branch from the goal node.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The bidirectional search is implemented by replacing the goal test with a check to see whether the frontiers of the two searches intersect; if they do, a solution has been found. The check can be done when each node is generated or selected for expansion and, with a hash table, will take constant time.

- The bidirectional search is complete.
- The first such solution found is not generally optimal. However, it is optimal when step costs are identical and both directions use the breadth-first algorithm.
- The time and space complexity of bidirectional search using breadth-first searches in both directions is $O\left(b^{d/2}\right)$.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

The reduction in time complexity makes bidirectional search attractive, but how do we search backward?

For the 8-puzzle and finding a route in Romania, there is just one goal state, and state space is reversible. So, the backward search is very much like the forward search.

For the n-queens problem ???

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Comparing Uninformed Search Strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]

Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: [a] complete if b is finite; [b] complete if step costs $\geq \epsilon$ for positive ϵ; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search.

This comparison is for tree-search versions. For graph searches, the main differences are that depth-first search is complete for finite state spaces and that the branching factor *b* is finite.

Solving Problems by Searching - 2

INTRODUCTION TO ARTIFICIAL INTELLIGENT

Main Reference

Stuart Russell, Peter Norvig; “Artificial Intelligence A Modern Approach”, Pearson, 3rd Edition, 2013.